

## Big Data Engineering

# PySpark Practice Guide

Easy · Medium

*32 pipeline-building problems across two difficulty tracks*

|                  |                                                                |
|------------------|----------------------------------------------------------------|
| Document         | PySpark Practice Guide – Learner Edition                       |
| Tracks           | 2 parts · 32 questions                                         |
| Difficulty split | Easy (16) · Medium (16)                                        |
| Core skills      | DataFrame API · schemas · joins · window functions · Spark SQL |
| Audience         | Big Data / Data Engineering learners                           |
| Prepared by      | iamneo – Learning & Development                                |
| Version          | 1.0                                                            |

# How to Use This Guide

Every question in this guide is a small ETL pipeline rather than a single query. You are given a raw dataset, a set of functions to implement in `solution.py`, and the exact output each function must return. The work is always the same shape: clean the input, derive new columns, classify rows against business thresholds, then aggregate.

## STRUCTURE OF EVERY QUESTION

| Block                 | What it gives you                                                          |
|-----------------------|----------------------------------------------------------------------------|
| Problem Statement     | The business context – who needs the pipeline and why.                     |
| Objective             | The pipeline's responsibilities, stated as a checklist.                    |
| File Structure        | Where your code goes. Implementations belong in <code>solution.py</code> . |
| Dataset Description   | Column names, data types and meaning for each input file.                  |
| Sample Data           | A handful of raw rows, including the invalid ones you must drop.           |
| Task / Functions      | Each function with its Purpose, Parameters, Logic and Returns.             |
| Sample Input / Output | A worked example per function – your unit test in miniature.               |

## HOW TO WORK THROUGH IT

- Read the Objective first. It is the pipeline in five lines, and every function below maps to one of those lines.
- Implement the functions in order. Each one consumes the DataFrame the previous one returned, so a wrong cleaning step silently corrupts everything downstream.
- Check the Sample Input against the Sample Output before writing code. The rows that disappear between them tell you the filter conditions more precisely than the prose does.
- Match column names and ordering exactly. These are graded by automated tests, so a correctly computed column under the wrong name still fails.
- Watch the rounding. Where an average or a monetary figure is specified to a number of decimal places, that rounding is part of the answer.

## EASY VERSUS MEDIUM

The Easy track works on a single dataset and stays inside the DataFrame API: `filter`, `withColumn`, `when/otherwise`, `groupBy`. The Medium track adds explicitly defined schemas, joins against a reference dataset, window functions for ranking and running comparisons, and Spark SQL for the final summary. If a Medium question feels overwhelming, find the equivalent Easy question first — the cleaning and classification stages are deliberately parallel.

# Contents

**PART 1**

# PySpark — Easy

DataFrame cleaning, derived columns, conditional classification and single-dataset aggregation

**In this track — 16 questions**

|     |                                                                             |
|-----|-----------------------------------------------------------------------------|
| Q1  | Pharmacy Medicine Stock & Expiry Monitor                                    |
| Q2  | Electronics Warranty & Stock Monitor                                        |
| Q3  | Hotel Guest Stay & Service Usage Analysis                                   |
| Q4  | Online Learning Course Activity Analysis                                    |
| Q5  | E-Commerce Orders Enrichment & Return Risk Summary                          |
| Q6  | Hospital Appointment Enrichment & Cancellation Risk Analysis                |
| Q7  | Food Delivery Order Performance & Customer Insights                         |
| Q8  | Online Shopping Sales & Customer Segment Analysis                           |
| Q9  | Food Delivery Revenue, Discount, and Weather Analysis                       |
| Q10 | Online Shopping Revenue, Cashback, and Delivery Condition Analysis          |
| Q11 | Online Food Delivery Customer Engagement and Inactive Customer Flagging     |
| Q12 | Online Music Streaming Listener Engagement and Inactive Subscriber Analysis |
| Q13 | Weather Monitoring Data Cleaning and City Temperature Analysis              |
| Q14 | E-Commerce Order Value Monitor                                              |
| Q15 | Food Delivery Service Quality Analyzer                                      |
| Q16 | Food Delivery Order Activity and Frequent Customer Flagging                 |

## Q1. Pharmacy Medicine Stock & Expiry Monitor

Part 1 · PySpark — Easy — question 1 of 16

### PROBLEM STATEMENT

A pharmacy chain manages medicine stock across several branches located in different parts of the city. The pharmacy operations team periodically checks the available medicines to ensure that expired medicines are removed and medicines approaching their expiry date are identified.

Each stock record contains the branch identifier, medicine identifier, medicine type, inspection date, expiry date, and the number of units currently available.

The pharmacy wants an automated PySpark pipeline that processes the raw stock records. The pipeline should remove invalid records, determine the remaining shelf life of each medicine, categorize medicines based on their expiry condition, and generate a medicine-type-wise summary.

### OBJECTIVE

The data processing pipeline should:

- Remove records where the branch ID or medicine ID is missing.
- Remove records having a negative stock quantity.
- Calculate the number of days between the inspection date and medicine expiry date.
- Classify medicines as Expired, Near-Expiry, or Safe.
- Generate a summary showing the number of medicines in each medicine type and expiry category.

### FILE STRUCTURE

```
Project
├── data
│   └── medicines.csv
├── main.py
├── solution.py
├── tests.py
└── requirements.txt
```

### PURPOSE OF FILES

- main.py – Driver program that loads the CSV and invokes the functions.
- solution.py – Contains four functions that must be implemented.
- tests.py – Contains test cases for validating the implementation.
- requirements.txt – Contains required Python dependencies.
- data/medicines.csv – Raw pharmacy stock dataset.

### DATASET DESCRIPTION

File: medicines.csv

| Column Name     | Data Type | Description                                  |
|-----------------|-----------|----------------------------------------------|
| branch_id       | integer   | Unique identifier of the pharmacy branch     |
| medicine_id     | integer   | Unique identifier of the medicine            |
| medicine_type   | string    | Type of medicine                             |
| check_date      | date      | Date on which the stock was inspected        |
| expiry_date     | date      | Expiry date of the medicine                  |
| units_available | integer   | Number of medicine units currently available |

## POSSIBLE MEDICINE TYPES

- Tablet
- Syrup
- Capsule
- Injection

## SAMPLE DATA

| branch_id | medicine_id | medicine_type | check_date | expiry_date | units_available |
|-----------|-------------|---------------|------------|-------------|-----------------|
| 201       | 7001        | Tablet        | 2026-01-10 | 2026-01-06  | 25              |
| 201       | 7002        | Syrup         | 2026-01-10 | 2026-01-15  | 40              |
| 202       | 7010        | Capsule       | 2026-01-10 | 2026-02-20  | 75              |
| 202       | 7020        | Injection     | 2026-01-10 | 2026-01-10  | 15              |
| 203       | 7030        | Tablet        | 2026-01-10 | 2026-01-14  | 30              |

## TASK

Implement the following functions in solution.py.

### Function 1 — clean\_medicine\_stock(df\_medicine)

#### PURPOSE

Remove invalid medicine stock records before further processing.

#### PARAMETER

- df\_medicine

A PySpark DataFrame containing:

- branch\_id
- medicine\_id
- medicine\_type
- check\_date
- expiry\_date
- units\_available

#### LOGIC

Retain only records where:

- branch\_id is not null.
- medicine\_id is not null.
- units\_available >= 0.

#### RETURNS

A cleaned DataFrame containing the same columns as the input.

#### EXAMPLE

##### INPUT

| branch_id | medicine_id | units_available |
|-----------|-------------|-----------------|
| 201       | 7001        | 25              |
| null      | 7002        | 40              |
| 202       | 7010        | -5              |

## OUTPUT

| branch_id | medicine_id | units_available |
|-----------|-------------|-----------------|
| 201       | 7001        | 25              |

### Function 2 — calculate\_medicine\_shelf\_life(df\_clean)

#### PURPOSE

Determine how many days are left before each medicine expires.

#### LOGIC

Create a new column:

- days\_until\_expiry

using:

- expiry\_date - check\_date

#### RETURNS

A DataFrame containing all original columns plus:

- days\_until\_expiry
- Sample

#### INPUT

| medicine_id | check_date | expiry_date |
|-------------|------------|-------------|
| 7001        | 2026-01-10 | 2026-01-06  |
| 7002        | 2026-01-10 | 2026-01-15  |
| 7010        | 2026-01-10 | 2026-02-20  |

## OUTPUT

| medicine_id | days_until_expiry |
|-------------|-------------------|
| 7001        | -4                |
| 7002        | 5                 |
| 7010        | 41                |

### Function 3 — assign\_expiry\_category(df\_shelf)

#### PURPOSE

Categorize medicines according to their remaining shelf life.

#### RULES

| Condition                       | expiry_category |
|---------------------------------|-----------------|
| days_until_expiry < 0           | Expired         |
| days_until_expiry >= 0 and <= 7 | Near-Expiry     |
| days_until_expiry > 7           | Safe            |

#### RETURNS

All existing columns plus:

- expiry\_category

## EXAMPLE

| medicine_id | days_until_expiry | expiry_category |
|-------------|-------------------|-----------------|
| 7001        | -4                | Expired         |
| 7002        | 5                 | Near-Expiry     |
| 7010        | 41                | Safe            |

## Function 4 — `medicine_expiry_summary(df_category)`

### PURPOSE

Create a summary of medicine counts by medicine type and expiry category.

### LOGIC

Group records using:

- `medicine_type`
- `expiry_category`
- and count the number of records in each group.

### RETURNS

A DataFrame containing:

- `medicine_type`
- `expiry_category`
- `count`

### SAMPLE OUTPUT

| medicine_type | expiry_category | count |
|---------------|-----------------|-------|
| Capsule       | Safe            | 1     |
| Injection     | Near-Expiry     | 1     |
| Syrup         | Near-Expiry     | 1     |
| Tablet        | Expired         | 1     |
| Tablet        | Near-Expiry     | 1     |

## Q2. Electronics Warranty & Stock Monitor

Part 1 · PySpark — Easy — question 2 of 16

### PROBLEM STATEMENT

A consumer electronics distributor maintains inventory records for products stored at multiple warehouses. The operations team performs periodic inventory checks to monitor product availability and warranty validity.

Each record contains warehouse information, product information, inspection date, warranty end date, product category, and the number of units currently available.

The company needs a PySpark-based processing pipeline to clean incorrect inventory records, calculate the remaining warranty period, classify products according to warranty status, determine their stock availability level, and produce summary reports for operational analysis.

### OBJECTIVE

The pipeline should:

- Remove inventory records with missing warehouse or product identifiers.
- Remove records having negative stock quantities.
- Calculate the number of days remaining until the warranty expires.
- Classify products as Warranty-Expired, Warranty-Ending, or Warranty-Active.
- Classify stock availability as Out-of-Stock, Low-Stock, or Available.
- Generate a category-wise warranty status summary.
- Generate an additional category-wise stock availability summary.

### FILE STRUCTURE

```
Project
├── data
│   └── electronics_inventory.csv
├── main.py
├── solution.py
├── tests.py
└── requirements.txt
```

### DATASET DESCRIPTION

File: electronics\_inventory.csv

| Column Name       | Data Type | Description                                 |
|-------------------|-----------|---------------------------------------------|
| warehouse_id      | integer   | Unique identifier of the warehouse          |
| product_code      | integer   | Unique identifier of the electronic product |
| product_category  | string    | Category of the electronic product          |
| inspection_date   | date      | Date on which inventory was inspected       |
| warranty_end_date | date      | Date on which product warranty ends         |
| stock_units       | integer   | Number of units currently available         |

Possible Product Categories

Laptop

Mobile

Monitor

## Tablet

## SAMPLE INPUT

| warehouse_id | product_code | product_category | inspection_date | warranty_end_date | stock_units |
|--------------|--------------|------------------|-----------------|-------------------|-------------|
| 501          | 9001         | Laptop           | 2026-03-10      | 2026-03-05        | 0           |
| 501          | 9002         | Mobile           | 2026-03-10      | 2026-03-14        | 12          |
| 502          | 9010         | Monitor          | 2026-03-10      | 2026-04-25        | 35          |
| 502          | 9020         | Tablet           | 2026-03-10      | 2026-03-10        | 5           |
| 503          | 9030         | Laptop           | 2026-03-10      | 2026-03-13        | 25          |

## TASK

Implement the following functions in solution.py.

## Function 1 — clean\_electronics\_inventory(df\_inventory)

## PURPOSE

Remove invalid inventory records.

## LOGIC

Keep a record only when:

- warehouse\_id is not null.
- product\_code is not null.
- stock\_units >= 0.

## RETURNS

The cleaned DataFrame with the same columns as the input.

Sample

## INPUT

| warehouse_id | product_code | stock_units |
|--------------|--------------|-------------|
| 501          | 9001         | 0           |
| null         | 9002         | 12          |
| 502          | null         | 25          |
| 503          | 9030         | -10         |

## OUTPUT

| warehouse_id | product_code | stock_units |
|--------------|--------------|-------------|
| 501          | 9001         | 0           |

## Function 2 — calculate\_warranty\_days(df\_clean)

## PURPOSE

Calculate the remaining warranty period for each product.

## LOGIC

Add:

- warranty\_days\_left

using:

- warranty\_end\_date - inspection\_date

## RETURNS

All existing columns plus:

- warranty\_days\_left

## SAMPLE OUTPUT

| product_code | inspection_date | warranty_end_date | warranty_days_left |
|--------------|-----------------|-------------------|--------------------|
| 9001         | 2026-03-10      | 2026-03-05        | -5                 |
| 9002         | 2026-03-10      | 2026-03-14        | 4                  |
| 9010         | 2026-03-10      | 2026-04-25        | 46                 |
| 9020         | 2026-03-10      | 2026-03-10        | 0                  |
| 9030         | 2026-03-10      | 2026-03-13        | 3                  |

## Function 3 — classify\_warranty\_status(df\_warranty)

### PURPOSE

Classify each product according to the remaining warranty period.

### RULES

| Condition                        | warranty_status  |
|----------------------------------|------------------|
| warranty_days_left < 0           | Warranty-Expired |
| warranty_days_left >= 0 and <= 7 | Warranty-Ending  |
| warranty_days_left > 7           | Warranty-Active  |

## RETURNS

All existing columns plus:

- warranty\_status
- Sample

| product_code | warranty_days_left | warranty_status  |
|--------------|--------------------|------------------|
| 9001         | -5                 | Warranty-Expired |
| 9002         | 4                  | Warranty-Ending  |
| 9010         | 46                 | Warranty-Active  |
| 9020         | 0                  | Warranty-Ending  |
| 9030         | 3                  | Warranty-Ending  |

## Function 4 — category\_warranty\_summary(df\_status)

### PURPOSE

Generate a category-wise summary of warranty conditions.

### LOGIC

Group records by:

- product\_category
- warranty\_status
- and count the number of products in each group.

## RETURNS

product\_category

warranty\_status

count

## SAMPLE OUTPUT

| product_category | warranty_status  | count |
|------------------|------------------|-------|
| Laptop           | Warranty-Expired | 1     |
| Laptop           | Warranty-Ending  | 1     |
| Mobile           | Warranty-Ending  | 1     |
| Monitor          | Warranty-Active  | 1     |
| Tablet           | Warranty-Ending  | 1     |

## Function 5 — classify\_stock\_level(df\_status)

### PURPOSE

In addition to warranty monitoring, the distributor wants to know whether each product has sufficient stock.

Add a new column:

- stock\_level

### RULES

| stock_units | stock_level  |
|-------------|--------------|
| 0           | Out-of-Stock |
| 1 to 20     | Low-Stock    |
| > 20        | Available    |

## RETURNS

The DataFrame containing all existing columns plus:

- stock\_level

## SAMPLE OUTPUT

| product_code | stock_units | stock_level  |
|--------------|-------------|--------------|
| 9001         | 0           | Out-of-Stock |
| 9002         | 12          | Low-Stock    |
| 9010         | 35          | Available    |
| 9020         | 5           | Low-Stock    |
| 9030         | 25          | Available    |

## Q3. Hotel Guest Stay & Service Usage Analysis

Part 1 · PySpark — Easy — question 3 of 16

### PROBLEM STATEMENT

A hotel group operates multiple properties across different cities. The management team collects daily guest service records to understand guest activity and evaluate the usage of different room plans.

Each service record contains the guest identifier, hotel identifier, date of service usage, type of service used, service duration in minutes, and the room package selected by the guest.

Due to manual data entry, some records contain missing guest identifiers or invalid service durations. The hotel analytics team requires an automated PySpark pipeline to clean the raw service records, calculate the total service duration for each guest per service date, classify guest activity into different engagement levels, and generate a summary showing activity across room packages.

### OBJECTIVE

The pipeline should:

- Remove records with missing guest identifiers.
- Remove records with invalid service durations.
- Aggregate the total service duration for each guest per service date while retaining the room package.
- Classify each aggregated record as Low, Medium, or High based on total service duration.
- Generate a summary showing the count of records for each room package and activity level.

### FILE STRUCTURE

```
Project
├── data
│   └── hotel_services.csv
├── main.py
├── solution.py
├── tests.py
└── requirements.txt
```

### PURPOSE OF FILES

- main.py – Driver script that loads the data and invokes the solution functions.
- solution.py – Contains four functions to be implemented.
- tests.py – Contains test cases.
- requirements.txt – Required Python dependencies.
- data/hotel\_services.csv – Raw hotel service dataset.

### DATASET DESCRIPTION

File: hotel\_services.csv

| Column Name           | Data Type | Description                              |
|-----------------------|-----------|------------------------------------------|
| guest_id              | integer   | Unique identifier for each hotel guest   |
| hotel_id              | integer   | Unique identifier for each hotel         |
| service_date          | date      | Date on which the guest used the service |
| service_type          | string    | Type of service used                     |
| service_duration_mins | integer   | Duration of the service in minutes       |
| room_package          | string    | Package selected by the guest            |

### Possible Service Types

Spa

Gym

Pool

Lounge

Possible Room Packages

Standard

Deluxe

Suite

**SAMPLE DATA**

| guest_id | hotel_id | service_date | service_type | service_duration_mins | room_package |
|----------|----------|--------------|--------------|-----------------------|--------------|
| 401      | 11       | 2026-04-12   | Spa          | 35                    | Standard     |
| 401      | 11       | 2026-04-12   | Gym          | 25                    | Standard     |
| 403      | 12       | 2026-04-12   | Pool         | 50                    | Deluxe       |
| 406      | 11       | 2026-04-12   | Lounge       | 75                    | Suite        |
| 407      | 13       | 2026-04-13   | Spa          | 20                    | Standard     |

**TASK**

Implement the following functions in solution.py.

**Function 1 — clean\_hotel\_services(df\_services)****PURPOSE**

Remove invalid service records.

**PARAMETERS**

df\_services – Raw DataFrame containing:

- guest\_id
- hotel\_id
- service\_date
- service\_type
- service\_duration\_mins
- room\_package

**LOGIC**

Retain only records where:

- guest\_id is not null.
- service\_duration\_mins is greater than zero.

**RETURNS**

A cleaned DataFrame with the same columns as the input.

**SAMPLE INPUT**

| guest_id | hotel_id | service_duration_mins |
|----------|----------|-----------------------|
| 401      | 11       | 35                    |
| null     | 12       | 40                    |
| 403      | 12       | -15                   |

| guest_id | hotel_id | service_duration_mins |
|----------|----------|-----------------------|
| 407      | 13       | 20                    |

## SAMPLE OUTPUT

| guest_id | hotel_id | service_duration_mins |
|----------|----------|-----------------------|
| 401      | 11       | 35                    |
| 407      | 13       | 20                    |

## Function 2 — aggregate\_guest\_service(df\_clean)

### PURPOSE

Calculate the total service duration for every guest on each service date.

### LOGIC

Group records by:

- guest\_id
- service\_date
- room\_package

Then calculate the sum of:

- service\_duration\_mins

and store it as:

- total\_service\_duration

### RETURNS

A DataFrame containing:

- guest\_id
- service\_date
- room\_package
- total\_service\_duration

## SAMPLE INPUT

| guest_id | service_date | service_type | service_duration_mins | room_package |
|----------|--------------|--------------|-----------------------|--------------|
| 401      | 2026-04-12   | Spa          | 35                    | Standard     |
| 401      | 2026-04-12   | Gym          | 25                    | Standard     |
| 403      | 2026-04-12   | Pool         | 50                    | Deluxe       |

## SAMPLE OUTPUT

| guest_id | service_date | room_package | total_service_duration |
|----------|--------------|--------------|------------------------|
| 401      | 2026-04-12   | Standard     | 60                     |
| 403      | 2026-04-12   | Deluxe       | 50                     |

## Function 3 — classify\_guest\_activity(df\_daily)

### PURPOSE

Classify each guest's daily activity based on total service duration.

## RULES

| Condition                          | activity_level |
|------------------------------------|----------------|
| total_service_duration < 30        | Low            |
| 30 <= total_service_duration <= 60 | Medium         |
| total_service_duration > 60        | High           |

## RETURNS

All existing columns plus:

- activity\_level
- Sample

| guest_id | total_service_duration | activity_level |
|----------|------------------------|----------------|
| 401      | 60                     | Medium         |
| 403      | 50                     | Medium         |
| 406      | 75                     | High           |
| 407      | 20                     | Low            |

## Function 4 — package\_activity\_summary(df\_activity)

### PURPOSE

Generate a summary of guest activity across room packages.

### LOGIC

Group records by:

- room\_package
- activity\_level
- and count the records in each group.

### RETURNS

room\_package

activity\_level

count

### SAMPLE OUTPUT

| room_package | activity_level | count |
|--------------|----------------|-------|
| Deluxe       | Medium         | 1     |
| Standard     | Low            | 1     |
| Standard     | Medium         | 1     |
| Suite        | High           | 1     |

## Q4. Online Learning Course Activity Analysis

Part 1 · PySpark — Easy — question 4 of 16

### PROBLEM STATEMENT

An online learning platform records learner activity across multiple courses. The training team wants to understand how actively learners are using the platform and whether their daily learning activity indicates satisfactory progress.

Each activity record contains the learner identifier, course identifier, activity date, type of learning activity, duration spent on the platform, and the learner's subscription category.

Because activity data is collected from multiple sources, some records may contain missing learner identifiers or invalid activity durations.

The analytics team requires a PySpark pipeline to clean the raw activity data, calculate the total learning time for each learner per day, classify learner engagement, determine whether the learner has achieved the expected daily activity threshold, and generate a subscription-wise engagement summary.

### OBJECTIVE

The pipeline should:

- Remove records with missing learner identifiers.
- Remove records with zero or negative activity duration.
- Calculate the total learning duration for each learner per activity date.
- Retain the subscription category during aggregation.
- Classify learners as Low, Moderate, or High engagement.
- Determine whether the learner's daily activity is Below-Target or Target-Achieved.
- Generate a summary grouped by subscription category and engagement level.

### FILE STRUCTURE

```
Project
├── data
│   └── learner_activity.csv
├── main.py
├── solution.py
├── tests.py
└── requirements.txt
```

### PURPOSE OF FILES

- main.py – Loads the activity data and calls the solution functions.
- solution.py – Contains five functions to be implemented.
- tests.py – Contains test cases.
- requirements.txt – Required Python dependencies.
- data/learner\_activity.csv – Raw learner activity dataset.

### DATASET DESCRIPTION

File: learner\_activity.csv

| Column Name   | Data Type | Description                      |
|---------------|-----------|----------------------------------|
| learner_id    | integer   | Unique identifier of the learner |
| course_id     | integer   | Unique identifier of the course  |
| activity_date | date      | Date of learner activity         |
| activity_type | string    | Type of learning activity        |
| duration_mins | integer   | Duration of the activity         |

| Column Name       | Data Type | Description                          |
|-------------------|-----------|--------------------------------------|
| subscription_type | string    | Subscription category of the learner |

### Possible Activity Types

Video

Coding

Quiz

Reading

### Possible Subscription Types

Free

Standard

Pro

### SAMPLE DATA

| learner_id | course_id | activity_date | activity_type | duration_mins | subscription_type |
|------------|-----------|---------------|---------------|---------------|-------------------|
| 601        | 101       | 2026-05-10    | Video         | 30            | Standard          |
| 601        | 101       | 2026-05-10    | Coding        | 35            | Standard          |
| 603        | 102       | 2026-05-10    | Quiz          | 20            | Free              |
| 604        | 103       | 2026-05-10    | Coding        | 80            | Pro               |
| 605        | 101       | 2026-05-11    | Reading       | 45            | Standard          |

### TASK

Implement the following functions in solution.py.

#### Function 1 — clean\_learner\_activity(df\_activity)

#### PURPOSE

Remove invalid learning activity records.

#### LOGIC

Keep only records where:

- learner\_id is not null.
- duration\_mins is greater than zero.

#### RETURNS

Cleaned DataFrame containing the same columns as the input.

#### SAMPLE INPUT

| learner_id | course_id | duration_mins |
|------------|-----------|---------------|
| 601        | 101       | 30            |
| null       | 101       | 40            |
| 603        | 102       | -10           |
| 604        | 103       | 80            |

#### SAMPLE OUTPUT

| learner_id | course_id | duration_mins |
|------------|-----------|---------------|
| 601        | 101       | 30            |
| 604        | 103       | 80            |

## Function 2 — aggregate\_daily\_learning(df\_clean)

### PURPOSE

Calculate the total learning duration for every learner on each activity date.

### LOGIC

Group records by:

- learner\_id
- activity\_date
- subscription\_type

Then calculate the sum of:

- duration\_mins

and store the result as:

- total\_learning\_mins

### RETURNS

learner\_id

activity\_date

subscription\_type

total\_learning\_mins

### SAMPLE INPUT

| learner_id | activity_date | activity_type | duration_mins | subscription_type |
|------------|---------------|---------------|---------------|-------------------|
| 601        | 2026-05-10    | Video         | 30            | Standard          |
| 601        | 2026-05-10    | Coding        | 35            | Standard          |
| 603        | 2026-05-10    | Quiz          | 20            | Free              |

### SAMPLE OUTPUT

| learner_id | activity_date | subscription_type | total_learning_mins |
|------------|---------------|-------------------|---------------------|
| 601        | 2026-05-10    | Standard          | 65                  |
| 603        | 2026-05-10    | Free              | 20                  |

## Function 3 — classify\_learning\_engagement(df\_daily)

### PURPOSE

Classify the learner's daily engagement based on total learning time.

### RULES

| Condition                       | engagement_level |
|---------------------------------|------------------|
| total_learning_mins < 30        | Low              |
| 30 <= total_learning_mins <= 60 | Moderate         |
| total_learning_mins > 60        | High             |

## RETURNS

All existing columns plus:

- engagement\_level
- Sample

| learner_id | total_learning_mins | engagement_level |
|------------|---------------------|------------------|
| 603        | 20                  | Low              |
| 605        | 45                  | Moderate         |
| 601        | 65                  | High             |
| 604        | 80                  | High             |

## Function 4 — classify\_daily\_target(df\_engagement)

### PURPOSE

Determine whether the learner achieved the expected daily learning target.

The platform considers 45 minutes as the minimum expected daily learning duration.

### RULES

| Condition                 | target_status   |
|---------------------------|-----------------|
| total_learning_mins < 45  | Below-Target    |
| total_learning_mins >= 45 | Target-Achieved |

## RETURNS

The DataFrame with all existing columns plus:

- target\_status

### SAMPLE OUTPUT

| learner_id | total_learning_mins | target_status   |
|------------|---------------------|-----------------|
| 603        | 20                  | Below-Target    |
| 605        | 45                  | Target-Achieved |
| 601        | 65                  | Target-Achieved |
| 604        | 80                  | Target-Achieved |

## Function 5 — subscription\_engagement\_summary(df\_final)

### PURPOSE

Generate a summary of learner engagement for each subscription category.

### LOGIC

Group records by:

- subscription\_type
- engagement\_level
- and count the number of records in each group.

## RETURNS

subscription\_type

engagement\_level

count

#### SAMPLE INPUT

| learner_id | subscription_type | engagement_level | target_status   |
|------------|-------------------|------------------|-----------------|
| 601        | Standard          | High             | Target-Achieved |
| 603        | Free              | Low              | Below-Target    |
| 604        | Pro               | High             | Target-Achieved |
| 605        | Standard          | Moderate         | Target-Achieved |

#### SAMPLE OUTPUT

| subscription_type | engagement_level | count |
|-------------------|------------------|-------|
| Free              | Low              | 1     |
| Pro               | High             | 1     |
| Standard          | High             | 1     |
| Standard          | Moderate         | 1     |

## Q5. E-Commerce Orders Enrichment & Return Risk Summary

Part 1 · PySpark — Easy — question 5 of 16

### E-Commerce Orders Enrichment & Return Risk Summary

#### PROBLEM STATEMENT

An online retail company receives thousands of customer orders every month from different regions. Each order contains information such as the order ID, customer ID, order category, order amount, and a return indicator generated from the company's internal processing system.

Separately, customer profile information such as the customer name and geographical zone is maintained in another system. Some older customer records have incomplete information, particularly missing zone details, which must be handled during data preparation.

The analytics team wants to build a PySpark-based data engineering pipeline that can:

- Load and clean both datasets.
- Combine order information with customer profile details.
- Generate an enriched order dataset.
- Summarize return activity by order category and geographical zone.
- Identify segments with higher return rates.

#### OBJECTIVE

The pipeline should:

- Load and clean both datasets: orders.csv and customers.csv.
- Replace missing values in order\_amount and is\_returned.
- Replace missing customer zones with "Unknown".
- Perform a left join using customer\_id.
- Produce an enriched order dataset with selected columns in the required order.
- Calculate total orders, returned orders, return rate, and average order amount.
- Group the results by order\_category and zone.
- Sort the final summary by descending return\_rate and then descending total\_orders.

#### FILE STRUCTURE

```
Project
├── data
│   ├── orders.csv
│   └── customers.csv
├── main.py
├── solution.py
├── tests.py
└── requirements.txt
```

#### MODULES

- main.py
  - Contains the driver code and invokes the pipeline functions.
- solution.py
  - Contains the functions that must be implemented.

#### DATASET DESCRIPTION

File: orders.csv

| Column Name | Type   | Description                      |
|-------------|--------|----------------------------------|
| order_id    | string | Unique identifier for each order |

| Column Name    | Type    | Description                                                       |
|----------------|---------|-------------------------------------------------------------------|
| customer_id    | string  | Identifier linking an order to a customer                         |
| order_date     | date    | Date when the order was placed                                    |
| order_category | string  | Category such as Electronics, Fashion, or Grocery                 |
| order_amount   | integer | Total amount of the order; missing values must be replaced with 0 |
| is_returned    | integer | Return indicator, where 1 means returned and 0 means not returned |

File: customers.csv

| Column Name   | Type   | Description                                                       |
|---------------|--------|-------------------------------------------------------------------|
| customer_id   | string | Unique customer identifier                                        |
| profile_id    | string | Customer profile identifier                                       |
| customer_name | string | Name of the customer                                              |
| zone          | string | Geographical zone; missing values must be replaced with "Unknown" |

## SAMPLE DATA

orders.csv

| order_id | customer_id | order_date | order_category | order_amount | is_returned |
|----------|-------------|------------|----------------|--------------|-------------|
| 0101     | C101        | 2026-02-01 | Electronics    | 45000        | 0           |
| 0102     | C102        | 2026-02-02 | Fashion        | 7000         | 1           |
| 0103     | C103        | 2026-02-03 | Grocery        | 3200         | 0           |
| 0104     | C104        | 2026-02-04 | Electronics    | 28000        | 1           |
| 0105     | C105        | 2026-02-05 | Fashion        | 9000         | 0           |

customers.csv

| customer_id | profile_id | customer_name | zone    |
|-------------|------------|---------------|---------|
| C101        | PR101      | Rohan Kapoor  | North   |
| C102        | PR102      | Meera Joshi   | West    |
| C103        | PR103      | Aditya Rao    | South   |
| C104        | PR104      | Kavya Singh   | East    |
| C105        | PR105      | Nitin Shah    | Unknown |

## IMPLEMENTATION REQUIREMENTS

### FUNCTIONS TO IMPLEMENT IN SOLUTION.PY

#### Function 1 — load\_orders(spark, orders\_csv\_path: str)

##### PURPOSE

Load the orders dataset and replace missing values in order\_amount and is\_returned.

##### INPUTS

spark – Active SparkSession.

orders\_csv\_path – Path to the orders CSV file.

## RETURNS

A DataFrame with:

- order\_id
- customer\_id
- order\_date
- order\_category
- order\_amount
- is\_returned

## LOGIC

Read the CSV using an inferred schema.

Fill null order\_amount values with 0.

Fill null is\_returned values with 0.

### Function 2 — load\_customers(spark, customers\_csv\_path: str)

#### PURPOSE

Load customer information and handle missing geographical zones.

#### INPUTS

spark – Active SparkSession.

customers\_csv\_path – Path to the customers CSV file.

#### RETURNS

DataFrame with missing zone values replaced with:

- Unknown

#### LOGIC

Read CSV with inferred schema.

Replace null values in zone.

### Function 3 — join\_orders\_customers(orders\_df, customers\_df)

#### PURPOSE

Enrich order information with customer details.

#### INPUTS

orders\_df – Output of load\_orders.

customers\_df – Output of load\_customers.

#### LOGIC

Perform a left join using:

- customer\_id

Select only the following columns in this exact order:

- order\_id
- order\_date
- customer\_id
- profile\_id
- customer\_name

- zone
- order\_category
- order\_amount
- is\_returned

## RETURNS

An enriched DataFrame containing the selected columns.

### Function 4 — summarize\_return\_risk\_by\_segment(df\_enriched)

## PURPOSE

Generate return statistics by order category and geographical zone.

## LOGIC

Group by:

- order\_category
- zone

## CALCULATE

total\_orders

returned\_orders

return\_rate

avg\_order\_amount

## WHERE

```
return_rate = returned_orders / total_orders
```

and:

```
avg_order_amount = average(order_amount)
```

Sort

Sort the result by:

- return\_rate descending
- total\_orders descending

## RETURNS

A DataFrame containing:

- order\_category
- zone
- total\_orders
- returned\_orders
- return\_rate
- avg\_order\_amount

## SAMPLE I/O

```
load_orders(spark, orders_csv_path)
```

## SAMPLE INPUT

order\_id,customer\_id,order\_date,order\_category,order\_amount,is\_returned

O101,C101,2026-02-01,Electronics,45000,0

O102,C102,2026-02-02,Fashion,,1

O103,C103,2026-02-03,Grocery,3200,

O104,C104,2026-02-04,Electronics,28000,1

### SAMPLE OUTPUT

| order_id | customer_id | order_date | order_category | order_amount | is_returned |
|----------|-------------|------------|----------------|--------------|-------------|
| O101     | C101        | 2026-02-01 | Electronics    | 45000        | 0           |
| O102     | C102        | 2026-02-02 | Fashion        | 0            | 1           |
| O103     | C103        | 2026-02-03 | Grocery        | 3200         | 0           |
| O104     | C104        | 2026-02-04 | Electronics    | 28000        | 1           |

```
load_customers(spark, customers_csv_path)
```

### SAMPLE INPUT

customer\_id,profile\_id,customer\_name,zone

C101,PR101,Rohan Kapoor,North

C102,PR102,Meera Joshi,West

C103,PR103,Aditya Rao,

C104,PR104,Kavya Singh,East

### SAMPLE OUTPUT

| customer_id | profile_id | customer_name | zone    |
|-------------|------------|---------------|---------|
| C101        | PR101      | Rohan Kapoor  | North   |
| C102        | PR102      | Meera Joshi   | West    |
| C103        | PR103      | Aditya Rao    | Unknown |
| C104        | PR104      | Kavya Singh   | East    |

```
join_orders_customers(orders_df, customers_df)
```

### SAMPLE OUTPUT

| order_id | order_date | customer_id | profile_id | customer_name | zone  | order_category | order_amount | is_returned |
|----------|------------|-------------|------------|---------------|-------|----------------|--------------|-------------|
| O101     | 2026-02-01 | C101        | PR101      | Rohan Kapoor  | North | Electronics    | 45000        | 0           |
| O102     | 2026-02-02 | C102        | PR102      | Meera Joshi   | West  | Fashion        | 7000         | 1           |
| O103     | 2026-02-03 | C103        | PR103      | Aditya Rao    | South | Grocery        | 3200         | 0           |
| O104     | 2026-02-04 | C104        | PR104      | Kavya Singh   | East  | Electronics    | 28000        | 1           |

```
summarize_return_risk_by_segment(df_enriched)
```

### SAMPLE INPUT

| order_category | zone  | order_amount | is_returned |
|----------------|-------|--------------|-------------|
| Electronics    | North | 45000        | 0           |
| Electronics    | East  | 28000        | 1           |
| Fashion        | West  | 7000         | 1           |
| Fashion        | North | 9000         | 0           |

| order_category | zone  | order_amount | is_returned |
|----------------|-------|--------------|-------------|
| Grocery        | South | 3200         | 0           |

## SAMPLE OUTPUT

| order_category | zone  | total_orders | returned_orders | return_rate | avg_order_amount |
|----------------|-------|--------------|-----------------|-------------|------------------|
| Fashion        | West  | 1            | 1               | 1.0         | 7000.0           |
| Electronics    | East  | 1            | 1               | 1.0         | 28000.0          |
| Electronics    | North | 1            | 0               | 0.0         | 45000.0          |
| Fashion        | North | 1            | 0               | 0.0         | 9000.0           |
| Grocery        | South | 1            | 0               | 0.0         | 3200.0           |

## Q6. Hospital Appointment Enrichment & Cancellation Risk Analysis

Part 1 · PySpark — Easy — question 6 of 16

### PROBLEM STATEMENT

A healthcare network operates several outpatient clinics and receives appointment bookings from patients across different service regions.

Appointment information is maintained in one system and contains details such as the appointment ID, patient ID, appointment type, consultation fee, and cancellation indicator.

Patient demographic information is maintained separately and contains the patient name and service region. Due to historical data issues, some patient records contain missing region information.

The healthcare analytics team requires a PySpark pipeline to clean both datasets, enrich appointment records with patient information, identify appointment risk levels, and generate a region-wise summary of cancellation activity.

### OBJECTIVE

The pipeline should:

- Load the appointment and patient datasets.
- Replace missing `consultation_fee` and `is_cancelled` values.
- Replace missing patient regions with "Unknown".
- Perform a left join using `patient_id`.
- Select relevant columns after joining.
- Create an `appointment_risk` column.
- Calculate total appointments.
- Calculate cancelled appointments.
- Calculate cancellation rate.
- Calculate average consultation fee.
- Group results by `appointment_type` and region.
- Sort the final result by descending cancellation rate and appointment count.

### FILE STRUCTURE

```
Project
├── data
│   ├── appointments.csv
│   └── patients.csv
├── main.py
├── solution.py
├── tests.py
└── requirements.txt
```

### DATASET DESCRIPTION

File: appointments.csv

| Column Name      | Type    | Description                                             |
|------------------|---------|---------------------------------------------------------|
| appointment_id   | string  | Unique appointment identifier                           |
| patient_id       | string  | Identifier linking the appointment to a patient         |
| appointment_date | date    | Date of the appointment                                 |
| appointment_type | string  | Type of appointment                                     |
| consultation_fee | integer | Consultation fee                                        |
| is_cancelled     | integer | Cancellation indicator, 1 for cancelled and 0 otherwise |

Possible appointment types:

- General
- Dental
- Cardiology
- Orthopedic
- File: patients.csv

| Column Name  | Type   | Description               |
|--------------|--------|---------------------------|
| patient_id   | string | Unique patient identifier |
| record_id    | string | Patient record identifier |
| patient_name | string | Patient name              |
| region       | string | Patient's service region  |

Possible regions:

- North
- South
- East
- West

Missing regions should be replaced with:

- Unknown

## SAMPLE DATA

appointments.csv

| appointment_id | patient_id | appointment_date | appointment_type | consultation_fee | is_cancelled |
|----------------|------------|------------------|------------------|------------------|--------------|
| A501           | PT101      | 2026-03-01       | General          | 800              | 0            |
| A502           | PT102      | 2026-03-02       | Dental           | 1200             | 1            |
| A503           | PT103      | 2026-03-03       | Cardiology       | 2500             | 0            |
| A504           | PT104      | 2026-03-03       | General          | 700              | 1            |
| A505           | PT105      | 2026-03-04       | Orthopedic       | 1800             | 0            |

patients.csv

| patient_id | record_id | patient_name | region  |
|------------|-----------|--------------|---------|
| PT101      | R101      | Anil Verma   | North   |
| PT102      | R102      | Priya Nair   | West    |
| PT103      | R103      | Karan Shah   | South   |
| PT104      | R104      | Sneha Rao    | East    |
| PT105      | R105      | Mohit Jain   | Unknown |

## IMPLEMENTATION REQUIREMENTS

### FUNCTIONS TO IMPLEMENT IN SOLUTION.PY

#### Function 1 — load\_appointments(spark, appointments\_csv\_path: str)

#### PURPOSE

Load appointment information and replace missing values.

## LOGIC

Read CSV using inferred schema.

Fill missing consultation\_fee with 0.

Fill missing is\_cancelled with 0.

## RETURNS

DataFrame containing:

- appointment\_id
- patient\_id
- appointment\_date
- appointment\_type
- consultation\_fee
- is\_cancelled

### Function 2 — load\_patients(spark, patients\_csv\_path: str)

## PURPOSE

Load patient information and handle missing regions.

## LOGIC

Read CSV with inferred schema.

Replace null region values with "Unknown".

## RETURNS

A cleaned patient DataFrame.

### Function 3 — join\_appointments\_patients(appointments\_df, patients\_df)

## PURPOSE

Combine appointment and patient information.

## LOGIC

Perform a:

## LEFT JOIN

using:

- patient\_id

Select columns in this exact order:

- appointment\_id
- appointment\_date
- patient\_id
- record\_id
- patient\_name
- region
- appointment\_type
- consultation\_fee
- is\_cancelled

### Function 4 — classify\_appointment\_risk(df\_enriched)

## PURPOSE

Create an additional risk classification after enrichment.

## RULES

| Condition                                      | appointment_risk |
|------------------------------------------------|------------------|
| is_cancelled == 1                              | High-Risk        |
| is_cancelled == 0 and consultation_fee >= 2000 | Medium-Risk      |
| Otherwise                                      | Low-Risk         |

## EXAMPLE

| consultation_fee | is_cancelled | appointment_risk |
|------------------|--------------|------------------|
| 800              | 0            | Low-Risk         |
| 1200             | 1            | High-Risk        |
| 2500             | 0            | Medium-Risk      |
| 700              | 1            | High-Risk        |
| 1800             | 0            | Low-Risk         |

## Function 5 — summarize\_cancellation\_by\_segment(df\_risk)

### PURPOSE

Generate cancellation statistics by appointment type and region.

### LOGIC

Group by:

- appointment\_type
- region

### CALCULATE

total\_appointments

cancelled\_appointments

cancellation\_rate

avg\_consultation\_fee

### WHERE

cancellation\_rate =

cancelled\_appointments / total\_appointments

Sort by:

- cancellation\_rate descending
- total\_appointments descending

### RETURNS

appointment\_type

region

total\_appointments

cancelled\_appointments

cancellation\_rate

avg\_consultation\_fee

## SAMPLE I/O

### Function 1 — load\_appointments()

#### SAMPLE INPUT

appointment\_id,patient\_id,appointment\_date,appointment\_type,consultation\_fee,is\_cancelled

A501,PT101,2026-03-01,General,800,0

A502,PT102,2026-03-02,Dental,,1

A503,PT103,2026-03-03,Cardiology,2500,

A504,PT104,2026-03-03,General,700,1

#### SAMPLE OUTPUT

| appointment_id | patient_id | appointment_date | appointment_type | consultation_fee | is_cancelled |
|----------------|------------|------------------|------------------|------------------|--------------|
| A501           | PT101      | 2026-03-01       | General          | 800              | 0            |
| A502           | PT102      | 2026-03-02       | Dental           | 0                | 1            |
| A503           | PT103      | 2026-03-03       | Cardiology       | 2500             | 0            |
| A504           | PT104      | 2026-03-03       | General          | 700              | 1            |

### Function 2 — load\_patients()

#### SAMPLE INPUT

patient\_id,record\_id,patient\_name,region

PT101,R101,Anil Verma,North

PT102,R102,Priya Nair,West

PT103,R103,Karan Shah,

PT104,R104,Sneha Rao,East

#### SAMPLE OUTPUT

| patient_id | record_id | patient_name | region  |
|------------|-----------|--------------|---------|
| PT101      | R101      | Anil Verma   | North   |
| PT102      | R102      | Priya Nair   | West    |
| PT103      | R103      | Karan Shah   | Unknown |
| PT104      | R104      | Sneha Rao    | East    |

### Function 3 — join\_appointments\_patients()

#### SAMPLE OUTPUT

| appointment_id | appointment_date | patient_id | record_id | patient_name | region | appointment_type | consultation_fee | is_cancelled |
|----------------|------------------|------------|-----------|--------------|--------|------------------|------------------|--------------|
| A501           | 2026-03-01       | PT101      | R101      | Anil Verma   | North  | General          | 800              | 0            |
| A502           | 2026-03-02       | PT102      | R102      | Priya Nair   | West   | Dental           | 1200             | 1            |
| A503           | 2026-03-03       | PT103      | R103      | Karan Shah   | South  | Cardiology       | 2500             | 0            |
| A504           | 2026-03-03       | PT104      | R104      | Sneha Rao    | East   | General          | 700              | 1            |

## Function 4 — classify\_appointment\_risk()

### SAMPLE INPUT

| appointment_id | appointment_type | consultation_fee | is_cancelled |
|----------------|------------------|------------------|--------------|
| A501           | General          | 800              | 0            |
| A502           | Dental           | 1200             | 1            |
| A503           | Cardiology       | 2500             | 0            |
| A504           | General          | 700              | 1            |

### SAMPLE OUTPUT

| appointment_id | consultation_fee | is_cancelled | appointment_risk |
|----------------|------------------|--------------|------------------|
| A501           | 800              | 0            | Low-Risk         |
| A502           | 1200             | 1            | High-Risk        |
| A503           | 2500             | 0            | Medium-Risk      |
| A504           | 700              | 1            | High-Risk        |

## Function 5 — summarize\_cancellation\_by\_segment()

### SAMPLE INPUT

| appointment_type | region  | consultation_fee | is_cancelled |
|------------------|---------|------------------|--------------|
| General          | North   | 800              | 0            |
| Dental           | West    | 1200             | 1            |
| Cardiology       | South   | 2500             | 0            |
| General          | East    | 700              | 1            |
| Orthopedic       | Unknown | 1800             | 0            |

### SAMPLE OUTPUT

| appointment_type | region  | total_appointments | cancelled_appointments | cancellation_rate | avg_consultation_fee |
|------------------|---------|--------------------|------------------------|-------------------|----------------------|
| Dental           | West    | 1                  | 1                      | 1.0               | 1200.0               |
| General          | East    | 1                  | 1                      | 1.0               | 700.0                |
| General          | North   | 1                  | 0                      | 0.0               | 800.0                |
| Cardiology       | South   | 1                  | 0                      | 0.0               | 2500.0               |
| Orthopedic       | Unknown | 1                  | 0                      | 0.0               | 1800.0               |

## Q7. Food Delivery Order Performance & Customer Insights

Part 1 · PySpark — Easy — question 7 of 16

### PROBLEM STATEMENT

A food delivery company receives thousands of orders every day from customers across different cities. Each order contains information such as the restaurant, order amount, delivery time, and order status. The company also maintains customer information, including customer segments.

The data team needs to process these datasets to generate meaningful order-level insights. Specifically, they want to clean the raw order data, calculate delivery performance, categorize orders based on delivery time, join the order information with customer demographics, and analyze order performance across customer segments.

The analysis will help the company identify delivery patterns and understand how different customer segments use the platform.

### OBJECTIVE

- Load and preprocess food delivery order data.
- Derive a new column representing the order date from the timestamp.
- Remove invalid order records.
- Calculate daily order amount for each customer.
- Categorize orders into delivery performance levels based on delivery time.
- Join order performance data with customer information.
- Generate insights showing order distribution by customer segment and delivery level.

### FILE STRUCTURE

```
Project
├── data
├── main.py
├── requirements.txt
├── solution.py
└── tests.py
```

### MODULES

- main.py - Contains the driver code/main invocation pipeline.
- solution.py - Must complete the required functions.

### DATASET DESCRIPTION

File: orders.csv

| Column Name      | Type      | Description                         |
|------------------|-----------|-------------------------------------|
| order_id         | integer   | Unique identifier for each order    |
| customer_id      | integer   | Unique customer identifier          |
| order_ts         | timestamp | Timestamp when the order was placed |
| order_amount     | integer   | Total order amount                  |
| delivery_minutes | integer   | Delivery time in minutes            |
| order_status     | string    | Status of the order                 |

### EXAMPLE

order\_id,customer\_id,order\_ts,order\_amount,delivery\_minutes,order\_status

1001,201,2025-09-25 12:15:00,450,32,Delivered

1002,201,2025-09-25 19:30:00,650,48,Delivered  
1003,202,2025-09-25 13:10:00,300,25,Delivered  
1004,203,2025-09-26 20:15:00,800,65,Delivered  
1005,204,2025-09-26 14:20:00,250,0,Cancelled

File: customers.csv

| Column Name      | Type    | Description                                     |
|------------------|---------|-------------------------------------------------|
| customer_id      | integer | Unique customer identifier                      |
| customer_name    | string  | Name of the customer                            |
| customer_segment | string  | Customer category such as New, Regular, Premium |

### EXAMPLE

customer\_id,customer\_name,customer\_segment

201,Amit Sharma,Regular

202,Neha Singh,New

203,Rahul Verma,Premium

204,Priya Nair,Regular

205,Arjun Mehta,Premium

### IMPLEMENTATION REQUIREMENTS

- Functions to implement in solution.py.

#### Function 1 — clean\_orders(df\_orders)

##### PURPOSE

Clean and prepare the order dataset.

##### RETURNS

```
DataFrame[  
  order_id,  
  customer_id,  
  order_date,  
  order_amount,  
  delivery_minutes,  
  order_status  
]
```

##### LOGIC

Derive order\_date from order\_ts.

Keep only rows where:

- customer\_id is not null.
- order\_amount >= 0.
- delivery\_minutes >= 0.
- order\_status is not null.
- Remove unnecessary columns.

#### Function 2 — aggregate\_daily\_orders(df\_clean)

##### PURPOSE

Calculate the total daily order amount and number of orders per customer.

## RETURNS

```
DataFrame[
  customer_id,
  order_date,
  daily_orders,
  daily_order_amount
]
```

## LOGIC

Group by customer\_id and order\_date.

Count orders.

Sum order\_amount.

## Function 3 — compute\_delivery\_level(df\_clean)

### PURPOSE

Categorize delivery performance.

### RULES

| Delivery Time        | Level  |
|----------------------|--------|
| Less than 30 minutes | Fast   |
| 30-59 minutes        | Normal |
| 60 minutes or more   | Slow   |

- For NULL or negative delivery time, delivery\_level should remain NULL.

## RETURNS

```
DataFrame[
  order_id,
  customer_id,
  order_date,
  order_amount,
  delivery_minutes,
  delivery_level
]
```

## Function 4 — join\_customer\_insights(df\_levels, df\_customers)

### PURPOSE

Join order performance with customer information and generate customer-segment insights.

### RETURNS

```
DataFrame[
  customer_segment,
  delivery_level,
  count
]
```

## LOGIC

Join on customer\_id.

Group by customer\_segment and delivery\_level.

Count the number of orders for each combination.

Return the resulting DataFrame.

## SAMPLE I/O

**Function 1 — clean\_orders(df\_orders)****SAMPLE INPUT**

| order_id | customer_id | order_ts            | order_amount | delivery_minutes | order_status |
|----------|-------------|---------------------|--------------|------------------|--------------|
| 1001     | 201         | 2025-09-25 12:15:00 | 450          | 32               | Delivered    |
| 1002     | null        | 2025-09-25 19:30:00 | 650          | 48               | Delivered    |
| 1003     | 202         | 2025-09-25 13:10:00 | -300         | 25               | Delivered    |
| 1004     | 203         | 2025-09-26 20:15:00 | 800          | 65               | Delivered    |
| 1005     | 204         | 2025-09-26 14:20:00 | 250          | 0                | Cancelled    |

**SAMPLE OUTPUT**

| order_id | customer_id | order_date | order_amount | delivery_minutes | order_status |
|----------|-------------|------------|--------------|------------------|--------------|
| 1001     | 201         | 2025-09-25 | 450          | 32               | Delivered    |
| 1004     | 203         | 2025-09-26 | 800          | 65               | Delivered    |
| 1005     | 204         | 2025-09-26 | 250          | 0                | Cancelled    |

**Function 2 — aggregate\_daily\_orders(df\_clean)****SAMPLE INPUT**

| customer_id | order_date | order_amount |
|-------------|------------|--------------|
| 201         | 2025-09-25 | 450          |
| 201         | 2025-09-25 | 650          |
| 202         | 2025-09-25 | 300          |
| 203         | 2025-09-26 | 800          |

**SAMPLE OUTPUT**

| customer_id | order_date | daily_orders | daily_order_amount |
|-------------|------------|--------------|--------------------|
| 201         | 2025-09-25 | 2            | 1100               |
| 202         | 2025-09-25 | 1            | 300                |
| 203         | 2025-09-26 | 1            | 800                |

**Function 3 — compute\_delivery\_level(df\_clean)****SAMPLE INPUT**

| order_id | customer_id | order_date | delivery_minutes |
|----------|-------------|------------|------------------|
| 1001     | 201         | 2025-09-25 | 25               |
| 1002     | 201         | 2025-09-25 | 45               |
| 1003     | 202         | 2025-09-25 | 70               |
| 1004     | 203         | 2025-09-26 | 10               |

**SAMPLE OUTPUT**

| order_id | customer_id | order_date | delivery_minutes | delivery_level |
|----------|-------------|------------|------------------|----------------|
| 1001     | 201         | 2025-09-25 | 25               | Fast           |
| 1002     | 201         | 2025-09-25 | 45               | Normal         |
| 1003     | 202         | 2025-09-25 | 70               | Slow           |

---

|      |     |                 |      |
|------|-----|-----------------|------|
| 1004 | 203 | 2025-09-26   10 | Fast |
|------|-----|-----------------|------|

**Function 4 — join\_customer\_insights(df\_levels, df\_customers)**

Sample Input — df\_levels

| order_id | customer_id | delivery_level |
|----------|-------------|----------------|
|----------|-------------|----------------|

|      |     |        |
|------|-----|--------|
| 1001 | 201 | Fast   |
| 1002 | 201 | Normal |
| 1003 | 202 | Slow   |
| 1004 | 203 | Slow   |
| 1005 | 204 | Fast   |

Sample Input — df\_customers

| customer_id | customer_name | customer_segment |
|-------------|---------------|------------------|
|-------------|---------------|------------------|

|     |             |         |
|-----|-------------|---------|
| 201 | Amit Sharma | Regular |
| 202 | Neha Singh  | New     |
| 203 | Rahul Verma | Premium |
| 204 | Priya Nair  | Regular |

**SAMPLE OUTPUT**

| customer_segment | delivery_level | count |
|------------------|----------------|-------|
|------------------|----------------|-------|

|         |        |   |
|---------|--------|---|
| New     | Slow   | 1 |
| Premium | Slow   | 1 |
| Regular | Fast   | 2 |
| Regular | Normal | 1 |

## Q8. Online Shopping Sales & Customer Segment Analysis

Part 1 · PySpark — Easy — question 8 of 16

### PROBLEM STATEMENT

An e-commerce company collects transaction information from its online shopping platform. Each transaction contains product category, purchase amount, discount, and transaction timestamp. The company also maintains customer demographic information such as membership level.

The data team wants to process these datasets to generate daily sales insights. They need to clean transaction data, calculate daily sales per customer, classify customers based on their spending, join transaction information with customer membership details, and analyze purchasing behavior across membership levels.

The analysis will help the company understand customer purchasing patterns and identify high-value customer segments.

### OBJECTIVE

- Load and preprocess transaction data.
- Derive a transaction date from timestamp information.
- Remove invalid transaction records.
- Calculate total daily spending per customer.
- Categorize customers based on daily spending.
- Join spending information with customer membership details.
- Generate insights showing the distribution of spending levels across membership groups.

### FILE STRUCTURE

```
Project
├── data
├── main.py
├── requirements.txt
├── solution.py
└── tests.py
```

### MODULES

- main.py - Contains the driver code/main invocation pipeline.
- solution.py - Must complete the required functions.

### DATASET DESCRIPTION

File: transactions.csv

| Column Name      | Type      | Description                   |
|------------------|-----------|-------------------------------|
| transaction_id   | integer   | Unique transaction identifier |
| customer_id      | integer   | Unique customer identifier    |
| transaction_ts   | timestamp | Timestamp of the transaction  |
| product_category | string    | Product category              |
| purchase_amount  | integer   | Amount spent                  |
| discount         | integer   | Discount amount               |

### EXAMPLE

transaction\_id,customer\_id,transaction\_ts,product\_category,purchase\_amount,discount

5001,301,2025-09-25 10:15:00,Electronics,25000,2000

5002,301,2025-09-25 18:20:00,Clothing,4500,500

5003,302,2025-09-25 11:30:00,Grocery,3200,200

5004,303,2025-09-26 14:10:00,Electronics,18000,1000

5005,304,2025-09-26 16:45:00,Clothing,6500,700

File: customers.csv

| Column Name   | Type    | Description                |
|---------------|---------|----------------------------|
| customer_id   | integer | Unique customer identifier |
| customer_name | string  | Customer name              |
| membership    | string  | Membership category        |

### EXAMPLE

customer\_id,customer\_name,membership

301,Rohit Sharma,Gold

302,Anjali Mehta,Silver

303,Vivek Rao,Platinum

304,Priya Kapoor,Gold

305,Karan Singh,Silver

### IMPLEMENTATION REQUIREMENTS

- Functions to implement in solution.py.

#### Function 1 — clean\_transactions(df\_transactions)

##### PURPOSE

Clean and prepare transaction data.

##### RETURNS

```
DataFrame[
  transaction_id,
  customer_id,
  transaction_date,
  product_category,
  purchase_amount,
  discount
]
```

##### LOGIC

Derive transaction\_date from transaction\_ts.

Keep only rows where:

- customer\_id is not null.
- purchase\_amount >= 0.
- discount >= 0.
- product\_category is not null.
- Remove unnecessary columns.

#### Function 2 — aggregate\_daily\_spending(df\_clean)

##### PURPOSE

Calculate the total daily spending for each customer.

##### RETURNS

```
DataFrame[
  customer_id,
  transaction_date,
  daily_transactions,
  daily_spending
]
```

## LOGIC

Group by customer\_id and transaction\_date.

Count transactions.

Sum purchase\_amount.

## Function 3 — compute\_spending\_level(df\_daily)

### PURPOSE

Classify customers based on their daily spending.

Spending Rules

| Daily Spending  | Spending Level |
|-----------------|----------------|
| Less than 5,000 | Low            |
| 5,000–19,999    | Medium         |
| 20,000 or more  | High           |

For NULL or negative daily\_spending, the spending\_level should remain NULL.

### RETURNS

```
DataFrame[
  customer_id,
  transaction_date,
  daily_transactions,
  daily_spending,
  spending_level
]
```

## Function 4 — join\_membership\_insights(df\_levels, df\_customers)

### PURPOSE

Join customer spending information with membership details and generate insights.

### INPUTS

df\_levels:

- customer\_id,
- transaction\_date,
- daily\_transactions,
- daily\_spending,
- spending\_level

df\_customers:

- customer\_id,
- customer\_name,
- membership

### RETURNS

```
DataFrame[
  membership,
  spending_level,
]
```

```
count
]
```

## LOGIC

Join both DataFrames using customer\_id.

Group by membership and spending\_level.

Count the number of customer-days for each combination.

Return the resulting DataFrame.

## SAMPLE I/O

### Function 1 — clean\_transactions(df\_transactions)

#### SAMPLE INPUT

| transaction_id | customer_id | transaction_ts      | product_category | purchase_amount | discount |
|----------------|-------------|---------------------|------------------|-----------------|----------|
| 5001           | 301         | 2025-09-25 10:15:00 | Electronics      | 25000           | 2000     |
| 5002           | null        | 2025-09-25 18:20:00 | Clothing         | 4500            | 500      |
| 5003           | 302         | 2025-09-25 11:30:00 | null             | 3200            | 200      |
| 5004           | 303         | 2025-09-26 14:10:00 | Electronics      | -18000          | 1000     |
| 5005           | 304         | 2025-09-26 16:45:00 | Clothing         | 6500            | 700      |

#### SAMPLE OUTPUT

| transaction_id | customer_id | transaction_date | product_category | purchase_amount | discount |
|----------------|-------------|------------------|------------------|-----------------|----------|
| 5001           | 301         | 2025-09-25       | Electronics      | 25000           | 2000     |
| 5005           | 304         | 2025-09-26       | Clothing         | 6500            | 700      |

### Function 2 — aggregate\_daily\_spending(df\_clean)

#### SAMPLE INPUT

| customer_id | transaction_date | purchase_amount |
|-------------|------------------|-----------------|
| 301         | 2025-09-25       | 25000           |
| 301         | 2025-09-25       | 4500            |
| 302         | 2025-09-25       | 3200            |
| 303         | 2025-09-26       | 18000           |

#### SAMPLE OUTPUT

| customer_id | transaction_date | daily_transactions | daily_spending |
|-------------|------------------|--------------------|----------------|
| 301         | 2025-09-25       | 2                  | 29500          |
| 302         | 2025-09-25       | 1                  | 3200           |
| 303         | 2025-09-26       | 1                  | 18000          |

### Function 3 — compute\_spending\_level(df\_daily)

#### SAMPLE INPUT

| customer_id | transaction_date | daily_transactions | daily_spending |
|-------------|------------------|--------------------|----------------|
| 301         | 2025-09-25       | 2                  | 29500          |
| 302         | 2025-09-25       | 1                  | 3200           |

|     |            |   |       |
|-----|------------|---|-------|
| 303 | 2025-09-26 | 1 | 18000 |
| 304 | 2025-09-26 | 2 | 7500  |

#### SAMPLE OUTPUT

| customer_id | transaction_date | daily_transactions | daily_spending | spending_level |
|-------------|------------------|--------------------|----------------|----------------|
| 301         | 2025-09-25       | 2                  | 29500          | High           |
| 302         | 2025-09-25       | 1                  | 3200           | Low            |
| 303         | 2025-09-26       | 1                  | 18000          | Medium         |
| 304         | 2025-09-26       | 2                  | 7500           | Medium         |

#### Function 4 — join\_membership\_insights(df\_levels, df\_customers)

Sample Input — df\_levels

| customer_id | transaction_date | daily_spending | spending_level |
|-------------|------------------|----------------|----------------|
| 301         | 2025-09-25       | 29500          | High           |
| 302         | 2025-09-25       | 3200           | Low            |
| 303         | 2025-09-26       | 18000          | Medium         |
| 304         | 2025-09-26       | 7500           | Medium         |
| 305         | 2025-09-26       | 22000          | High           |

Sample Input — df\_customers

| customer_id | customer_name | membership |
|-------------|---------------|------------|
| 301         | Rohit Sharma  | Gold       |
| 302         | Anjali Mehta  | Silver     |
| 303         | Vivek Rao     | Platinum   |
| 304         | Priya Kapoor  | Gold       |
| 305         | Karan Singh   | Silver     |

#### SAMPLE OUTPUT

| membership | spending_level | count |
|------------|----------------|-------|
| Gold       | High           | 1     |
| Gold       | Medium         | 1     |
| Platinum   | Medium         | 1     |
| Silver     | High           | 1     |
| Silver     | Low            | 1     |

## Q9. Food Delivery Revenue, Discount, and Weather Analysis

Part 1 · PySpark — Easy — question 9 of 16

### PROBLEM STATEMENT

A food delivery platform wants to analyze how order revenue, customer discounts, and delivery conditions vary across different payment methods and environmental factors. Each order record contains information about the order amount, discount received, payment method, and delivery locations. A separate daily conditions dataset contains information such as rainfall, public holidays, and average temperature.

The data engineering team needs to build a PySpark-based ETL and analytics pipeline that combines these datasets, calculates discount percentages for individual orders, and produces aggregated business insights based on payment methods and daily conditions.

The resulting analysis will help the platform understand how weather and holidays affect order values and customer discount behavior.

### OBJECTIVE

- Load and standardize food delivery order and daily-condition datasets.
- Calculate the discount percentage for each order while safely handling zero or missing order amounts.
- Enrich order data with weather and holiday information using the order date.
- Aggregate order and discount metrics by payment method and external conditions.
- Generate clean, rounded summaries suitable for reporting and visualization.

### FILE STRUCTURE

```
Project
├── data
│   ├── orders.csv
│   └── daily_conditions.csv
├── main.py
├── requirements.txt
├── solution.py
└── tests.py
```

### MODULES

- main.py – Contains the driver code / main invocation pipeline.
- solution.py – Must complete the required functions.

### DATASET DESCRIPTION

File: orders.csv

| Column Name     | Type   | Description                                      |
|-----------------|--------|--------------------------------------------------|
| order_id        | string | Unique identifier for each food order.           |
| order_date      | date   | Date on which the order was placed.              |
| payment_type    | string | Payment method used, such as Card, Cash, or UPI. |
| order_amount    | double | Original value of the food order.                |
| discount_amount | double | Discount provided on the order.                  |
| restaurant_zone | string | Zone where the restaurant is located.            |
| delivery_zone   | string | Zone where the order is delivered.               |

## SAMPLE DATA

order\_id,order\_date,payment\_type,order\_amount,discount\_amount,restaurant\_zone,delivery\_zone  
O101,2025-08-01,Card,500.0,50.0,Central,North  
O102,2025-08-01,Cash,320.0,20.0,West,Central  
O103,2025-08-01,UPI,250.0,25.0,South,Central  
O104,2025-08-02,Card,450.0,45.0,North,South  
O105,2025-08-02,Card,600.0,90.0,Central,West  
File: daily\_conditions.csv

| Column Name | Type    | Description                                     |
|-------------|---------|-------------------------------------------------|
| order_date  | date    | Date key used to join with order data.          |
| is_rainy    | integer | 1 if it rained that day, otherwise 0.           |
| is_holiday  | integer | 1 if the day was a public holiday, otherwise 0. |
| avg_temp_c  | double  | Average temperature of the day in Celsius.      |

## SAMPLE DATA

order\_date,is\_rainy,is\_holiday,avg\_temp\_c  
2025-08-01,0,0,30.5  
2025-08-02,1,0,27.8  
2025-08-03,0,0,31.7  
2025-08-04,0,1,32.1  
2025-08-05,1,0,28.4

## IMPLEMENTATION REQUIREMENTS

### FUNCTIONS TO IMPLEMENT IN SOLUTION.PY

#### Function 1 — load\_orders(spark, orders\_csv\_path: str)

##### PURPOSE

Load food delivery order data and standardize the column types for downstream processing.

##### INPUTS

spark: Active SparkSession.  
orders\_csv\_path: Path to orders.csv.

##### COLUMNS USED

- order\_id
- order\_date
- payment\_type
- order\_amount
- discount\_amount
- restaurant\_zone
- delivery\_zone

## RETURNS

DataFrame containing:

- order\_id (string)
- order\_date (date)
- payment\_type (string)
- order\_amount (double)
- discount\_amount (double)
- restaurant\_zone (string)
- delivery\_zone (string)

## LOGIC TO PERFORM

- Read the CSV file with header and schema inference.
- Cast fields to appropriate Spark data types.
- Select only the required columns.
- Return columns in the specified order.

## SAMPLE INPUT

order\_id,order\_date,payment\_type,order\_amount,discount\_amount,restaurant\_zone,delivery\_zone

O101,2025-08-01,Card,500.0,50.0,Central,North

O102,2025-08-01,Cash,320.0,20.0,West,Central

O103,2025-08-01,UPI,250.0,25.0,South,Central

O104,2025-08-02,Card,450.0,45.0,North,South

O105,2025-08-02,Card,600.0,90.0,Central,West

## SAMPLE OUTPUT

| order_id | order_date | payment_type | order_amount | discount_amount | restaurant_zone | delivery_zone |
|----------|------------|--------------|--------------|-----------------|-----------------|---------------|
| O101     | 2025-08-01 | Card         | 500.0        | 50.0            | Central         | North         |
| O102     | 2025-08-01 | Cash         | 320.0        | 20.0            | West            | Central       |
| O103     | 2025-08-01 | UPI          | 250.0        | 25.0            | South           | Central       |
| O104     | 2025-08-02 | Card         | 450.0        | 45.0            | North           | South         |
| O105     | 2025-08-02 | Card         | 600.0        | 90.0            | Central         | West          |

## Function 2 — load\_daily\_conditions(spark, conditions\_csv\_path: str)

### PURPOSE

Load daily weather and holiday information for enriching food delivery orders.

### INPUTS

spark: Active SparkSession.

conditions\_csv\_path: Path to daily\_conditions.csv.

### COLUMNS USED

- order\_date
- is\_rainy
- is\_holiday
- avg\_temp\_c

## RETURNS

DataFrame containing:

- order\_date (date)
- is\_rainy (int)
- is\_holiday (int)
- avg\_temp\_c (double)

## LOGIC TO PERFORM

- Read the CSV file with header and schema inference.
- Cast columns to appropriate data types.
- Return only the required columns in the specified order.

## SAMPLE INPUT

order\_date,is\_rainy,is\_holiday,avg\_temp\_c

2025-08-01,0,0,30.5

2025-08-02,1,0,27.8

2025-08-03,0,0,31.7

2025-08-04,0,1,32.1

2025-08-05,1,0,28.4

## SAMPLE OUTPUT

| order_date | is_rainy | is_holiday | avg_temp_c |
|------------|----------|------------|------------|
| 2025-08-01 | 0        | 0          | 30.5       |
| 2025-08-02 | 1        | 0          | 27.8       |
| 2025-08-03 | 0        | 0          | 31.7       |
| 2025-08-04 | 0        | 1          | 32.1       |
| 2025-08-05 | 1        | 0          | 28.4       |

## Function 3 — compute\_discount\_percentage(df\_orders: DataFrame)

### PURPOSE

Calculate the discount percentage for every order in a safe and readable format.

### INPUTS

df\_orders: DataFrame containing order and discount details.

### COLUMNS USED

- order\_amount
- discount\_amount

### RETURNS

Original DataFrame with a new column:

- discount\_percentage (double)
- rounded to 2 decimal places.

### LOGIC TO PERFORM

### CALCULATE

```
discount_percentage =
```

```
(discount_amount / order_amount) * 100
```

Perform the calculation only when order\_amount > 0.

Prevent divide-by-zero situations.

Handle null values appropriately.

Round the result to two decimal places.

Preserve all original columns.

#### SAMPLE INPUT

| order_id | order_date | payment_type | order_amount | discount_amount | restaurant_zone | delivery_zone |
|----------|------------|--------------|--------------|-----------------|-----------------|---------------|
| 0101     | 2025-08-01 | Card         | 500.0        | 50.0            | Central         | North         |
| 0102     | 2025-08-01 | Cash         | 320.0        | 20.0            | West            | Central       |
| 0103     | 2025-08-01 | UPI          | 250.0        | 25.0            | South           | Central       |
| 0104     | 2025-08-02 | Card         | 450.0        | 45.0            | North           | South         |
| 0105     | 2025-08-02 | Card         | 600.0        | 90.0            | Central         | West          |

#### SAMPLE OUTPUT

| order_id | order_amount | discount_amount | discount_percentage |
|----------|--------------|-----------------|---------------------|
| 0101     | 500.0        | 50.0            | 10.00               |
| 0102     | 320.0        | 20.0            | 6.25                |
| 0103     | 250.0        | 25.0            | 10.00               |
| 0104     | 450.0        | 45.0            | 10.00               |
| 0105     | 600.0        | 90.0            | 15.00               |

**Function 4 — aggregate\_by\_payment\_and\_conditions(df\_orders\_with\_discount: DataFrame, df\_conditions: DataFrame)**

#### PURPOSE

Analyze how payment methods and external conditions influence average order value and discount percentages.

#### INPUTS

df\_orders\_with\_discount: Order DataFrame containing discount\_percentage.

df\_conditions: Daily weather and holiday DataFrame.

#### COLUMNS USED

- order\_date
- payment\_type
- is\_rainy
- is\_holiday
- discount\_percentage
- order\_amount

#### RETURNS

DataFrame containing:

- payment\_type

- is\_rainy
- is\_holiday
- avg\_discount\_percentage
- avg\_order\_amount

## LOGIC TO PERFORM

- Perform a left join between order data and daily conditions using order\_date.

Group the combined data by:

- payment\_type
- is\_rainy
- is\_holiday

## CALCULATE

Average discount percentage as avg\_discount\_percentage.

Average order amount as avg\_order\_amount.

Round both averages to 2 decimal places.

Return a clean summary DataFrame.

## SAMPLE INPUT

df\_orders\_with\_discount

| order_id | order_date | payment_type | order_amount | discount_percentage |
|----------|------------|--------------|--------------|---------------------|
| 0101     | 2025-08-01 | Card         | 500.0        | 10.0                |
| 0102     | 2025-08-01 | Cash         | 320.0        | 6.25                |
| 0103     | 2025-08-02 | Card         | 400.0        | 10.0                |
| 0104     | 2025-08-02 | Card         | 600.0        | 15.0                |

df\_conditions

| order_date | is_rainy | is_holiday | avg_temp_c |
|------------|----------|------------|------------|
| 2025-08-01 | 0        | 0          | 30.5       |
| 2025-08-02 | 1        | 0          | 27.8       |

## SAMPLE OUTPUT

| payment_type | is_rainy | is_holiday | avg_discount_percentage | avg_order_amount |
|--------------|----------|------------|-------------------------|------------------|
| Card         | 0        | 0          | 10.00                   | 500.00           |
| Card         | 1        | 0          | 12.50                   | 500.00           |
| Cash         | 0        | 0          | 6.25                    | 320.00           |

## Q10. Online Shopping Revenue, Cashback, and Delivery Condition Analysis

Part 1 · PySpark — Easy — question 10 of 16

### PROBLEM STATEMENT

An online shopping company wants to understand how customer cashback behavior and purchase values change under different delivery conditions. Every transaction contains information about the purchase amount, cashback received, payment channel, and shopping regions. A separate daily operations dataset records whether the day experienced heavy traffic, a promotional event, and the average delivery time.

The data engineering team needs to create a PySpark-based ETL and analytics pipeline that standardizes transaction and operational data, calculates cashback percentages, enriches transactions with daily operational conditions, and generates aggregated insights.

In addition to payment-channel analysis, the company wants to identify whether transactions happened on normal or delayed-delivery days based on the average delivery time.

This additional classification should be incorporated into the final analysis without significantly changing the overall pipeline complexity.

### OBJECTIVE

- Load and standardize online shopping transaction and operational datasets.
- Calculate cashback percentage for every valid transaction.
- Categorize delivery conditions based on average delivery time.
- Enrich transaction data using transaction date.
- Aggregate cashback and purchase metrics by payment channel and operational conditions.
- Produce clean, rounded summaries for business reporting.

### FILE STRUCTURE

```
Project
├── data
│   ├── transactions.csv
│   └── delivery_conditions.csv
├── main.py
├── requirements.txt
├── solution.py
└── tests.py
```

### MODULES

- main.py – Contains the driver code / main invocation pipeline.
- solution.py – Must complete the required functions.

### DATASET DESCRIPTION

File: transactions.csv

| Column Name      | Type   | Description                                      |
|------------------|--------|--------------------------------------------------|
| transaction_id   | string | Unique identifier for each shopping transaction. |
| transaction_date | date   | Date when the transaction was completed.         |
| payment_channel  | string | Payment method such as Card, UPI, or Wallet.     |
| purchase_amount  | double | Total purchase amount before cashback.           |
| cashback_amount  | double | Cashback received by the customer.               |
| store_region     | string | Region where the store/order originated.         |

| Column Name     | Type   | Description                          |
|-----------------|--------|--------------------------------------|
| customer_region | string | Region associated with the customer. |

### SAMPLE DATA

transaction\_id,transaction\_date,payment\_channel,purchase\_amount,cashback\_amount,store\_region,customer\_region

TX101,2025-09-01,Card,800.0,80.0,North,Central

TX102,2025-09-01,UPI,500.0,25.0,West,North

TX103,2025-09-01,Wallet,300.0,30.0,South,West

TX104,2025-09-02,Card,1000.0,100.0,Central,South

TX105,2025-09-02,Card,750.0,60.0,North,Central

File: delivery\_conditions.csv

| Column Name       | Type    | Description                                          |
|-------------------|---------|------------------------------------------------------|
| transaction_date  | date    | Date key used to enrich transaction data.            |
| heavy_traffic     | integer | 1 if heavy traffic was recorded, otherwise 0.        |
| is_promotion_day  | integer | 1 if a promotional campaign was active, otherwise 0. |
| avg_delivery_mins | double  | Average delivery time for the day in minutes.        |

### SAMPLE DATA

transaction\_date,heavy\_traffic,is\_promotion\_day,avg\_delivery\_mins

2025-09-01,0,0,28.5

2025-09-02,1,0,42.0

2025-09-03,0,1,25.5

2025-09-04,1,1,48.0

2025-09-05,0,0,30.0

### IMPLEMENTATION REQUIREMENTS

#### FUNCTIONS TO IMPLEMENT IN SOLUTION.PY

#### Function 1 — load\_transactions(spark, transactions\_csv\_path: str)

##### PURPOSE

Load online shopping transactions and standardize their data types.

##### INPUTS

spark: Active SparkSession.

transactions\_csv\_path: Path to transactions.csv.

##### COLUMNS USED

- transaction\_id
- transaction\_date
- payment\_channel

- purchase\_amount
- cashback\_amount
- store\_region
- customer\_region

## RETURNS

DataFrame containing:

- transaction\_id (string)
- transaction\_date (date)
- payment\_channel (string)
- purchase\_amount (double)
- cashback\_amount (double)
- store\_region (string)
- customer\_region (string)

## LOGIC TO PERFORM

- Read the CSV using header and schema inference.
- Cast fields to suitable Spark data types.
- Select only the required columns.
- Return columns in the specified order.

## SAMPLE INPUT

transaction\_id,transaction\_date,payment\_channel,purchase\_amount,cashback\_amount,store\_region,customer\_region

TX101,2025-09-01,Card,800.0,80.0,North,Central

TX102,2025-09-01,UPI,500.0,25.0,West,North

TX103,2025-09-01,Wallet,300.0,30.0,South,West

TX104,2025-09-02,Card,1000.0,100.0,Central,South

TX105,2025-09-02,Card,750.0,60.0,North,Central

## SAMPLE OUTPUT

| transaction_id | transaction_date | payment_channel | purchase_amount | cashback_amount | store_region | customer_region |
|----------------|------------------|-----------------|-----------------|-----------------|--------------|-----------------|
| TX101          | 2025-09-01       | Card            | 800.0           | 80.0            | North        | Central         |
| TX102          | 2025-09-01       | UPI             | 500.0           | 25.0            | West         | North           |
| TX103          | 2025-09-01       | Wallet          | 300.0           | 30.0            | South        | West            |
| TX104          | 2025-09-02       | Card            | 1000.0          | 100.0           | Central      | South           |
| TX105          | 2025-09-02       | Card            | 750.0           | 60.0            | North        | Central         |

## Function 2 — load\_delivery\_conditions(spark, conditions\_csv\_path: str)

### PURPOSE

Load daily operational conditions and prepare them for transaction enrichment.

### INPUTS

spark: Active SparkSession.

conditions\_csv\_path: Path to delivery\_conditions.csv.

### COLUMNS USED

- transaction\_date
- heavy\_traffic
- is\_promotion\_day
- avg\_delivery\_mins

## RETURNS

DataFrame containing:

- transaction\_date (date)
- heavy\_traffic (int)
- is\_promotion\_day (int)
- avg\_delivery\_mins (double)

## LOGIC TO PERFORM

- Read the CSV with header and inferred schema.
- Cast the fields to appropriate data types.
- Return only the required columns.
- Maintain the specified column order.

## SAMPLE INPUT

transaction\_date,heavy\_traffic,is\_promotion\_day,avg\_delivery\_mins

2025-09-01,0,0,28.5

2025-09-02,1,0,42.0

2025-09-03,0,1,25.5

2025-09-04,1,1,48.0

2025-09-05,0,0,30.0

## SAMPLE OUTPUT

| transaction_date | heavy_traffic | is_promotion_day | avg_delivery_mins |
|------------------|---------------|------------------|-------------------|
| 2025-09-01       | 0             | 0                | 28.5              |
| 2025-09-02       | 1             | 0                | 42.0              |
| 2025-09-03       | 0             | 1                | 25.5              |
| 2025-09-04       | 1             | 1                | 48.0              |
| 2025-09-05       | 0             | 0                | 30.0              |

## Function 3 — compute\_cashback\_percentage(df\_transactions: DataFrame)

### PURPOSE

Calculate the cashback percentage associated with each transaction.

### INPUTS

df\_transactions: Transaction DataFrame containing purchase and cashback information.

### COLUMNS USED

- purchase\_amount
- cashback\_amount

### RETURNS

Original DataFrame with an additional column:

- cashback\_percentage (double)
- rounded to 2 decimal places.

## LOGIC TO PERFORM

### CALCULATE

cashback\_percentage =

$(\text{cashback\_amount} / \text{purchase\_amount}) * 100$

The implementation must:

- Calculate the percentage only when purchase\_amount > 0.
- Avoid divide-by-zero errors.
- Handle null purchase/cashback values appropriately.
- Round the calculated value to 2 decimal places.
- Preserve all original transaction columns.

### SAMPLE INPUT

| transaction_id | purchase_amount | cashback_amount |
|----------------|-----------------|-----------------|
| TX101          | 800.0           | 80.0            |
| TX102          | 500.0           | 25.0            |
| TX103          | 300.0           | 30.0            |
| TX104          | 1000.0          | 100.0           |
| TX105          | 750.0           | 60.0            |

### SAMPLE OUTPUT

| transaction_id | purchase_amount | cashback_amount | cashback_percentage |
|----------------|-----------------|-----------------|---------------------|
| TX101          | 800.0           | 80.0            | 10.00               |
| TX102          | 500.0           | 25.0            | 5.00                |
| TX103          | 300.0           | 30.0            | 10.00               |
| TX104          | 1000.0          | 100.0           | 10.00               |
| TX105          | 750.0           | 60.0            | 8.00                |

**Function 4 — aggregate\_by\_channel\_and\_conditions(df\_transactions\_with\_cashback: DataFrame, df\_conditions: DataFrame)**

### PURPOSE

Study how payment channels and delivery conditions affect purchase values and cashback behavior.

### INPUTS

df\_transactions\_with\_cashback: Transaction DataFrame containing cashback\_percentage.

df\_conditions: Delivery condition DataFrame.

### COLUMNS USED

- transaction\_date
- payment\_channel
- heavy\_traffic
- is\_promotion\_day
- avg\_delivery\_mins
- cashback\_percentage

- purchase\_amount

## RETURNS

DataFrame containing:

- payment\_channel
- heavy\_traffic
- is\_promotion\_day
- delivery\_status
- avg\_cashback\_percentage
- avg\_purchase\_amount
- Additional Requirement

Create a derived column called delivery\_status based on avg\_delivery\_mins:

- avg\_delivery\_mins <= 35 → Normal
- avg\_delivery\_mins > 35 → Delayed
- If avg\_delivery\_mins is NULL, delivery\_status should remain NULL.

## LOGIC TO PERFORM

- Perform a left join between transaction data and delivery-condition data using transaction\_date.
- Derive delivery\_status from avg\_delivery\_mins.

Group the enriched data by:

- payment\_channel
- heavy\_traffic
- is\_promotion\_day
- delivery\_status

## CALCULATE

Average cashback percentage as avg\_cashback\_percentage.

Average purchase amount as avg\_purchase\_amount.

Round both averages to 2 decimal places.

Return a clean summary DataFrame.

## SAMPLE INPUT

df\_transactions\_with\_cashback

| transaction_id | transaction_date | payment_channel | purchase_amount | cashback_percentage |
|----------------|------------------|-----------------|-----------------|---------------------|
| TX101          | 2025-09-01       | Card            | 800.0           | 10.00               |
| TX102          | 2025-09-01       | UPI             | 500.0           | 5.00                |
| TX103          | 2025-09-02       | Card            | 1000.0          | 10.00               |
| TX104          | 2025-09-02       | Card            | 600.0           | 8.00                |

df\_conditions

| transaction_date | heavy_traffic | is_promotion_day | avg_delivery_mins |
|------------------|---------------|------------------|-------------------|
| 2025-09-01       | 0             | 0                | 28.5              |
| 2025-09-02       | 1             | 0                | 42.0              |

## SAMPLE OUTPUT

| payment_channel | heavy_traffic | is_promotion_day | delivery_status | avg_cashback_percentage | avg_purchase_amount |
|-----------------|---------------|------------------|-----------------|-------------------------|---------------------|
| Card            | 0             | 0                | Normal          | 10.00                   | 800.00              |
| UPI             | 0             | 0                | Normal          | 5.00                    | 500.00              |
| Card            | 1             | 0                | Delayed         | 9.00                    | 800.00              |

## Q11. Online Food Delivery Customer Engagement and Inactive Customer Flagging

Part 1 · PySpark — Easy — question 11 of 16

### PROBLEM STATEMENT

A nationwide online food delivery platform receives thousands of food orders every day from customers across different cities. The analytics team maintains a customer master file containing customer registration details and account status, along with a daily order transaction file that records each order using the customer identifier, order date, order hour, and food category.

To support customer engagement campaigns and operational planning, the data engineering team needs an automated PySpark pipeline that removes invalid order records, calculates order activity for each customer, identifies customers who have not placed any valid orders, and produces an hourly order summary across the complete order dataset.

The pipeline will help the company understand customer ordering patterns, identify inactive customers for targeted campaigns, and determine the hours with higher order activity.

### OBJECTIVE

- Clean raw order records by removing rows with missing identifiers and invalid order hours.
- Calculate the total number of valid orders placed by each customer.
- Identify customers who have no recorded valid order activity by joining the customer master with the activity summary.
- Generate an hourly order summary showing the total number of orders placed during each hour of the day.

### FILE STRUCTURE

```
Project
├── data
│   ├── orders.csv
│   └── customers.csv
├── main.py
├── solution.py
├── tests.py
└── requirements.txt
```

### PURPOSE OF FILES

- main.py - Driver script that loads both CSV files, invokes the PySpark pipeline, and displays intermediate results.
- solution.py - Module containing the four PySpark transformation functions to be implemented.
- data/orders.csv - Raw food order transaction dataset used as input.
- data/customers.csv - Customer master file containing registration details and account status.
- tests.py - Automated tests for validating the implemented functions.

### DATASET DESCRIPTION

File: orders.csv

### SCHEMA

| Column Name | Data Type | Description                                             |
|-------------|-----------|---------------------------------------------------------|
| orderid     | string    | Unique identifier for each food order.                  |
| customerid  | string    | Unique identifier of the customer who placed the order. |

| Column Name  | Data Type | Description                                                                  |
|--------------|-----------|------------------------------------------------------------------------------|
| orderdate    | date      | Date on which the order was placed.                                          |
| orderhour    | integer   | Hour of the day when the order was placed (0 to 23).                         |
| foodcategory | string    | Food category such as Pizza, Indian, Chinese, Burger, Dessert, or Beverages. |

## SAMPLE DATA

| orderid | customerid | orderdate  | orderhour | foodcategory |
|---------|------------|------------|-----------|--------------|
| 0001    | C101       | 2025-11-01 | 12        | Indian       |
| 0002    | C101       | 2025-11-02 | 19        | Pizza        |
| 0007    | C102       | 2025-11-01 | 13        | Chinese      |
| 0014    | C103       | 2025-11-02 | 20        | Burger       |
| 0019    | C105       | 2025-11-01 | 21        | Dessert      |
| 0024    | C106       | 2025-11-03 | 13        | Beverages    |
| 0028    | C107       | 2025-11-04 | 19        | Pizza        |
| 0031    | C108       | 2025-11-05 | 18        | Indian       |

File: customers.csv

## SCHEMA

| Column Name      | Data Type | Description                                          |
|------------------|-----------|------------------------------------------------------|
| customerid       | string    | Unique identifier for each customer.                 |
| registrationdate | date      | Date when the customer registered with the platform. |
| status           | string    | Customer account status such as ACTIVE or BLOCKED.   |

## SAMPLE DATA

| customerid | registrationdate | status  |
|------------|------------------|---------|
| C101       | 2025-01-10       | ACTIVE  |
| C102       | 2025-02-14       | ACTIVE  |
| C103       | 2025-03-21       | ACTIVE  |
| C104       | 2025-04-08       | BLOCKED |
| C105       | 2025-05-12       | ACTIVE  |
| C109       | 2025-08-17       | ACTIVE  |
| C110       | 2025-09-25       | ACTIVE  |

## TASK

### FUNCTIONS TO IMPLEMENT IN SOLUTION.PY

#### Function 1 — clean\_orders(df\_orders)

#### PURPOSE

Remove invalid food order records before performing downstream analysis.

## PARAMETERS

df\_orders is a PySpark DataFrame loaded from orders.csv.

The DataFrame contains orderid, customerid, orderdate, orderhour, and foodcategory.

## LOGIC

Retain only records where:

- orderid is not null.
- customerid is not null.
- orderhour is between 0 and 23, inclusive.
- Any record failing one or more conditions must be removed.

## RETURNS

A cleaned DataFrame containing the same columns as the input.

### Function 2 — compute\_customer\_activity(df\_clean)

## PURPOSE

Calculate the total number of valid orders placed by each customer.

## PARAMETERS

df\_clean is the cleaned orders DataFrame returned by clean\_orders.

## LOGIC

Group the DataFrame by customerid and count the number of order records for every customer.

The count column must be named:

- order\_count

## RETURNS

A DataFrame containing:

| Column      | Type   |
|-------------|--------|
| customerid  | string |
| order_count | long   |

### Function 3 — flag\_inactive\_customers(df\_customers, df\_activity)

## PURPOSE

Identify customers who have no valid order activity.

## PARAMETERS

df\_customers contains customerid, registrationdate, and status.

df\_activity contains customerid and order\_count.

## LOGIC

Perform a left join from the customer master DataFrame to the activity DataFrame using customerid.

If a customer does not have a matching record in df\_activity, mark:

```
isinactive = 1
```

Otherwise:

```
isinactive = 0
```

The final DataFrame must contain only:

- customerid
- status
- isinactive

## RETURNS

A DataFrame containing:

| Column     | Type    |
|------------|---------|
| customerid | string  |
| status     | string  |
| isinactive | integer |

## Function 4 — engagement\_summary\_by\_hour(df\_clean)

### PURPOSE

Generate an hourly order engagement summary.

### PARAMETERS

df\_clean is the cleaned order DataFrame.

### LOGIC

Group records by orderhour.

Count the number of orders for each hour.

Rename the count column to ordercount.

Sort the result by orderhour in ascending order.

## RETURNS

A DataFrame containing:

| Column     | Type    |
|------------|---------|
| orderhour  | integer |
| ordercount | long    |

## SAMPLE INPUT AND OUTPUT

### Function 1 — clean\_orders(df\_orders)

#### SAMPLE INPUT

| orderid | customerid | orderdate  | orderhour | foodcategory |
|---------|------------|------------|-----------|--------------|
| 0001    | C101       | 2025-11-01 | 12        | Indian       |
| null    | C102       | 2025-11-02 | 14        | Pizza        |
| 0003    | null       | 2025-11-03 | 11        | Chinese      |
| 0004    | C103       | 2025-11-04 | 25        | Burger       |
| 0005    | C105       | 2025-11-05 | 18        | Dessert      |

#### SAMPLE OUTPUT

| orderid | customerid | orderdate  | orderhour | foodcategory |
|---------|------------|------------|-----------|--------------|
| 0001    | C101       | 2025-11-01 | 12        | Indian       |
| 0005    | C105       | 2025-11-05 | 18        | Dessert      |

## Function 2 — compute\_customer\_activity(df\_clean)

### SAMPLE INPUT

| orderid | customerid | orderdate  | orderhour | foodcategory |
|---------|------------|------------|-----------|--------------|
| 0001    | C101       | 2025-11-01 | 12        | Indian       |
| 0002    | C101       | 2025-11-02 | 19        | Pizza        |
| 0003    | C101       | 2025-11-03 | 20        | Chinese      |
| 0007    | C102       | 2025-11-01 | 13        | Burger       |
| 0008    | C102       | 2025-11-02 | 14        | Pizza        |
| 0014    | C103       | 2025-11-02 | 20        | Burger       |

### SAMPLE OUTPUT

| customerid | order_count |
|------------|-------------|
| C101       | 3           |
| C102       | 2           |
| C103       | 1           |

## Function 3 — flag\_inactive\_customers(df\_customers, df\_activity)

Sample Input — df\_customers

| customerid | registrationdate | status  |
|------------|------------------|---------|
| C101       | 2025-01-10       | ACTIVE  |
| C102       | 2025-02-14       | ACTIVE  |
| C103       | 2025-03-21       | ACTIVE  |
| C104       | 2025-04-08       | BLOCKED |
| C109       | 2025-08-17       | ACTIVE  |

Sample Input — df\_activity

| customerid | order_count |
|------------|-------------|
| C101       | 3           |
| C102       | 2           |
| C103       | 1           |

### SAMPLE OUTPUT

| customerid | status  | isinactive |
|------------|---------|------------|
| C101       | ACTIVE  | 0          |
| C102       | ACTIVE  | 0          |
| C103       | ACTIVE  | 0          |
| C104       | BLOCKED | 1          |
| C109       | ACTIVE  | 1          |

**Function 4 — engagement\_summary\_by\_hour(df\_clean)****SAMPLE INPUT**

| orderid | customerid | orderdate  | orderhour | foodcategory |
|---------|------------|------------|-----------|--------------|
| 0001    | C101       | 2025-11-01 | 12        | Indian       |
| 0002    | C102       | 2025-11-01 | 12        | Pizza        |
| 0003    | C103       | 2025-11-01 | 19        | Chinese      |
| 0004    | C101       | 2025-11-02 | 19        | Burger       |
| 0005    | C105       | 2025-11-02 | 19        | Dessert      |
| 0006    | C102       | 2025-11-03 | 21        | Pizza        |

**SAMPLE OUTPUT**

| orderhour | ordercount |
|-----------|------------|
| 12        | 2          |
| 19        | 3          |
| 21        | 1          |

## Q12. Online Music Streaming Listener Engagement and Inactive Subscriber Analysis

Part 1 · PySpark — Easy — question 12 of 16

### PROBLEM STATEMENT

A digital music streaming platform records thousands of listening sessions every day from subscribers using its mobile and web applications. The analytics team maintains a subscriber master file containing registration information, subscription status, and membership tier. A separate listening-session dataset records each playback event using the subscriber identifier, listening date, hour of playback, and music genre.

The data engineering team wants to build a PySpark pipeline that cleans invalid listening records, calculates listening activity for every subscriber, identifies subscribers with no recorded listening activity, and generates an hourly engagement report.

In addition to identifying inactive subscribers, the platform also wants to retain the subscriber's membership tier in the inactivity report so that engagement teams can understand which subscription groups require attention.

### OBJECTIVE

- Clean listening-session records by removing rows with missing identifiers and invalid listening hours.
- Calculate the total number of listening sessions for every subscriber.
- Join subscriber master data with listening activity and identify subscribers with no recorded sessions.
- Preserve the subscriber's membership tier while generating the inactive-subscriber report.
- Produce an hourly listening summary to identify peak engagement periods.

### FILE STRUCTURE

```
Project
├── data
│   ├── listening_sessions.csv
│   └── subscribers.csv
├── main.py
├── solution.py
├── tests.py
└── requirements.txt
```

### PURPOSE OF FILES

- main.py - Driver script that loads both datasets and executes the PySpark pipeline.
- solution.py - Contains the four PySpark functions that must be implemented.
- data/listening\_sessions.csv - Raw music listening-session dataset.
- data/subscribers.csv - Subscriber master dataset.
- tests.py - Automated tests for the required transformations.

### DATASET DESCRIPTION

File: listening\_sessions.csv

### SCHEMA

| Column Name   | Data Type | Description                                           |
|---------------|-----------|-------------------------------------------------------|
| sessionid     | string    | Unique identifier for each listening session.         |
| subscriberid  | string    | Identifier of the subscriber who started the session. |
| listeningdate | date      | Date on which the listening session occurred.         |

| Column Name   | Data Type | Description                                                         |
|---------------|-----------|---------------------------------------------------------------------|
| listeninghour | integer   | Hour of the day when the session started (0 to 23).                 |
| genre         | string    | Music genre such as Pop, Rock, Jazz, Classical, HipHop, or Country. |

## SAMPLE DATA

| sessionid | subscriberid | listeningdate | listeninghour | genre     |
|-----------|--------------|---------------|---------------|-----------|
| S001      | U101         | 2025-12-01    | 8             | Pop       |
| S002      | U101         | 2025-12-02    | 18            | Rock      |
| S007      | U102         | 2025-12-01    | 9             | Jazz      |
| S014      | U103         | 2025-12-02    | 20            | HipHop    |
| S019      | U105         | 2025-12-01    | 21            | Classical |
| S024      | U106         | 2025-12-03    | 9             | Pop       |
| S028      | U107         | 2025-12-04    | 18            | Rock      |
| S031      | U108         | 2025-12-05    | 16            | Country   |

File: subscribers.csv

## SCHEMA

| Column Name      | Data Type | Description                                            |
|------------------|-----------|--------------------------------------------------------|
| subscriberid     | string    | Unique identifier for each subscriber.                 |
| registrationdate | date      | Date when the subscriber registered.                   |
| status           | string    | Subscriber account status such as ACTIVE or SUSPENDED. |
| membershiptier   | string    | Subscription tier such as BASIC, STANDARD, or PREMIUM. |

## SAMPLE DATA

| subscriberid | registrationdate | status    | membershiptier |
|--------------|------------------|-----------|----------------|
| U101         | 2025-01-12       | ACTIVE    | PREMIUM        |
| U102         | 2025-02-18       | ACTIVE    | STANDARD       |
| U103         | 2025-03-22       | ACTIVE    | BASIC          |
| U104         | 2025-04-10       | SUSPENDED | BASIC          |
| U105         | 2025-05-15       | ACTIVE    | PREMIUM        |
| U109         | 2025-08-20       | ACTIVE    | STANDARD       |
| U110         | 2025-09-28       | ACTIVE    | PREMIUM        |

## TASK

### FUNCTIONS TO IMPLEMENT IN SOLUTION.PY

#### Function 1 — clean\_listening\_sessions(df\_sessions)

#### PURPOSE

Remove invalid listening-session records before analysis.

## PARAMETERS

df\_sessions is a PySpark DataFrame loaded from listening\_sessions.csv containing:

- sessionid
- subscriberid
- listeningdate
- listeninghour
- genre

## LOGIC

Retain only rows where:

- sessionid is not null.
- subscriberid is not null.
- listeninghour is between 0 and 23, inclusive.
- Invalid records must be removed.

## RETURNS

A cleaned DataFrame containing the same columns as the input.

### Function 2 — compute\_subscriber\_activity(df\_clean)

## PURPOSE

Calculate the total number of listening sessions for every subscriber.

## PARAMETERS

df\_clean is the cleaned listening-session DataFrame.

## LOGIC

Group the DataFrame by subscriberid and count the number of session records.

The resulting count column must be named:

- session\_count

## RETURNS

A DataFrame containing:

| Column        | Type   |
|---------------|--------|
| subscriberid  | string |
| session_count | long   |

### Function 3 — flag\_inactive\_subscribers(df\_subscribers, df\_activity)

## PURPOSE

Identify subscribers who have no recorded listening activity while retaining their subscription information.

## PARAMETERS

df\_subscribers contains:

- subscriberid
- registrationdate
- status
- membershiptier

df\_activity contains:

- subscriberid
- session\_count

## LOGIC

Perform a left join from df\_subscribers to df\_activity using:

- subscriberid

If no matching activity record exists, set:

```
isinactive = 1
```

Otherwise:

```
isinactive = 0
```

The final DataFrame must contain only:

- subscriberid
- status
- membership tier
- isinactive

## RETURNS

A DataFrame containing:

| Column          | Type    |
|-----------------|---------|
| subscriberid    | string  |
| status          | string  |
| membership tier | string  |
| isinactive      | integer |

Additional requirement in Q2: The membership tier column must be preserved in the final output. This changes the business requirement without introducing a new PySpark transformation pattern.

## Function 4 — engagement\_summary\_by\_hour(df\_clean)

### PURPOSE

Generate an hourly listening engagement summary.

### PARAMETERS

df\_clean is the cleaned listening-session DataFrame.

### LOGIC

Group the data by listeninghour.

Count the number of listening sessions.

Rename the count column to sessioncount.

Sort the result by listeninghour in ascending order.

## RETURNS

A DataFrame containing:

| Column        | Type    |
|---------------|---------|
| listeninghour | integer |
| sessioncount  | long    |

## SAMPLE INPUT AND OUTPUT

## Function 1 — clean\_listening\_sessions(df\_sessions)

## SAMPLE INPUT

| sessionid | subscriberid | listeningdate | listeninghour | genre     |
|-----------|--------------|---------------|---------------|-----------|
| S001      | U101         | 2025-12-01    | 8             | Pop       |
| null      | U102         | 2025-12-02    | 14            | Rock      |
| S003      | null         | 2025-12-03    | 11            | Jazz      |
| S004      | U103         | 2025-12-04    | 25            | HipHop    |
| S005      | U105         | 2025-12-05    | 18            | Classical |

## SAMPLE OUTPUT

| sessionid | subscriberid | listeningdate | listeninghour | genre     |
|-----------|--------------|---------------|---------------|-----------|
| S001      | U101         | 2025-12-01    | 8             | Pop       |
| S005      | U105         | 2025-12-05    | 18            | Classical |

## Function 2 — compute\_subscriber\_activity(df\_clean)

## SAMPLE INPUT

| sessionid | subscriberid | listeningdate | listeninghour | genre     |
|-----------|--------------|---------------|---------------|-----------|
| S001      | U101         | 2025-12-01    | 8             | Pop       |
| S002      | U101         | 2025-12-02    | 18            | Rock      |
| S003      | U101         | 2025-12-03    | 20            | Jazz      |
| S007      | U102         | 2025-12-01    | 9             | Classical |
| S008      | U102         | 2025-12-02    | 14            | Pop       |
| S014      | U103         | 2025-12-02    | 20            | HipHop    |

## SAMPLE OUTPUT

| subscriberid | session_count |
|--------------|---------------|
| U101         | 3             |
| U102         | 2             |
| U103         | 1             |

## Function 3 — flag\_inactive\_subscribers(df\_subscribers, df\_activity)

Sample Input — df\_subscribers

| subscriberid | registrationdate | status    | membership tier |
|--------------|------------------|-----------|-----------------|
| U101         | 2025-01-12       | ACTIVE    | PREMIUM         |
| U102         | 2025-02-18       | ACTIVE    | STANDARD        |
| U103         | 2025-03-22       | ACTIVE    | BASIC           |
| U104         | 2025-04-10       | SUSPENDED | BASIC           |
| U109         | 2025-08-20       | ACTIVE    | STANDARD        |

Sample Input — df\_activity

| subscriberid | session_count |
|--------------|---------------|
| U101         | 3             |
| U102         | 2             |
| U103         | 1             |

#### SAMPLE OUTPUT

| subscriberid | status    | membershiptier | isinactive |
|--------------|-----------|----------------|------------|
| U101         | ACTIVE    | PREMIUM        | 0          |
| U102         | ACTIVE    | STANDARD       | 0          |
| U103         | ACTIVE    | BASIC          | 0          |
| U104         | SUSPENDED | BASIC          | 1          |
| U109         | ACTIVE    | STANDARD       | 1          |

#### Function 4 — engagement\_summary\_by\_hour(df\_clean)

#### SAMPLE INPUT

| sessionid | subscriberid | listeningdate | listeninghour | genre     |
|-----------|--------------|---------------|---------------|-----------|
| S001      | U101         | 2025-12-01    | 8             | Pop       |
| S002      | U102         | 2025-12-01    | 8             | Rock      |
| S003      | U103         | 2025-12-01    | 18            | Jazz      |
| S004      | U101         | 2025-12-02    | 18            | HipHop    |
| S005      | U105         | 2025-12-02    | 18            | Classical |
| S006      | U102         | 2025-12-03    | 21            | Pop       |

#### SAMPLE OUTPUT

| listeninghour | sessioncount |
|---------------|--------------|
| 8             | 2            |
| 18            | 3            |
| 21            | 1            |

## Q13. Weather Monitoring Data Cleaning and City Temperature Analysis

Part 1 · PySpark — Easy — question 13 of 16

### PROBLEM STATEMENT

A regional data analytics team collects daily temperature information from weather monitoring stations located in different cities across India. The data is received every day in CSV format and is later consumed by reporting and visualization systems.

Because the source data is collected from multiple weather stations, the incoming feed may contain incomplete records. Some records may not contain a valid date, some may be missing the city name, and certain temperature readings may be unavailable. These incomplete records can affect city-level temperature calculations and downstream reports.

The analytics team requires a reliable PySpark pipeline that prepares the incoming weather data for analysis. The pipeline should remove records that cannot be properly identified, replace missing temperature values using a value supplied by the driver program, generate city-level temperature statistics, identify days that satisfy a specified temperature threshold, and determine the cities having the highest average temperatures.

### OBJECTIVE

- Clean the incoming weather dataset by removing records that do not contain a valid date or city, and replace missing temperature values with a driver-provided value.
- Generate a city-level summary containing the average temperature and the number of recorded days for each city.
- Extract weather records where the temperature is greater than or equal to a specified threshold.
- Arrange cities according to their average temperature and return the required number of cities with the highest average temperatures.
- Ensure that all transformations preserve the required DataFrame schemas and produce outputs suitable for downstream reporting.

### FILE STRUCTURE

```
Project
├── data
│   └── weather.csv
├── main.py
├── solution.py
├── tests.py
└── requirements.txt
```

### PURPOSE OF FILES

- main.py - Driver program that creates the Spark session, loads the weather dataset, invokes the pipeline functions, displays the intermediate DataFrame results, and terminates the Spark session.
- solution.py - Python module to be completed by the candidate; it contains the four functions required for the weather analytics pipeline.
- data/weather.csv - Raw weather observations provided as input to the pipeline.
- tests.py - Contains automated validation cases used to verify the behavior of the implemented functions.
- requirements.txt - Contains the Python packages and dependencies required to execute the project.

### DATASET DESCRIPTION

The incoming weather feed can contain two different types of incomplete records. A record without a valid date or city cannot be reliably associated with a weather observation and must therefore be removed. A record with a missing temperature can still be retained because its temperature can be replaced with the value supplied to the cleaning function.

Schema — data/weather.csv

| Column Name | Data Type | Description                                                                                              |
|-------------|-----------|----------------------------------------------------------------------------------------------------------|
| date        | date      | Calendar date of the weather observation in YYYY-MM-DD format; the value may be missing in the raw feed. |
| city        | string    | Name of the city where the observation was recorded; the value may be missing.                           |
| temperature | double    | Temperature recorded for the city on that date in Celsius; the value may be missing.                     |

Sample Data — data/weather.csv

| date       | city      | temperature |
|------------|-----------|-------------|
| 2025-06-01 | Delhi     | 40.2        |
| 2025-06-01 | Chennai   | 34.3        |
| 2025-06-03 | Kolkata   | 31.8        |
| 2025-06-04 | Mumbai    | 29.8        |
| 2025-06-06 | Bengaluru | 25.3        |
| 2025-06-08 | Chennai   | 35.1        |
| 2025-06-11 | Mumbai    | (blank)     |
| 2025-06-11 | (blank)   | 30.0        |
| 2025-06-14 | Kolkata   | 32.6        |

## TASK

### IMPLEMENTATION REQUIREMENTS

#### FUNCTIONS TO IMPLEMENT IN SOLUTION.PY

##### Function 1 — clean\_weather(df, fill\_value)

#### PURPOSE

Prepare the raw weather DataFrame for analysis by removing observations that cannot be uniquely identified and filling unavailable temperature readings with the supplied replacement value.

#### PARAMETERS

df is the raw Spark DataFrame loaded from data/weather.csv by main.py. It contains the columns date, city, and temperature.

fill\_value is a floating-point value supplied by main.py. It represents the replacement temperature that should be used whenever the original temperature value is missing.

#### REQUIREMENT

Records where either date or city is missing must be removed because those observations cannot be properly identified.

After removing those records, any remaining missing temperature values must be replaced with fill\_value.

The function must retain the original input schema and column structure.

#### RETURNS

A cleaned Spark DataFrame containing the same three columns:

- date

- city
- temperature
- The resulting DataFrame must not contain null values in date, city, or temperature.

## Function 2 — summarize\_by\_city(df)

### PURPOSE

Create a city-level weather summary that can be used for reporting and downstream analytical dashboards.

### PARAMETERS

df is the cleaned Spark DataFrame returned by clean\_weather.

The DataFrame contains the columns date, city, and temperature, with no missing values.

### REQUIREMENT

Group the weather observations by city so that every city appears only once in the resulting DataFrame.

For each city:

- Calculate the average temperature.
- Round the average temperature to two decimal places.
- Calculate the total number of recorded days for that city.
- The number of recorded days should represent the count of weather observations available for that city.

### RETURNS

A Spark DataFrame containing the following columns:

- city
- avg\_temperature
- days\_count

### WHERE

avg\_temperature is a double value rounded to two decimal places.

days\_count is a long integer representing the number of observations.

## Function 3 — filter\_hot\_days(df, threshold)

### PURPOSE

Identify weather observations where the recorded temperature reaches or exceeds a specified temperature limit.

### PARAMETERS

df is the cleaned Spark DataFrame containing date, city, and temperature.

threshold is a floating-point value supplied by main.py. It represents the minimum temperature that qualifies an observation as a hot day.

### REQUIREMENT

Only retain records where:

- temperature  $\geq$  threshold
- The resulting records must be arranged chronologically according to date.
- The function must use exactly two DataFrame operations and preserve the input schema.

### RETURNS

A Spark DataFrame containing only the qualifying observations with the columns:

- date
- city
- temperature
- The returned records must be ordered by date.

#### Function 4 — top\_n\_hottest\_cities(summary\_df, n)

##### PURPOSE

Determine the cities with the highest average temperatures and return the requested number of cities.

##### PARAMETERS

summary\_df is the city-level summary Spark DataFrame returned by summarize\_by\_city. It contains city, avg\_temperature, and days\_count.

n is an integer supplied by main.py indicating how many cities should be included in the final result.

##### REQUIREMENT

Sort the cities from the highest average temperature to the lowest average temperature using avg\_temperature.

After sorting, retain only the first n rows.

The function must use exactly two DataFrame operations.

##### RETURNS

A Spark DataFrame containing the top n cities with the following columns:

- city
- avg\_temperature
- days\_count
- SAMPLE INPUT / OUTPUT

#### Function 1 — clean\_weather(df, fill\_value)

##### SAMPLE INPUT

```
fill_value = 27.0
```

| date       | city    | temperature |
|------------|---------|-------------|
| (blank)    | Jaipur  | 24.5        |
| 2025-06-15 | (blank) | 28.3        |
| 2025-06-16 | Kolkata | (blank)     |
| 2025-06-17 | Delhi   | 22.7        |

##### SAMPLE OUTPUT

| date       | city    | temperature |
|------------|---------|-------------|
| 2025-06-16 | Kolkata | 27.0        |
| 2025-06-17 | Delhi   | 22.7        |

The first record is removed because its date is missing.

The second record is removed because its city is missing.

The third record is retained, but its missing temperature is replaced with the supplied fill\_value of 27.0.

The fourth record remains unchanged.

## Function 2 — summarize\_by\_city(df)

### SAMPLE INPUT

| date       | city   | temperature |
|------------|--------|-------------|
| 2025-06-15 | Pune   | 20.0        |
| 2025-06-16 | Pune   | 30.0        |
| 2025-06-17 | Nagpur | 26.0        |

### SAMPLE OUTPUT

| city   | avg_temperature | days_count |
|--------|-----------------|------------|
| Pune   | 25.0            | 2          |
| Nagpur | 26.0            | 1          |

For Pune:

- Average temperature =  $(20.0 + 30.0) / 2$
- = 25.0

Therefore:

```
days_count = 2
avg_temperature = 25.0
```

For Nagpur:

- Average temperature = 26.0

```
days_count = 1
```

## Function 3 — filter\_hot\_days(df, threshold)

### SAMPLE INPUT

```
threshold = 35.0
```

| date       | city   | temperature |
|------------|--------|-------------|
| 2025-06-15 | Pune   | 26.5        |
| 2025-06-16 | Jaipur | 41.0        |
| 2025-06-17 | Nagpur | 37.0        |

### SAMPLE OUTPUT

| date       | city   | temperature |
|------------|--------|-------------|
| 2025-06-16 | Jaipur | 41.0        |
| 2025-06-17 | Nagpur | 37.0        |

The Pune record is removed because:

- $26.5 < 35.0$

The Jaipur record is retained because:

- $41.0 \geq 35.0$

The Nagpur record is retained because:

- $37.0 \geq 35.0$
- The resulting records are arranged by date.

**Function 4 — top\_n\_hottest\_cities(summary\_df, n)****SAMPLE INPUT**

```
n = 2
```

| city    | avg_temperature | days_count |
|---------|-----------------|------------|
| Jaipur  | 41.0            | 2          |
| Lucknow | 34.0            | 2          |
| Nagpur  | 37.5            | 4          |

**SAMPLE OUTPUT**

| city   | avg_temperature | days_count |
|--------|-----------------|------------|
| Jaipur | 41.0            | 2          |
| Nagpur | 37.5            | 4          |

## Q14. E-Commerce Order Value Monitor

Part 1 · PySpark — Easy — question 14 of 16

### PROBLEM STATEMENT

A growing online marketplace is experiencing an increase in unusually large customer orders across its digital sales channels. The operations team needs a daily Spark job that reviews the order feed, keeps only orders that were successfully completed, and highlights high-value purchasing activity at the customer level so analysts can quickly review potentially unusual orders. The data engineering team must build a compact and reliable PySpark pipeline that produces customer-level summaries suitable for direct use by the operations dashboard.

### OBJECTIVE

- Load the daily order feed from CSV and retain only the records whose status is COMPLETED.
- Summarise large-value orders per customer using a driver-supplied amount threshold.
- List the distinct regions represented in the order feed.
- Flag each order as large-value or not against a driver-supplied amount threshold.

### FILE STRUCTURE

```
Project
├── data
│   └── orders.csv
├── main.py
├── solution.py
├── tests.py
└── requirements.txt
```

### PURPOSE OF FILES

- main.py - Driver script that creates the Spark session, executes the pipeline, displays all intermediate DataFrames, and stops the session.
- solution.py - Module to be completed by the candidate; contains the four pipeline functions.
- data/orders.csv - Raw order feed used as the input to the pipeline.
- requirements.txt - Python dependencies required to execute the project.

### DATASET DESCRIPTION

Schema (data/orders.csv)

| Column Name  | Data Type | Description                                                         |
|--------------|-----------|---------------------------------------------------------------------|
| order_id     | string    | Unique identifier assigned to an order.                             |
| customer_id  | string    | Customer identifier associated with the order.                      |
| order_time   | timestamp | Date and time when the order was placed in YYYY-MM-DD HH:MM format. |
| total_amount | double    | Total monetary value of the order.                                  |
| region       | string    | Region from which the order was placed.                             |
| payment_mode | string    | Payment method used for the order (CARD, UPI, COD).                 |
| status       | string    | Order processing status (COMPLETED or CANCELLED).                   |

SAMPLE DATA (data/orders.csv)

| order_id | customer_id | order_time       | total_amount | region | payment_mode | status    |
|----------|-------------|------------------|--------------|--------|--------------|-----------|
| 0101     | C2001       | 2025-11-01 09:20 | 5200.0       | NORTH  | CARD         | COMPLETED |
| 0102     | C2001       | 2025-11-01 14:10 | 18500.0      | NORTH  | UPI          | COMPLETED |
| 0105     | C2002       | 2025-11-01 16:30 | 22300.0      | WEST   | CARD         | COMPLETED |
| 0106     | C2002       | 2025-11-02 10:45 | 3100.0       | WEST   | COD          | CANCELLED |
| 0108     | C2003       | 2025-11-02 13:20 | 27500.0      | SOUTH  | UPI          | COMPLETED |
| 0110     | C2004       | 2025-11-01 11:15 | 9400.0       | EAST   | UPI          | COMPLETED |
| 0113     | C2005       | 2025-11-01 13:40 | 16800.0      | NORTH  | CARD         | COMPLETED |
| 0120     | C2007       | 2025-11-02 11:30 | 20500.0      | WEST   | CARD         | COMPLETED |
| 0127     | C2009       | 2025-11-03 12:00 | 4600.0       | SOUTH  | COD          | COMPLETED |
| 0134     | C2012       | 2025-11-01 17:20 | 11700.0      | EAST   | UPI          | COMPLETED |

## TASK

### Function 1 — filter\_completed\_orders(orders\_df)

#### PURPOSE

Retain only the orders that were successfully completed by the marketplace.

#### PARAMETERS

orders\_df is the raw Spark DataFrame loaded from data/orders.csv by main.py, containing the seven columns order\_id, customer\_id, order\_time, total\_amount, region, payment\_mode and status.

#### REQUIREMENT

Rows whose status equals "COMPLETED" are retained and every other row is discarded; the schema must remain unchanged.

#### RETURNS

A Spark DataFrame with the same seven columns, containing only the rows where status is COMPLETED.

### Function 2 — summarize\_large\_orders\_by\_customer(orders\_df, threshold)

#### PURPOSE

Generate a customer-level summary that identifies large-value orders for operational review.

#### PARAMETERS

orders\_df is the completed-order Spark DataFrame returned by filter\_completed\_orders. threshold is a float supplied by main.py as a configurable value representing the minimum total\_amount required for an order to be considered large-value.

#### REQUIREMENT

Only rows whose total\_amount is at or above the threshold are retained. The remaining rows are then grouped by customer\_id to calculate the number of large-value orders and the total value of those orders. Customers with no large-value orders must not appear in the output.

## RETURNS

A Spark DataFrame with the columns:

- customer\_id
- large\_order\_count (long)
- large\_order\_amount (double)

### Function 3 — list\_distinct\_regions(orders\_df)

#### PURPOSE

Identify all unique regions represented in the order feed.

#### PARAMETERS

orders\_df is a Spark DataFrame containing the order columns, including region.

#### REQUIREMENT

The distinct values from the region column must be returned. The function must use exactly two DataFrame operations.

#### RETURNS

A single-column Spark DataFrame named region, containing each distinct region exactly once.

### Function 4 — flag\_large\_orders(orders\_df, threshold)

#### PURPOSE

Mark each order as large-value or regular-value for downstream operational review.

#### PARAMETERS

orders\_df is a Spark DataFrame containing the order columns, including total\_amount. threshold is a float supplied by main.py, representing the minimum total\_amount considered large-value.

#### REQUIREMENT

A boolean is\_large\_order column must be added. It must be set to true when total\_amount is at or above the threshold and false otherwise. A compact set of columns — order\_id, customer\_id, total\_amount and is\_large\_order — must be returned. The function must use exactly two DataFrame operations.

#### RETURNS

A Spark DataFrame with the columns:

- order\_id
- customer\_id
- total\_amount
- is\_large\_order (boolean)

#### SAMPLE I/O

### Function 1 — filter\_completed\_orders(orders\_df)

#### SAMPLE INPUT

| order_id | customer_id | order_time          | total_amount | region | payment_mode | status    |
|----------|-------------|---------------------|--------------|--------|--------------|-----------|
| 0101     | C2001       | 2025-11-01<br>09:20 | 5200.0       | NORTH  | CARD         | COMPLETED |
| 0102     | C2001       | 2025-11-01<br>14:10 | 18500.0      | NORTH  | UPI          | COMPLETED |

| order_id | customer_id | order_time       | total_amount | region | payment_mode | status    |
|----------|-------------|------------------|--------------|--------|--------------|-----------|
| 0104     | C2002       | 2025-11-01 08:50 | 7600.0       | WEST   | CARD         | COMPLETED |
| 0106     | C2002       | 2025-11-02 10:45 | 3100.0       | WEST   | COD          | CANCELLED |
| 0107     | C2003       | 2025-11-01 10:15 | 6800.50      | SOUTH  | UPI          | COMPLETED |
| 0131     | C2011       | 2025-11-01 14:30 | 11200.0      | NORTH  | CARD         | COMPLETED |
| 0133     | C2011       | 2025-11-03 11:45 | 24500.0      | NORTH  | UPI          | COMPLETED |

## SAMPLE OUTPUT

| order_id | customer_id | order_time       | total_amount | region | payment_mode | status    |
|----------|-------------|------------------|--------------|--------|--------------|-----------|
| 0101     | C2001       | 2025-11-01 09:20 | 5200.0       | NORTH  | CARD         | COMPLETED |
| 0102     | C2001       | 2025-11-01 14:10 | 18500.0      | NORTH  | UPI          | COMPLETED |
| 0104     | C2002       | 2025-11-01 08:50 | 7600.0       | WEST   | CARD         | COMPLETED |
| 0107     | C2003       | 2025-11-01 10:15 | 6800.50      | SOUTH  | UPI          | COMPLETED |
| 0131     | C2011       | 2025-11-01 14:30 | 11200.0      | NORTH  | CARD         | COMPLETED |
| 0133     | C2011       | 2025-11-03 11:45 | 24500.0      | NORTH  | UPI          | COMPLETED |

## Function 2 — summarize\_large\_orders\_by\_customer(orders\_df, threshold)

Sample Input (threshold = 15000.0)

| order_id | customer_id | order_time       | total_amount | region | payment_mode | status    |
|----------|-------------|------------------|--------------|--------|--------------|-----------|
| 0101     | C2001       | 2025-11-01 09:20 | 5200.0       | NORTH  | CARD         | COMPLETED |
| 0102     | C2001       | 2025-11-01 14:10 | 18500.0      | NORTH  | UPI          | COMPLETED |
| 0104     | C2002       | 2025-11-01 08:50 | 7600.0       | WEST   | CARD         | COMPLETED |
| 0107     | C2003       | 2025-11-01 10:15 | 6800.50      | SOUTH  | UPI          | COMPLETED |
| 0131     | C2011       | 2025-11-01 14:30 | 11200.0      | NORTH  | CARD         | COMPLETED |
| 0133     | C2011       | 2025-11-03 11:45 | 24500.0      | NORTH  | UPI          | COMPLETED |

## SAMPLE OUTPUT

| customer_id | large_order_count | large_order_amount |
|-------------|-------------------|--------------------|
| C2001       | 1                 | 18500.0            |
| C2011       | 1                 | 24500.0            |

## Function 3 — list\_distinct\_regions(orders\_df)

### SAMPLE INPUT

| order_id | customer_id | order_time       | total_amount | region | payment_mode | status    |
|----------|-------------|------------------|--------------|--------|--------------|-----------|
| 0101     | C2001       | 2025-11-01 09:20 | 5200.0       | NORTH  | CARD         | COMPLETED |
| 0102     | C2001       | 2025-11-01 14:10 | 18500.0      | NORTH  | UPI          | COMPLETED |
| 0104     | C2002       | 2025-11-01 08:50 | 7600.0       | WEST   | CARD         | COMPLETED |
| 0107     | C2003       | 2025-11-01 10:15 | 6800.50      | SOUTH  | UPI          | COMPLETED |
| 0131     | C2011       | 2025-11-01 14:30 | 11200.0      | NORTH  | CARD         | COMPLETED |
| 0133     | C2011       | 2025-11-03 11:45 | 24500.0      | NORTH  | UPI          | COMPLETED |

## SAMPLE OUTPUT

region

**NORTH**

**SOUTH**

**WEST**

## Function 4 — flag\_large\_orders(orders\_df, threshold)

Sample Input (threshold = 15000.0)

| order_id | customer_id | order_time       | total_amount | region | payment_mode | status    |
|----------|-------------|------------------|--------------|--------|--------------|-----------|
| 0101     | C2001       | 2025-11-01 09:20 | 5200.0       | NORTH  | CARD         | COMPLETED |
| 0102     | C2001       | 2025-11-01 14:10 | 18500.0      | NORTH  | UPI          | COMPLETED |
| 0104     | C2002       | 2025-11-01 08:50 | 7600.0       | WEST   | CARD         | COMPLETED |
| 0107     | C2003       | 2025-11-01 10:15 | 6800.50      | SOUTH  | UPI          | COMPLETED |
| 0131     | C2011       | 2025-11-01 14:30 | 11200.0      | NORTH  | CARD         | COMPLETED |
| 0133     | C2011       | 2025-11-03 11:45 | 24500.0      | NORTH  | UPI          | COMPLETED |

## SAMPLE OUTPUT

| order_id | customer_id | total_amount | is_large_order |
|----------|-------------|--------------|----------------|
| 0101     | C2001       | 5200.0       | false          |
| 0102     | C2001       | 18500.0      | true           |
| 0104     | C2002       | 7600.0       | false          |
| 0107     | C2003       | 6800.50      | false          |
| 0131     | C2011       | 11200.0      | false          |
| 0133     | C2011       | 24500.0      | true           |

## Q15. Food Delivery Service Quality Analyzer

Part 1 · PySpark — Easy — question 15 of 16

### PROBLEM STATEMENT

A food delivery company is monitoring delivery performance across different delivery partners and service zones. Operations teams have reported an increase in delayed and cancelled orders in certain zones, but the existing dashboard is too coarse-grained to identify problematic areas quickly.

The data engineering team must build a daily PySpark pipeline that ingests raw delivery records, standardizes the diagnostic columns, keeps only meaningful delivery outcomes, produces delivery reliability metrics per partner and per zone, and ranks the worst-performing zones for the operations team to investigate.

### OBJECTIVE

- Ingest the raw delivery records from CSV into Spark using an explicit schema.
- Standardize the diagnostic columns by selecting and renaming them with clean aliases.
- Filter out non-diagnostic order statuses, keeping only DELIVERED and CANCELLED orders.
- Summarize delivery reliability per delivery partner.
- Aggregate per-zone delivery quality metrics.
- Rank the worst-performing zones by cancellation rate.

### FILE STRUCTURE

```
Project
├── data
│   └── delivery_logs.csv
├── main.py
├── solution.py
├── tests.py
└── requirements.txt
```

### PURPOSE OF FILES

- main.py – Driver script that builds the Spark session, runs the pipeline, prints all intermediate DataFrames, and stops the session.
- solution.py – Module the candidate completes; contains the six pipeline functions.
- data/delivery\_logs.csv – Daily food delivery records used as input to the pipeline.
- requirements.txt – Python dependencies required to run the project.

### DATASET DESCRIPTION

Schema (data/delivery\_logs.csv)

| Column Name       | Data Type | Description                                                    |
|-------------------|-----------|----------------------------------------------------------------|
| order_id          | integer   | Unique identifier of the food delivery order.                  |
| partner           | string    | Delivery partner handling the order.                           |
| delivery_time_min | integer   | Time taken to complete or attempt the delivery, in minutes.    |
| order_status      | string    | Outcome of the order such as DELIVERED, CANCELLED or ASSIGNED. |
| zone              | string    | Delivery zone where the order was placed.                      |

SAMPLE DATA (data/delivery\_logs.csv)

| order_id | partner   | delivery_time_min | order_status | zone   |
|----------|-----------|-------------------|--------------|--------|
| 1001     | SwiftGo   | 32                | DELIVERED    | ZONE_A |
| 1002     | QuickKart | 28                | DELIVERED    | ZONE_A |
| 1003     | SwiftGo   | 45                | DELIVERED    | ZONE_B |
| 1004     | FastDrop  | 50                | CANCELLED    | ZONE_C |
| 1005     | QuickKart | 35                | DELIVERED    | ZONE_B |
| 1006     | FastDrop  | 42                | CANCELLED    | ZONE_C |
| 1007     | SwiftGo   | 30                | CANCELLED    | ZONE_A |
| 1008     | FastDrop  | 38                | DELIVERED    | ZONE_C |
| 1009     | QuickKart | 25                | CANCELLED    | ZONE_B |
| 1010     | SwiftGo   | 40                | ASSIGNED     | ZONE_A |

## TASK

### IMPLEMENTATION REQUIREMENTS

#### FUNCTIONS TO IMPLEMENT IN SOLUTION.PY

##### Function 1 — load\_orders(spark, orders\_path)

###### PURPOSE

Load the delivery records into a typed Spark DataFrame with a clearly defined set of columns.

###### PARAMETERS

spark – The active SparkSession created in main.py.

orders\_path – The filesystem path to the delivery logs CSV, for example data/delivery\_logs.csv.

###### REQUIREMENT

Load the five fields with their declared types:

- order\_id → integer
- partner → string
- delivery\_time\_min → integer
- order\_status → string
- zone → string

###### RETURNS

A Spark DataFrame containing the five columns in the same order as the schema.

##### Function 2 — standardize\_orders(df\_orders)

###### PURPOSE

Standardize the delivery diagnostic columns so that downstream processing uses clean and consistent column names.

###### PARAMETERS

df\_orders is the Spark DataFrame returned by load\_orders.

###### REQUIREMENT

Select and rename the columns as follows:

| Original Column   | Output Column    |
|-------------------|------------------|
| order_id          | delivery_id      |
| partner           | delivery_partner |
| delivery_time_min | duration_min     |
| order_status      | delivery_status  |
| zone              | service_zone     |

The resulting DataFrame must contain only these five columns in this order:

- delivery\_id
- delivery\_partner
- duration\_min
- delivery\_status
- service\_zone

## RETURNS

A standardized Spark DataFrame with the renamed columns.

### Function 3 — filter\_valid\_deliveries(df\_standardized)

#### PURPOSE

Keep only delivery outcomes that are meaningful for delivery-quality analysis.

#### PARAMETERS

df\_standardized is the standardized DataFrame returned by standardize\_orders.

#### REQUIREMENT

Keep only records where delivery\_status is:

#### DELIVERED

#### CANCELLED

Rows with any other status must be removed.

#### RETURNS

A Spark DataFrame containing only DELIVERED and CANCELLED records.

### Function 4 — summarize\_by\_partner(df\_filtered)

#### PURPOSE

Summarize delivery reliability for each delivery partner.

#### PARAMETERS

df\_filtered is the filtered Spark DataFrame containing only meaningful delivery outcomes.

#### REQUIREMENT

For each delivery\_partner, calculate:

- total\_orders – total number of orders handled by the partner.
- cancelled\_orders – number of orders with status CANCELLED.
- cancel\_rate – cancelled orders divided by total orders.

The result must contain:

- delivery\_partner

- total\_orders
- cancelled\_orders
- cancel\_rate

## RETURNS

A Spark DataFrame containing one row per delivery partner.

### Function 5 — aggregate\_quality\_by\_zone(df\_filtered)

#### PURPOSE

Calculate delivery-quality metrics for each service zone so that operations teams can identify problematic zones.

#### PARAMETERS

df\_filtered is the filtered Spark DataFrame containing only DELIVERED and CANCELLED records.

#### REQUIREMENT

For each service\_zone, calculate:

- total\_orders – total number of orders.
- cancelled\_orders – number of cancelled orders.
- cancel\_rate – cancelled orders divided by total orders.
- avg\_duration\_min – average delivery duration.

## RETURNS

A Spark DataFrame with the following four columns:

- service\_zone
- total\_orders
- cancelled\_orders
- cancel\_rate
- avg\_duration\_min

### Function 6 — rank\_worst\_zones(zone\_metrics\_df, top\_n)

#### PURPOSE

Identify the service zones with the highest cancellation rates so that the operations team can prioritize investigation.

#### PARAMETERS

zone\_metrics\_df – The aggregated Spark DataFrame returned by aggregate\_quality\_by\_zone.

top\_n – An integer supplied by main.py indicating how many zones should be returned.

#### REQUIREMENT

Rank the service zones:

- By cancel\_rate from highest to lowest.
- Break ties using service\_zone in ascending order.
- Return only the first top\_n zones.

## RETURNS

A Spark DataFrame with the same five columns:

- service\_zone

- total\_orders
- cancelled\_orders
- cancel\_rate
- avg\_duration\_min
- limited to the requested top\_n zones.

## SAMPLE INPUT AND OUTPUT

### Function 1 — load\_orders(spark, orders\_path)

#### SAMPLE INPUT

order\_id,partner,delivery\_time\_min,order\_status,zone

1001,SwiftGo,32,DELIVERED,ZONE\_A

1002,QuickKart,28,DELIVERED,ZONE\_A

1003,SwiftGo,45,DELIVERED,ZONE\_B

1004,FastDrop,50,CANCELLED,ZONE\_C

#### SAMPLE OUTPUT

| order_id | partner   | delivery_time_min | order_status | zone   |
|----------|-----------|-------------------|--------------|--------|
| 1001     | SwiftGo   | 32                | DELIVERED    | ZONE_A |
| 1002     | QuickKart | 28                | DELIVERED    | ZONE_A |
| 1003     | SwiftGo   | 45                | DELIVERED    | ZONE_B |
| 1004     | FastDrop  | 50                | CANCELLED    | ZONE_C |

### Function 2 — standardize\_orders(df\_orders)

#### SAMPLE INPUT

| order_id | partner   | delivery_time_min | order_status | zone   |
|----------|-----------|-------------------|--------------|--------|
| 1001     | SwiftGo   | 32                | DELIVERED    | ZONE_A |
| 1002     | QuickKart | 28                | DELIVERED    | ZONE_A |
| 1003     | SwiftGo   | 45                | DELIVERED    | ZONE_B |
| 1004     | FastDrop  | 50                | CANCELLED    | ZONE_C |

#### SAMPLE OUTPUT

| delivery_id | delivery_partner | duration_min | delivery_status | service_zone |
|-------------|------------------|--------------|-----------------|--------------|
| 1001        | SwiftGo          | 32           | DELIVERED       | ZONE_A       |
| 1002        | QuickKart        | 28           | DELIVERED       | ZONE_A       |
| 1003        | SwiftGo          | 45           | DELIVERED       | ZONE_B       |
| 1004        | FastDrop         | 50           | CANCELLED       | ZONE_C       |

### Function 3 — filter\_valid\_deliveries(df\_standardized)

#### SAMPLE INPUT

| delivery_id | delivery_partner | duration_min | delivery_status | service_zone |
|-------------|------------------|--------------|-----------------|--------------|
| 1001        | SwiftGo          | 32           | DELIVERED       | ZONE_A       |
| 1002        | QuickKart        | 28           | DELIVERED       | ZONE_A       |
| 1003        | SwiftGo          | 45           | DELIVERED       | ZONE_B       |

| delivery_id | delivery_partner | duration_min | delivery_status | service_zone |
|-------------|------------------|--------------|-----------------|--------------|
| 1004        | FastDrop         | 50           | CANCELLED       | ZONE_C       |
| 1005        | QuickKart        | 35           | DELIVERED       | ZONE_B       |
| 1006        | FastDrop         | 42           | CANCELLED       | ZONE_C       |
| 1007        | SwiftGo          | 30           | CANCELLED       | ZONE_A       |
| 1008        | FastDrop         | 38           | DELIVERED       | ZONE_C       |
| 1009        | QuickKart        | 25           | CANCELLED       | ZONE_B       |
| 1010        | SwiftGo          | 40           | ASSIGNED        | ZONE_A       |

## SAMPLE OUTPUT

| delivery_id | delivery_partner | duration_min | delivery_status | service_zone |
|-------------|------------------|--------------|-----------------|--------------|
| 1001        | SwiftGo          | 32           | DELIVERED       | ZONE_A       |
| 1002        | QuickKart        | 28           | DELIVERED       | ZONE_A       |
| 1003        | SwiftGo          | 45           | DELIVERED       | ZONE_B       |
| 1004        | FastDrop         | 50           | CANCELLED       | ZONE_C       |
| 1005        | QuickKart        | 35           | DELIVERED       | ZONE_B       |
| 1006        | FastDrop         | 42           | CANCELLED       | ZONE_C       |
| 1007        | SwiftGo          | 30           | CANCELLED       | ZONE_A       |
| 1008        | FastDrop         | 38           | DELIVERED       | ZONE_C       |
| 1009        | QuickKart        | 25           | CANCELLED       | ZONE_B       |

## Function 4 — summarize\_by\_partner(df\_filtered)

### SAMPLE INPUT

| delivery_id | delivery_partner | duration_min | delivery_status | service_zone |
|-------------|------------------|--------------|-----------------|--------------|
| 1001        | SwiftGo          | 32           | DELIVERED       | ZONE_A       |
| 1002        | QuickKart        | 28           | DELIVERED       | ZONE_A       |
| 1003        | SwiftGo          | 45           | DELIVERED       | ZONE_B       |
| 1004        | FastDrop         | 50           | CANCELLED       | ZONE_C       |
| 1005        | QuickKart        | 35           | DELIVERED       | ZONE_B       |
| 1006        | FastDrop         | 42           | CANCELLED       | ZONE_C       |
| 1007        | SwiftGo          | 30           | CANCELLED       | ZONE_A       |
| 1008        | FastDrop         | 38           | DELIVERED       | ZONE_C       |
| 1009        | QuickKart        | 25           | CANCELLED       | ZONE_B       |

## SAMPLE OUTPUT

| delivery_partner | total_orders | cancelled_orders | cancel_rate        |
|------------------|--------------|------------------|--------------------|
| FastDrop         | 3            | 2                | 0.6666666666666666 |
| QuickKart        | 3            | 1                | 0.3333333333333333 |
| SwiftGo          | 3            | 1                | 0.3333333333333333 |

## Function 5 — aggregate\_quality\_by\_zone(df\_filtered)

### SAMPLE INPUT

| delivery_id | delivery_partner | duration_min | delivery_status | service_zone |
|-------------|------------------|--------------|-----------------|--------------|
| 1001        | SwiftGo          | 32           | DELIVERED       | ZONE_A       |
| 1002        | QuickKart        | 28           | DELIVERED       | ZONE_A       |
| 1003        | SwiftGo          | 45           | DELIVERED       | ZONE_B       |
| 1004        | FastDrop         | 50           | CANCELLED       | ZONE_C       |
| 1005        | QuickKart        | 35           | DELIVERED       | ZONE_B       |
| 1006        | FastDrop         | 42           | CANCELLED       | ZONE_C       |
| 1007        | SwiftGo          | 30           | CANCELLED       | ZONE_A       |
| 1008        | FastDrop         | 38           | DELIVERED       | ZONE_C       |
| 1009        | QuickKart        | 25           | CANCELLED       | ZONE_B       |

## SAMPLE OUTPUT

| service_zone | total_orders | cancelled_orders | cancel_rate        | avg_duration_min   |
|--------------|--------------|------------------|--------------------|--------------------|
| ZONE_A       | 3            | 1                | 0.3333333333333333 | 30.0               |
| ZONE_B       | 3            | 1                | 0.3333333333333333 | 35.0               |
| ZONE_C       | 3            | 2                | 0.6666666666666666 | 43.333333333333336 |

## Function 6 — rank\_worst\_zones(zone\_metrics\_df, top\_n)

Sample Input (top\_n = 2)

| service_zone | total_orders | cancelled_orders | cancel_rate        | avg_duration_min   |
|--------------|--------------|------------------|--------------------|--------------------|
| ZONE_A       | 3            | 1                | 0.3333333333333333 | 30.0               |
| ZONE_B       | 3            | 1                | 0.3333333333333333 | 35.0               |
| ZONE_C       | 3            | 2                | 0.6666666666666666 | 43.333333333333336 |

## SAMPLE OUTPUT

| service_zone | total_orders | cancelled_orders | cancel_rate        | avg_duration_min   |
|--------------|--------------|------------------|--------------------|--------------------|
| ZONE_C       | 3            | 2                | 0.6666666666666666 | 43.333333333333336 |
| ZONE_A       | 3            | 1                | 0.3333333333333333 | 30.0               |

## Q16. Food Delivery Order Activity and Frequent Customer Flagging

Part 1 · PySpark — Easy — question 16 of 16

### PROBLEM STATEMENT

A food delivery platform wants to understand how customers place orders across different days and hours and identify customers who frequently use the service. The platform maintains a daily order fact file containing individual food orders placed through different delivery zones. Analysts have requested a compact PySpark pipeline that summarizes order activity by day and hour, counts the total orders made by each customer, and flags frequent customers based on a configurable order-count threshold.

The data engineering team has been asked to build a small, reproducible PySpark pipeline that loads the order data, aggregates order activity at a day-and-hour level, calculates customer-level order counts, and identifies customers who meet the required frequency threshold.

### OBJECTIVE

- Ingest the food delivery orders CSV into Spark, taking column names from the header and inferring field types.
- Aggregate order activity at (order\_date, order\_hour) granularity and generate a per-group order count.
- Count the number of recorded orders for each customer.
- Flag frequent customers: is\_frequent is 1 when a customer's order\_count meets a driver-supplied threshold, and 0 otherwise.

### FILE STRUCTURE

```
Project
|-- data
|   |-- orders.csv
|-- main.py
|-- solution.py
|-- tests.py
|-- requirements.txt
```

### PURPOSE OF FILES

- main.py - Driver script that builds the Spark session, runs the pipeline, prints all intermediate DataFrames, and stops the session.
- solution.py - Module the candidate completes; contains the four pipeline functions.
- data/orders.csv - Food delivery order fact file with one row per order.
- requirements.txt - Python dependencies required to run the project.

### DATASET DESCRIPTION

Schema (data/orders.csv)

| Column Name   | Data Type | Description                                                                        |
|---------------|-----------|------------------------------------------------------------------------------------|
| order_id      | string    | Unique identifier for the food delivery order.                                     |
| customer_id   | string    | Identifier of the customer who placed the order.                                   |
| order_date    | date      | Date on which the order was placed, inferred from YYYY-MM-DD.                      |
| order_hour    | integer   | Hour of the order in 24-hour format (0 to 23).                                     |
| delivery_zone | string    | Delivery zone where the order was placed, such as Central, NorthZone or SouthZone. |

SAMPLE DATA (data/orders.csv)

| order_id | customer_id | order_date | order_hour | delivery_zone |
|----------|-------------|------------|------------|---------------|
| 0001     | C003        | 2025-09-20 | 12         | Central       |
| 0002     | C010        | 2025-09-21 | 13         | NorthZone     |
| 0003     | C008        | 2025-09-22 | 12         | Central       |
| 0004     | C019        | 2025-09-22 | 19         | SouthZone     |
| 0007     | C002        | 2025-09-26 | 19         | Central       |
| 0008     | C012        | 2025-09-26 | 19         | NorthZone     |
| 0009     | C022        | 2025-09-26 | 19         | Central       |
| 0013     | C001        | 2025-09-28 | 20         | Central       |
| 0022     | C002        | 2025-09-30 | 13         | Central       |
| 0029     | C001        | 2025-09-30 | 20         | NorthZone     |

## TASK

### IMPLEMENTATION REQUIREMENTS

#### FUNCTIONS TO IMPLEMENT IN SOLUTION.PY

##### Function 1 — load\_orders(spark, orders\_csv\_path)

###### PURPOSE

Load the food delivery orders CSV into a typed Spark DataFrame.

###### PARAMETERS

spark is the active SparkSession created in main.py.

orders\_csv\_path is the filesystem path to the orders CSV, for example:

- data/orders.csv

###### REQUIREMENT

Load the order data so that the column names come from the file header and each field takes the type implied by its values.

The data must be returned as-is, with no further transformations.

###### RETURNS

A Spark DataFrame with five columns:

- order\_id - string
- customer\_id - string
- order\_date - date
- order\_hour - integer
- delivery\_zone - string

##### Function 2 — aggregate\_orders\_by\_day\_hour(orders\_df)

###### PURPOSE

Aggregate food delivery activity at (order\_date, order\_hour) granularity so downstream reports can identify busy ordering periods.

###### PARAMETERS

orders\_df is the Spark DataFrame returned by load\_orders, containing the five order columns.

## REQUIREMENT

For each combination of `order_date` and `order_hour`, report `order_count`, representing the number of orders recorded during that day-and-hour window.

## RETURNS

A Spark DataFrame with three columns:

- `order_date` - date
- `order_hour` - integer
- `order_count` - long, representing the number of orders in that group

### Function 3 — `summarize_by_customer(orders_df)`

## PURPOSE

Count how many orders each customer has placed, producing a per-customer order total.

## PARAMETERS

`orders_df` is the Spark DataFrame returned by `load_orders`, containing the five order columns.

## REQUIREMENT

For each `customer_id`, report `order_count`, representing the total number of orders recorded for that customer.

## RETURNS

A Spark DataFrame with two columns:

- `customer_id` - string
- `order_count` - long, representing the number of orders placed by that customer

### Function 4 — `flag_frequent_customers(customer_counts_df, threshold)`

## PURPOSE

Flag customers who place orders frequently based on whether their `order_count` meets a driver-supplied threshold.

## PARAMETERS

`customer_counts_df` is the per-customer Spark DataFrame returned by `summarize_by_customer`, containing `customer_id` and `order_count`.

`threshold` is an integer supplied by `main.py` giving the minimum order count for a customer to be considered frequent.

## REQUIREMENT

Add a new integer field `is_frequent` that is:

- 1 when a customer's `order_count` is at or above the supplied threshold
- 0 otherwise
- The original fields must be preserved.

## RETURNS

A Spark DataFrame with three columns:

- `customer_id` - string
- `order_count` - long
- `is_frequent` - integer (0 or 1)

## SAMPLE INPUT AND OUTPUT

**Function 1 — load\_orders(spark, orders\_csv\_path)****SAMPLE INPUT**

order\_id,customer\_id,order\_date,order\_hour,delivery\_zone

OD01,CU01,2024-08-15,12,Central

OD02,CU02,2024-08-15,12,Central

OD03,CU03,2024-08-15,19,NorthZone

**SAMPLE OUTPUT**

| order_id | customer_id | order_date | order_hour | delivery_zone |
|----------|-------------|------------|------------|---------------|
| OD01     | CU01        | 2024-08-15 | 12         | Central       |
| OD02     | CU02        | 2024-08-15 | 12         | Central       |
| OD03     | CU03        | 2024-08-15 | 19         | NorthZone     |

**Function 2 — aggregate\_orders\_by\_day\_hour(orders\_df)****SAMPLE INPUT**

| order_id | customer_id | order_date | order_hour | delivery_zone |
|----------|-------------|------------|------------|---------------|
| OD01     | CU01        | 2024-08-15 | 12         | Central       |
| OD02     | CU02        | 2024-08-15 | 12         | Central       |
| OD03     | CU03        | 2024-08-15 | 19         | NorthZone     |
| OD04     | CU04        | 2024-08-15 | 12         | Central       |
| OD05     | CU01        | 2024-08-16 | 13         | Central       |

**SAMPLE OUTPUT**

| order_date | order_hour | order_count |
|------------|------------|-------------|
| 2024-08-15 | 12         | 3           |
| 2024-08-15 | 19         | 1           |
| 2024-08-16 | 13         | 1           |

**Function 3 — summarize\_by\_customer(orders\_df)****SAMPLE INPUT**

| order_id | customer_id | order_date | order_hour | delivery_zone |
|----------|-------------|------------|------------|---------------|
| OD01     | CU01        | 2024-08-15 | 12         | Central       |
| OD02     | CU02        | 2024-08-15 | 12         | Central       |
| OD03     | CU03        | 2024-08-15 | 19         | NorthZone     |
| OD04     | CU01        | 2024-08-15 | 12         | Central       |
| OD05     | CU01        | 2024-08-16 | 13         | Central       |

**SAMPLE OUTPUT**

| customer_id | order_count |
|-------------|-------------|
| CU01        | 3           |
| CU02        | 1           |
| CU03        | 1           |

**Function 4 — flag\_frequent\_customers(customer\_counts\_df, threshold)**

Sample Input (threshold = 2)

| customer_id | order_count |
|-------------|-------------|
| CU01        | 3           |
| CU02        | 1           |
| CU03        | 1           |
| CU04        | 2           |

**SAMPLE OUTPUT**

| customer_id | order_count | is_frequent |
|-------------|-------------|-------------|
| CU01        | 3           | 1           |
| CU02        | 1           | 0           |
| CU03        | 1           | 0           |
| CU04        | 2           | 1           |

**IMPORTANT IMPLEMENTATION NOTES**

The function names and parameters must remain exactly as specified.

Do not modify the input CSV data.

load\_orders() should use the CSV header and infer the appropriate field types.

aggregate\_orders\_by\_day\_hour() must group by both order\_date and order\_hour.

summarize\_by\_customer() must group by customer\_id.

flag\_frequent\_customers() must preserve customer\_id and order\_count.

A customer whose order\_count is exactly equal to the threshold must receive is\_frequent = 1.

The implementation must use PySpark DataFrame operations.

**PART 2**

## PySpark — Medium

Explicit schemas, multi-dataset joins, window functions, Spark SQL and multi-level aggregation

**In this track — 16 questions**

|     |                                                                       |
|-----|-----------------------------------------------------------------------|
| Q1  | Airline Delay Monitoring and Passenger Refund Analytics               |
| Q2  | Railway Delay and Passenger Refund Analytics                          |
| Q3  | Restaurant Order Revenue and Cancellation Analytics                   |
| Q4  | Online Grocery Order Revenue and Delivery Performance Analytics       |
| Q5  | Retail Store Inventory Consumption & Replenishment Forecasting System |
| Q6  | Fuel Station Consumption & Tank Refill Forecasting System             |
| Q7  | E-Commerce Customer Purchase Analytics & Customer Segmentation        |
| Q8  | Hospital Patient Billing Analytics & Patient Segmentation             |
| Q9  | E-Commerce Order Fulfillment Delay Analytics                          |
| Q10 | Airline Baggage Handling Delay Analytics                              |
| Q11 | Warehouse Inventory KPI Analysis                                      |
| Q12 | Hotel Booking Revenue KPI Analysis                                    |
| Q13 | City-Level Ride Service Analytics                                     |
| Q14 | Delivery Performance Analyzer                                         |
| Q15 | Flight Reservation Cancellation Analytics                             |
| Q16 | Employee Training Assessment Analysis                                 |

## Q1. Airline Delay Monitoring and Passenger Refund Analytics

Part 2 · PySpark — Medium — question 1 of 16

### PROBLEM STATEMENT

A domestic airline maintains operational records for its scheduled flights between major Indian cities. The analytics team needs to monitor flight punctuality and estimate passenger refund obligations whenever flights experience delays.

The airline stores flight information such as flight identifier, route identifier, scheduled departure date, passenger count, delay duration, and current flight status. A separate route master contains airport information, route distance, aircraft type, and standard fare.

The operations team requires a PySpark data-processing pipeline that validates flight records, enriches them with route information, categorizes delays according to predefined thresholds, calculates passenger refunds, and generates a route-wise summary of flight performance and refund expenses.

### OBJECTIVE

- Load flight and route CSV files using explicitly defined PySpark schemas.
- Remove invalid flight records such as missing flight IDs, cancelled flights, and flights having zero or negative passengers.
- Join valid flight records with their corresponding route information.
- Convert negative delay values to zero and classify flights based on delay duration.
- Calculate passenger refund amounts according to the delay category.
- Generate a route-level summary containing total flights, average delay, and total refund amount.

### FILE STRUCTURE

```
Project
├── data
│   ├── flights.csv
│   └── routes.csv
├── main.py
├── solution.py
├── tests.py
└── requirements.txt
```

### PURPOSE OF FILES

- main.py – Driver program that loads both datasets, invokes the functions, and displays intermediate results.
- solution.py – Contains six PySpark functions that must be implemented.
- tests.py – Contains automated tests.
- requirements.txt – Contains required Python dependencies.
- data/flights.csv – Raw flight transaction records.
- data/routes.csv – Route reference information.

### DATASET DESCRIPTION

File: flights.csv

### SCHEMA

| Column Name     | Data Type | Description                                    |
|-----------------|-----------|------------------------------------------------|
| flight_id       | string    | Unique identifier assigned to a flight         |
| route_id        | string    | Identifier of the route followed by the flight |
| departure_date  | date      | Scheduled departure date                       |
| passenger_count | integer   | Number of passengers booked                    |

| Column Name   | Data Type | Description                                  |
|---------------|-----------|----------------------------------------------|
| delay_minutes | integer   | Delay duration in minutes                    |
| flight_status | string    | Flight status such as Scheduled or Cancelled |

## SAMPLE DATA

| flight_id | route_id | departure_date | passenger_count | delay_minutes | flight_status |
|-----------|----------|----------------|-----------------|---------------|---------------|
| FL2001    | R2001    | 2025-11-01     | 145             | 0             | Scheduled     |
| FL2002    | R2002    | 2025-11-01     | 132             | 20            | Scheduled     |
| FL2003    | R2003    | 2025-11-02     | 118             | 65            | Scheduled     |
| FL2004    | R2004    | 2025-11-02     | 105             | 140           | Scheduled     |
| FL2005    | R2001    | 2025-11-03     | 150             | -8            | Scheduled     |
| FL2006    | R2002    | 2025-11-03     | 120             | 0             | Cancelled     |

File: routes.csv

## SCHEMA

| Column Name       | Data Type | Description                          |
|-------------------|-----------|--------------------------------------|
| route_id          | string    | Unique identifier for a route        |
| departure_airport | string    | IATA code of the origin airport      |
| arrival_airport   | string    | IATA code of the destination airport |
| route_distance_km | integer   | Distance covered by the route        |
| aircraft_type     | string    | Aircraft assigned to the route       |
| base_fare         | integer   | Standard fare for the route          |

## SAMPLE DATA

| route_id | departure_airport | arrival_airport | route_distance_km | aircraft_type | base_fare |
|----------|-------------------|-----------------|-------------------|---------------|-----------|
| R2001    | DEL               | BOM             | 1148              | A320          | 5200      |
| R2002    | DEL               | BLR             | 1740              | A320          | 6100      |
| R2003    | BOM               | MAA             | 1032              | B737          | 4700      |
| R2004    | DEL               | HYD             | 1264              | B737          | 4900      |

## TASK

### FUNCTIONS TO IMPLEMENT IN SOLUTION.PY

#### Function 1 — load\_flights(spark, path)

#### PURPOSE

Read the flight CSV file using a predefined schema.

#### PARAMETERS

spark – Active SparkSession.

path – Location of flights.csv.

#### LOGIC

Create a schema containing:

- flight\_id string
- route\_id string
- departure\_date date
- passenger\_count integer
- delay\_minutes integer
- flight\_status string
- Read the CSV with the header enabled and the defined schema.

## RETURNS

A DataFrame containing the six flight columns.

## SAMPLE INPUT

flight\_id,route\_id,departure\_date,passenger\_count,delay\_minutes,flight\_status

FL2001,R2001,2025-11-01,145,0,Scheduled

FL2002,R2002,2025-11-01,132,20,Scheduled

## SAMPLE OUTPUT

```
+-----+-----+-----+-----+-----+-----+
|flight_id|route_id|departure_date|passenger_count|delay_minutes|flight_status|
+-----+-----+-----+-----+-----+-----+
|FL2001   |R2001   |2025-11-01   |145           |0           |Scheduled   |
|FL2002   |R2002   |2025-11-01   |132           |20          |Scheduled   |
+-----+-----+-----+-----+-----+-----+
```

## Function 2 — load\_routes(spark, path)

## PURPOSE

Read the route reference dataset using an explicitly defined schema.

## LOGIC

Create the following schema:

- route\_id string
- departure\_airport string
- arrival\_airport string
- route\_distance\_km integer
- aircraft\_type string
- base\_fare integer
- Read the CSV using the schema and header.

## RETURNS

A DataFrame with six columns.

## SAMPLE INPUT

route\_id,departure\_airport,arrival\_airport,route\_distance\_km,aircraft\_type,base\_fare

R2001,DEL,BOM,1148,A320,5200

R2002,DEL,BLR,1740,A320,6100

## SAMPLE OUTPUT

```
+-----+-----+-----+-----+-----+-----+
|route_id|departure_airport|arrival_airport|route_distance_km|aircraft_type|base_fare|
+-----+-----+-----+-----+-----+-----+
|R2001   |DEL              |BOM              |1148             |A320         |5200      |
+-----+-----+-----+-----+-----+-----+
```

|       |     |     |      |      |      |  |
|-------|-----|-----|------|------|------|--|
| R2002 | DEL | BLR | 1740 | A320 | 6100 |  |
|-------|-----|-----|------|------|------|--|

### Function 3 — clean\_and\_join(df\_flights, df\_routes)

#### PURPOSE

Remove unusable flight records and enrich the remaining records with airport information.

#### LOGIC

Remove rows where:

- flight\_id is null.
- flight\_status is "Cancelled".
- passenger\_count <= 0.
- Then perform an inner join with df\_routes using route\_id.

Return only:

- flight\_id
- departure\_airport
- arrival\_airport
- passenger\_count
- delay\_minutes

#### RETURNS

DataFrame with five columns.

#### SAMPLE INPUT

flight\_id | route\_id | passenger\_count | delay\_minutes | flight\_status

|        |       |     |     |           |
|--------|-------|-----|-----|-----------|
| FL2001 | R2001 | 145 | 0   | Scheduled |
| FL2004 | R2004 | 105 | 140 | Scheduled |
| null   | R2001 | 150 | 20  | Scheduled |
| FL2006 | R2002 | 120 | 0   | Cancelled |
| FL2007 | R2003 | 0   | 15  | Scheduled |

#### SAMPLE OUTPUT

| flight_id | departure_airport | arrival_airport | passenger_count | delay_minutes |
|-----------|-------------------|-----------------|-----------------|---------------|
| FL2001    | DEL               | BOM             | 145             | 0             |
| FL2004    | DEL               | HYD             | 105             | 140           |

### Function 4 — classify\_delay(df\_joined)

#### PURPOSE

Normalize delay values and assign a delay severity.

#### LOGIC

Replace negative delay\_minutes with 0.

Then create delay\_category:

| Delay  | Category |
|--------|----------|
| 0      | On-Time  |
| 1-29   | Minor    |
| 30-120 | Major    |
| >120   | Severe   |

## RETURNS

Original columns plus delay\_category.

## SAMPLE INPUT

flight\_id | delay\_minutes

```
-----
FL2001  | 0
FL2002  | 20
FL2003  | 65
FL2004  | 140
FL2005  | -8
```

## SAMPLE OUTPUT

```
+-----+-----+-----+
|flight_id|delay_minutes|delay_category|
+-----+-----+-----+
|FL2001   |0           |On-Time      |
|FL2002   |20          |Minor        |
|FL2003   |65          |Major        |
|FL2004   |140         |Severe       |
|FL2005   |0           |On-Time      |
+-----+-----+-----+
```

## Function 5 — compute\_refund(df\_classified)

### PURPOSE

Calculate the total passenger refund for each flight.

Refund Rules

On-Time → 0

Minor → 50 × passenger\_count

Major → 200 × passenger\_count

Severe → 500 × passenger\_count

Create:

- total\_refund

Return only:

- flight\_id
- departure\_airport
- arrival\_airport
- delay\_minutes
- delay\_category
- total\_refund

**SAMPLE INPUT**

flight\_id | passenger\_count | delay\_minutes | delay\_category

```

-----
FL2001  | 145          | 0          | On-Time
FL2002  | 132          | 20         | Minor
FL2003  | 118          | 65         | Major
FL2004  | 105          | 140        | Severe

```

**SAMPLE OUTPUT**

```

+-----+-----+-----+-----+
|flight_id|delay_minutes|delay_category|total_refund |
+-----+-----+-----+-----+
|FL2001  |0           |On-Time      |0             |
|FL2002  |20          |Minor        |6600          |
|FL2003  |65          |Major        |23600         |
|FL2004  |140         |Severe       |52500         |
+-----+-----+-----+-----+

```

**Function 6 — route\_delay\_summary(df\_refunded)****PURPOSE**

Create a route-level performance report.

**LOGIC**

Group records using:

- departure\_airport
- arrival\_airport

**CALCULATE**

total\_flights – number of flights.

avg\_delay\_minutes – average delay rounded to 2 decimal places.

total\_refund – sum of passenger refunds.

**SAMPLE INPUT**

flight\_id | departure\_airport | arrival\_airport | delay\_minutes | total\_refund

```

-----
FL2001  | DEL          | BOM          | 0            | 0
FL2005  | DEL          | BOM          | 0            | 0
FL2002  | DEL          | BLR          | 20           | 6600
FL2003  | BOM          | MAA          | 65           | 23600

```

**SAMPLE OUTPUT**

```

+-----+-----+-----+-----+-----+
|departure_airport|arrival_airport|total_flights|avg_delay_minutes|total_refund|
+-----+-----+-----+-----+-----+
|BOM              |MAA            |1            |65.00           |23600       |
|DEL              |BLR            |1            |20.00           |6600        |
|DEL              |BOM            |2            |0.00            |0           |
+-----+-----+-----+-----+-----+

```

## Q2. Railway Delay and Passenger Refund Analytics

Part 2 · PySpark — Medium — question 2 of 16

This version follows the same PySpark pattern, but the scenario and wording are changed. It also has one additional simple requirement in the final summary: `delayed_trips`, which counts how many trips had a delay greater than zero.

The complexity remains at the same medium level.

### PROBLEM STATEMENT

A railway operator records thousands of scheduled journeys between cities every day. The operations department wants to analyze journey delays and determine the refund amount applicable to passengers affected by delayed services.

The railway maintains a journey dataset containing journey identifiers, route assignments, travel dates, passenger bookings, delay duration, and journey status. A separate route dataset contains station information, route distance, train type, and base ticket fare.

The analytics team wants to build a PySpark pipeline that validates journey data, combines it with route information, categorizes delays, calculates passenger refunds, and generates a route-wise operational report.

The report must show the number of journeys, average delay, total refund amount, and the number of delayed journeys for every origin-destination combination.

### OBJECTIVE

- Load journey and route CSV files with explicitly defined schemas.
- Remove invalid journeys such as missing journey IDs, cancelled journeys, and journeys with no passengers.
- Enrich valid journey records using the route master.
- Normalize negative delay values to zero.
- Categorize journeys as On-Time, Minor, Major, or Severe.
- Calculate refund amounts based on passenger count and delay category.
- Generate a route-level summary.
- Additionally calculate the number of delayed journeys on each route.

### FILE STRUCTURE

```
Project
├── data
│   ├── journeys.csv
│   └── routes.csv
├── main.py
├── solution.py
├── tests.py
└── requirements.txt
```

### PURPOSE OF FILES

- `main.py` – Driver script responsible for creating the Spark session, loading data, calling functions, and displaying results.
- `solution.py` – Contains the PySpark transformation functions.
- `tests.py` – Automated tests for the implemented functions.
- `requirements.txt` – Required dependencies.
- `data/journeys.csv` – Raw railway journey records.
- `data/routes.csv` – Railway route reference information.

### DATASET DESCRIPTION

File: `journeys.csv`

## SCHEMA

| Column Name     | Data Type | Description                                   |
|-----------------|-----------|-----------------------------------------------|
| journey_id      | string    | Unique identifier for a journey               |
| route_id        | string    | Identifier of the assigned railway route      |
| journey_date    | date      | Scheduled journey date                        |
| passenger_count | integer   | Number of passengers booked                   |
| delay_minutes   | integer   | Delay duration in minutes                     |
| journey_status  | string    | Current status such as Scheduled or Cancelled |

## SAMPLE DATA

| journey_id | route_id | journey_date | passenger_count | delay_minutes | journey_status |
|------------|----------|--------------|-----------------|---------------|----------------|
| J3001      | RT3001   | 2025-12-01   | 180             | 0             | Scheduled      |
| J3002      | RT3002   | 2025-12-01   | 155             | 25            | Scheduled      |
| J3003      | RT3003   | 2025-12-02   | 140             | 75            | Scheduled      |
| J3004      | RT3004   | 2025-12-02   | 120             | 135           | Scheduled      |
| J3005      | RT3001   | 2025-12-03   | 165             | -6            | Scheduled      |
| J3006      | RT3002   | 2025-12-03   | 130             | 10            | Cancelled      |

File: routes.csv

## SCHEMA

| Column Name         | Data Type | Description                          |
|---------------------|-----------|--------------------------------------|
| route_id            | string    | Unique route identifier              |
| source_station      | string    | Code of the originating station      |
| destination_station | string    | Code of the destination station      |
| distance_km         | integer   | Route distance in kilometres         |
| train_type          | string    | Type of train operating on the route |
| base_fare           | integer   | Standard fare for the route          |

## SAMPLE DATA

| route_id | source_station | destination_station | distance_km | train_type | base_fare |
|----------|----------------|---------------------|-------------|------------|-----------|
| RT3001   | NDLS           | BCT                 | 1384        | Express    | 1800      |
| RT3002   | NDLS           | CSMT                | 1520        | Superfast  | 2100      |
| RT3003   | BCT            | MAS                 | 1280        | Express    | 1950      |
| RT3004   | NDLS           | HYB                 | 1670        | Superfast  | 2250      |

## TASK

## FUNCTIONS TO IMPLEMENT IN SOLUTION.PY

## Function 1 — load\_journeys(spark, path)

## PURPOSE

Load the railway journey dataset using an explicitly declared schema.

**SCHEMA**

```
journey_id    string
route_id      string
journey_date  date
passenger_count integer
delay_minutes integer
journey_status string
```

**RETURNS**

DataFrame containing six columns.

**SAMPLE INPUT**

```
journey_id,route_id,journey_date,passenger_count,delay_minutes,journey_status
J3001,RT3001,2025-12-01,180,0,Scheduled
J3002,RT3002,2025-12-01,155,25,Scheduled
```

**SAMPLE OUTPUT**

| journey_id | route_id | journey_date | passenger_count | delay_minutes | journey_status |
|------------|----------|--------------|-----------------|---------------|----------------|
| J3001      | RT3001   | 2025-12-01   | 180             | 0             | Scheduled      |
| J3002      | RT3002   | 2025-12-01   | 155             | 25            | Scheduled      |

**Function 2 — load\_routes(spark, path)****PURPOSE**

Load the railway route master using a predefined schema.

**SCHEMA**

```
route_id      string
source_station string
destination_station string
distance_km    integer
train_type     string
base_fare      integer
```

**SAMPLE INPUT**

```
route_id,source_station,destination_station,distance_km,train_type,base_fare
RT3001,NDLS,BCT,1384,Express,1800
RT3002,NDLS,CSMT,1520,Superfast,2100
```

**SAMPLE OUTPUT**

| route_id | source_station | destination_station | distance_km | train_type | base_fare |
|----------|----------------|---------------------|-------------|------------|-----------|
| RT3001   | NDLS           | BCT                 | 1384        | Express    | 1800      |
| RT3002   | NDLS           | CSMT                | 1520        | Superfast  | 2100      |

**Function 3 — clean\_and\_join(df\_journeys, df\_routes)**

## PURPOSE

Filter invalid journey records and combine valid journeys with route details.

## LOGIC

Remove records where:

- journey\_id is null.

```
journey_status == "Cancelled".
```

passenger\_count <= 0.

Perform an inner join on route\_id.

Return only:

- journey\_id
- source\_station
- destination\_station
- passenger\_count
- delay\_minutes

## SAMPLE INPUT

journey\_id | route\_id | passenger\_count | delay\_minutes | journey\_status

```
-----
J3001   | RT3001 | 180          | 0           | Scheduled
J3002   | RT3002 | 155          | 25          | Scheduled
null    | RT3001 | 160          | 15          | Scheduled
J3006   | RT3002 | 130          | 10          | Cancelled
J3007   | RT3003 | 0            | 20          | Scheduled
```

## SAMPLE OUTPUT

```
+-----+-----+-----+-----+-----+
|journey_id|source_station|destination_station|passenger_count|delay_minutes|
+-----+-----+-----+-----+-----+
|J3001     |NDLS         |BCT               |180           |0            |
|J3002     |NDLS         |CSMT              |155           |25           |
+-----+-----+-----+-----+-----+
```

## Function 4 — classify\_delay(df\_joined)

## PURPOSE

Normalize journey delay values and classify their severity.

## LOGIC

First replace negative delay\_minutes with 0.

Then create delay\_category:

- 0 → On-Time
- 1–29 → Minor
- 30–120 → Major
- >120 → Severe

## SAMPLE INPUT

journey\_id | delay\_minutes

```
J3001    | 0
J3002    | 25
J3003    | 75
J3004    | 135
J3005    | -6
```

### SAMPLE OUTPUT

```
+-----+-----+-----+
|journey_id|delay_minutes|delay_category|
+-----+-----+-----+
|J3001     |0           |On-Time      |
|J3002     |25          |Minor        |
|J3003     |75          |Major        |
|J3004     |135         |Severe       |
|J3005     |0           |On-Time      |
+-----+-----+-----+
```

## Function 5 — compute\_refund(df\_classified)

### PURPOSE

Calculate the total passenger refund for every valid journey.

Refund Rules

On-Time → 0

Minor → 50 × passenger\_count

Major → 200 × passenger\_count

Severe → 500 × passenger\_count

Create:

- total\_refund

### RETURN

journey\_id

source\_station

destination\_station

delay\_minutes

delay\_category

total\_refund

### SAMPLE INPUT

journey\_id | passenger\_count | delay\_minutes | delay\_category

```
-----
J3001    | 180          | 0            | On-Time
J3002    | 155          | 25           | Minor
J3003    | 140          | 75           | Major
J3004    | 120          | 135          | Severe
```

### SAMPLE OUTPUT

```
+-----+-----+-----+-----+
|journey_id|delay_minutes|delay_category|total_refund |
+-----+-----+-----+-----+
```

|       |     |         |       |
|-------|-----|---------|-------|
| J3001 | 0   | On-Time | 0     |
| J3002 | 25  | Minor   | 7750  |
| J3003 | 75  | Major   | 28000 |
| J3004 | 135 | Severe  | 60000 |

## Function 6 — route\_delay\_summary(df\_refunded)

### PURPOSE

Generate an operational report for every source-destination pair.

### LOGIC

Group by:

- source\_station
- destination\_station

### CALCULATE

total\_journeys – total number of journeys.

avg\_delay\_minutes – average delay rounded to two decimal places.

total\_refund – total refund amount.

delayed\_journeys – number of journeys where delay\_minutes > 0.

This is the additional requirement compared with Question 1.

### RETURNS

source\_station

destination\_station

total\_journeys

avg\_delay\_minutes

total\_refund

delayed\_journeys

### SAMPLE INPUT

journey\_id | source | destination | delay\_minutes | total\_refund

|       |      |      |    |       |
|-------|------|------|----|-------|
| J3001 | NDLS | BCT  | 0  | 0     |
| J3005 | NDLS | BCT  | 0  | 0     |
| J3002 | NDLS | CSMT | 25 | 7750  |
| J3003 | BCT  | MAS  | 75 | 28000 |

### SAMPLE OUTPUT

| source | destination | total_journeys | avg_delay_minutes | total_refund | delayed_journeys |
|--------|-------------|----------------|-------------------|--------------|------------------|
| BCT    | MAS         | 1              | 75.00             | 28000        | 1                |
| NDLS   | BCT         | 2              | 0.00              | 0            | 0                |
| NDLS   | CSMT        | 1              | 25.00             | 7750         | 1                |

## Q3. Restaurant Order Revenue and Cancellation Analytics

Part 2 · PySpark — Medium — question 3 of 16

A food delivery platform manages restaurant orders across major Indian cities. The analytics team is responsible for monitoring order performance, identifying revenue loss caused by cancelled and failed orders, and generating restaurant-level business insights.

Each order record contains information such as the order identifier, customer identifier, restaurant assignment, order date, number of items, meal category, item price, and order status. Restaurant reference data contains the restaurant name, city, cuisine type, and rating.

The analytics team requires an automated PySpark data pipeline that loads and cleans order records, enriches them with restaurant information, calculates expected and realized revenue, classifies orders based on their final status, calculates revenue loss, and generates a city-level summary grouped by meal category and order outcome.

### OBJECTIVE

- Load order and restaurant datasets using explicitly defined schemas with appropriate data types.
- Clean order records by removing entries with missing order or restaurant identifiers, invalid item quantities, and negative item prices, then join them with restaurant data.
- Calculate expected revenue as  $\text{item\_count} \times \text{item\_price}$ .
- Calculate realized revenue as expected revenue for successfully delivered orders and zero otherwise.
- Classify each order into an outcome category such as Completed, Lost-Cancelled, or Lost-Failed.
- Calculate revenue loss caused by unsuccessful orders.
- Produce a city-level summary showing total orders, expected revenue, realized revenue, and revenue loss grouped by city, meal category, and order outcome.

### FILE STRUCTURE

```
Project
+-- data
|   +-- orders.csv
|   +-- restaurants.csv
+-- main.py
+-- solution.py
+-- tests.py
+-- requirements.txt
```

### PURPOSE OF FILES

- main.py - Driver script that loads data, invokes solution functions, and displays intermediate results.
- solution.py - Contains six functions to be implemented by the candidate.
- requirements.txt - Python dependencies required to run the project (pyspark, pytest).
- data/orders.csv - Raw order dataset used as input.
- data/restaurants.csv - Restaurant reference dataset.

### MODULES

- main.py - Contains the driver code/main invocation pipeline.
- solution.py - Candidate must implement all six functions.

### DATASET DESCRIPTION

File: orders.csv

### SCHEMA

| Column Name | Data Type | Description                      |
|-------------|-----------|----------------------------------|
| order_id    | string    | Unique identifier for each order |
| customer_id | string    | Identifier for the customer      |

| Column Name   | Data Type | Description                                  |
|---------------|-----------|----------------------------------------------|
| restaurant_id | string    | Identifier linking the order to a restaurant |
| order_date    | date      | Date on which the order was placed           |
| item_count    | integer   | Number of items in the order                 |
| meal_type     | string    | Type of meal ordered                         |
| item_price    | integer   | Price per item                               |
| order_status  | string    | Status of the order                          |

## SAMPLE DATA

| order_id | customer_id | restaurant_id | order_date | item_count | meal_type   | item_price | order_status |
|----------|-------------|---------------|------------|------------|-------------|------------|--------------|
| OR5001   | C301        | R201          | 2025-12-01 | 3          | Main-Course | 450        | Delivered    |
| OR5002   | C302        | R201          | 2025-12-02 | 2          | Dessert     | 300        | Delivered    |
| OR5003   | C303        | R201          | 2025-12-03 | 1          | Main-Course | 500        | Cancelled    |
| OR5004   | C304        | R202          | 2025-12-01 | 4          | Beverage    | 150        | Delivered    |
| OR5005   | C305        | R202          | 2025-12-02 | 2          | Main-Course | 350        | Failed       |
| OR5039   | C339        | null          | 2025-12-02 | 3          | Dessert     | 400        | Delivered    |

File: restaurants.csv

## SCHEMA

| Column Name     | Data Type | Description                           |
|-----------------|-----------|---------------------------------------|
| restaurant_id   | string    | Unique identifier for each restaurant |
| restaurant_name | string    | Name of the restaurant                |
| city            | string    | City where the restaurant operates    |
| cuisine_type    | string    | Type of cuisine served                |
| rating          | integer   | Restaurant rating                     |

## SAMPLE DATA

| restaurant_id | restaurant_name | city      | cuisine_type | rating |
|---------------|-----------------|-----------|--------------|--------|
| R201          | Spice Garden    | Mumbai    | Indian       | 5      |
| R202          | Urban Bites     | Mumbai    | Continental  | 3      |
| R203          | Royal Kitchen   | Delhi     | Indian       | 5      |
| R204          | Food Junction   | Delhi     | Chinese      | 2      |
| R205          | Green Bowl      | Bangalore | Healthy      | 4      |

## TASK

### FUNCTIONS TO IMPLEMENT IN SOLUTION.PY

#### Function 1 — load\_orders(spark, path)

#### PURPOSE

Load the orders dataset from a CSV file using an explicitly defined schema.

#### PARAMETERS

spark - Active SparkSession.

path - Path to the orders CSV file.

## LOGIC

---

Define a schema containing:

- order\_id - StringType
- customer\_id - StringType
- restaurant\_id - StringType
- order\_date - DateType
- item\_count - IntegerType
- meal\_type - StringType
- item\_price - IntegerType
- order\_status - StringType
- Read the CSV with headers enabled using the defined schema.

## RETURNS

---

DataFrame containing the eight columns.

### Function 2 — load\_restaurants(spark, path)

## PURPOSE

---

Load restaurant reference data from CSV using an explicitly defined schema.

## LOGIC

---

Define:

- restaurant\_id - StringType
- restaurant\_name - StringType
- city - StringType
- cuisine\_type - StringType
- rating - IntegerType
- Read the CSV with headers enabled.

## RETURNS

---

DataFrame containing the five columns.

### Function 3 — clean\_and\_join(df\_orders, df\_restaurants)

## PURPOSE

---

Remove invalid order records and enrich valid records with restaurant information.

## LOGIC

---

Filter out records where:

- order\_id is null
- restaurant\_id is null
- item\_count is zero or less
- item\_price is zero or less
- Perform an inner join with the restaurant DataFrame using restaurant\_id.

Select only:

- order\_id
- restaurant\_name
- city

- item\_count
- meal\_type
- item\_price
- order\_status

## RETURNS

DataFrame with seven columns.

### Function 4 — compute\_revenue(df\_joined)

## PURPOSE

Calculate expected and realized revenue for every valid order.

## LOGIC

Add:

```
expected_revenue = item_count * item_price
```

Add:

- realized\_revenue =
- expected\_revenue when order\_status = "Delivered"
- 0 otherwise

## RETURNS

Original columns plus:

- expected\_revenue
- realized\_revenue

### Function 5 — classify\_and\_compute\_loss(df\_revenue)

## PURPOSE

Classify each order and calculate revenue loss.

## LOGIC

Create order\_flag:

- Delivered → Completed
- Cancelled → Lost-Cancelled
- Any other status → Lost-Failed

Create revenue\_loss:

- If order is not Delivered → expected\_revenue
- Otherwise → 0

## RETURNS

Original columns plus:

- order\_flag
- revenue\_loss

### Function 6 — city\_meal\_revenue\_summary(df\_classified)

## PURPOSE

Generate a city-level revenue summary grouped by meal type and order outcome.

## LOGIC

Group by:

- city
- meal\_type
- order\_flag

## CALCULATE

total\_orders → count of orders

total\_expected\_revenue → sum of expected revenue

total\_realized\_revenue → sum of realized revenue

total\_revenue\_loss → sum of revenue loss

## RETURNS

city

meal\_type

order\_flag

total\_orders

total\_expected\_revenue

total\_realized\_revenue

total\_revenue\_loss

## SAMPLE INPUT AND OUTPUT

### Function 1 — load\_orders(spark, path)

#### SAMPLE INPUT

orders.csv

| order_id | customer_id | restaurant_id | order_date | item_count | meal_type   | item_price | order_status |
|----------|-------------|---------------|------------|------------|-------------|------------|--------------|
| OR5001   | C301        | R201          | 2025-12-01 | 3          | Main-Course | 450        | Delivered    |
| OR5002   | C302        | R201          | 2025-12-02 | 2          | Dessert     | 300        | Delivered    |
| OR5003   | C303        | R201          | 2025-12-03 | 1          | Main-Course | 500        | Cancelled    |
| OR5004   | C304        | R202          | 2025-12-01 | 4          | Beverage    | 150        | Delivered    |
| OR5005   | C305        | R202          | 2025-12-02 | 2          | Main-Course | 350        | Failed       |
| OR5033   | C333        | R201          | 2025-12-06 | 2          | Main-Course | 450        | Delivered    |
| null     | C337        | R201          | 2025-12-03 | 2          | Dessert     | 400        | Delivered    |
| OR5040   | C340        | R202          | 2025-12-03 | 0          | Beverage    | 200        | Delivered    |
| OR5041   | C341        | R202          | 2025-12-04 | 2          | Main-Course | -300       | Delivered    |

#### SAMPLE OUTPUT

| order_id | customer_id | restaurant_id | order_date | item_count | meal_type   | item_price | order_status |
|----------|-------------|---------------|------------|------------|-------------|------------|--------------|
| OR5001   | C301        | R201          | 2025-12-01 | 3          | Main-Course | 450        | Delivered    |
| OR5002   | C302        | R201          | 2025-12-02 | 2          | Dessert     | 300        | Delivered    |
| OR5003   | C303        | R201          | 2025-12-03 | 1          | Main-Course | 500        | Cancelled    |
| OR5004   | C304        | R202          | 2025-12-01 | 4          | Beverage    | 150        | Delivered    |
| OR5005   | C305        | R202          | 2025-12-02 | 2          | Main-Course | 350        | Failed       |

| order_id | customer_id | restaurant_id | order_date | item_count | meal_type   | item_price | order_status |
|----------|-------------|---------------|------------|------------|-------------|------------|--------------|
| OR5033   | C333        | R201          | 2025-12-06 | 2          | Main-Course | 450        | Delivered    |
| null     | C337        | R201          | 2025-12-03 | 2          | Dessert     | 400        | Delivered    |
| OR5040   | C340        | R202          | 2025-12-03 | 0          | Beverage    | 200        | Delivered    |
| OR5041   | C341        | R202          | 2025-12-04 | 2          | Main-Course | -300       | Delivered    |

## Function 2 — load\_restaurants(spark, path)

### SAMPLE INPUT

| restaurant_id | restaurant_name | city   | cuisine_type | rating |
|---------------|-----------------|--------|--------------|--------|
| R201          | Spice Garden    | Mumbai | Indian       | 5      |
| R202          | Urban Bites     | Mumbai | Continental  | 3      |
| R203          | Royal Kitchen   | Delhi  | Indian       | 5      |
| R204          | Food Junction   | Delhi  | Chinese      | 2      |

### SAMPLE OUTPUT

| restaurant_id | restaurant_name | city   | cuisine_type | rating |
|---------------|-----------------|--------|--------------|--------|
| R201          | Spice Garden    | Mumbai | Indian       | 5      |
| R202          | Urban Bites     | Mumbai | Continental  | 3      |
| R203          | Royal Kitchen   | Delhi  | Indian       | 5      |
| R204          | Food Junction   | Delhi  | Chinese      | 2      |

## Function 3 — clean\_and\_join(df\_orders, df\_restaurants)

Sample Input — Orders

| order_id | customer_id | restaurant_id | order_date | item_count | meal_type   | item_price | order_status |
|----------|-------------|---------------|------------|------------|-------------|------------|--------------|
| OR5001   | C301        | R201          | 2025-12-01 | 3          | Main-Course | 450        | Delivered    |
| OR5002   | C302        | R201          | 2025-12-02 | 2          | Dessert     | 300        | Delivered    |
| OR5003   | C303        | R201          | 2025-12-03 | 1          | Main-Course | 500        | Cancelled    |
| OR5004   | C304        | R202          | 2025-12-01 | 4          | Beverage    | 150        | Delivered    |
| OR5005   | C305        | R202          | 2025-12-02 | 2          | Main-Course | 350        | Failed       |
| OR5033   | C333        | R201          | 2025-12-06 | 2          | Main-Course | 450        | Delivered    |
| null     | C337        | R201          | 2025-12-03 | 2          | Dessert     | 400        | Delivered    |
| OR5040   | C340        | R202          | 2025-12-03 | 0          | Beverage    | 200        | Delivered    |
| OR5041   | C341        | R202          | 2025-12-04 | 2          | Main-Course | -300       | Delivered    |

Sample Input — Restaurants

| restaurant_id | restaurant_name | city   | cuisine_type | rating |
|---------------|-----------------|--------|--------------|--------|
| R201          | Spice Garden    | Mumbai | Indian       | 5      |
| R202          | Urban Bites     | Mumbai | Continental  | 3      |
| R203          | Royal Kitchen   | Delhi  | Indian       | 5      |

### SAMPLE OUTPUT

Invalid records are removed and the remaining records are joined with restaurant data.

| order_id | restaurant_name | city   | item_count | meal_type   | item_price | order_status |
|----------|-----------------|--------|------------|-------------|------------|--------------|
| OR5001   | Spice Garden    | Mumbai | 3          | Main-Course | 450        | Delivered    |
| OR5002   | Spice Garden    | Mumbai | 2          | Dessert     | 300        | Delivered    |
| OR5003   | Spice Garden    | Mumbai | 1          | Main-Course | 500        | Cancelled    |
| OR5004   | Urban Bites     | Mumbai | 4          | Beverage    | 150        | Delivered    |
| OR5005   | Urban Bites     | Mumbai | 2          | Main-Course | 350        | Failed       |
| OR5033   | Spice Garden    | Mumbai | 2          | Main-Course | 450        | Delivered    |

#### Function 4 — compute\_revenue(df\_joined)

##### SAMPLE INPUT

| order_id | restaurant_name | city   | item_count | meal_type   | item_price | order_status |
|----------|-----------------|--------|------------|-------------|------------|--------------|
| OR5001   | Spice Garden    | Mumbai | 3          | Main-Course | 450        | Delivered    |
| OR5002   | Spice Garden    | Mumbai | 2          | Dessert     | 300        | Delivered    |
| OR5003   | Spice Garden    | Mumbai | 1          | Main-Course | 500        | Cancelled    |
| OR5004   | Urban Bites     | Mumbai | 4          | Beverage    | 150        | Delivered    |
| OR5005   | Urban Bites     | Mumbai | 2          | Main-Course | 350        | Failed       |
| OR5033   | Spice Garden    | Mumbai | 2          | Main-Course | 450        | Delivered    |

##### SAMPLE OUTPUT

| order_id | restaurant_name | city   | item_count | meal_type   | item_price | order_status | expected_revenue | realized_revenue |
|----------|-----------------|--------|------------|-------------|------------|--------------|------------------|------------------|
| OR5001   | Spice Garden    | Mumbai | 3          | Main-Course | 450        | Delivered    | 1350             | 1350             |
| OR5002   | Spice Garden    | Mumbai | 2          | Dessert     | 300        | Delivered    | 600              | 600              |
| OR5003   | Spice Garden    | Mumbai | 1          | Main-Course | 500        | Cancelled    | 500              | 0                |
| OR5004   | Urban Bites     | Mumbai | 4          | Beverage    | 150        | Delivered    | 600              | 600              |
| OR5005   | Urban Bites     | Mumbai | 2          | Main-Course | 350        | Failed       | 700              | 0                |
| OR5033   | Spice Garden    | Mumbai | 2          | Main-Course | 450        | Delivered    | 900              | 900              |

#### Function 5 — classify\_and\_compute\_loss(df\_revenue)

##### SAMPLE INPUT

| order_id | restaurant_name | city   | item_count | meal_type   | item_price | order_status | expected_revenue | realized_revenue |
|----------|-----------------|--------|------------|-------------|------------|--------------|------------------|------------------|
| OR5001   | Spice Garden    | Mumbai | 3          | Main-Course | 450        | Delivered    | 1350             | 1350             |
| OR5002   | Spice Garden    | Mumbai | 2          | Dessert     | 300        | Delivered    | 600              | 600              |
| OR5003   | Spice Garden    | Mumbai | 1          | Main-Course | 500        | Cancelled    | 500              | 0                |
| OR5004   | Urban Bites     | Mumbai | 4          | Beverage    | 150        | Delivered    | 600              | 600              |
| OR5005   | Urban Bites     | Mumbai | 2          | Main-Course | 350        | Failed       | 700              | 0                |
| OR5033   | Spice Garden    | Mumbai | 2          | Main-Course | 450        | Delivered    | 900              | 900              |

##### SAMPLE OUTPUT

| order_id | restaurant_name | city   | item_count | meal_type   | item_price | order_status | expected_revenue | realized_revenue |
|----------|-----------------|--------|------------|-------------|------------|--------------|------------------|------------------|
| OR5001   | Spice Garden    | Mumbai | 3          | Main-Course | 450        | Delivered    | 1350             | 1350             |
| OR5002   | Spice Garden    | Mumbai | 2          | Dessert     | 300        | Delivered    | 600              | 600              |
| OR5003   | Spice Garden    | Mumbai | 1          | Main-Course | 500        | Cancelled    | 500              | 0                |
| OR5004   | Urban Bites     | Mumbai | 4          | Beverage    | 150        | Delivered    | 600              | 600              |
| OR5005   | Urban Bites     | Mumbai | 2          | Main-Course | 350        | Failed       | 700              | 0                |
| OR5033   | Spice Garden    | Mumbai | 2          | Main-Course | 450        | Delivered    | 900              | 900              |

## Function 6 — city\_meal\_revenue\_summary(df\_classified)

### SAMPLE INPUT

| order_id | city   | meal_type   | expected_revenue | realized_revenue | order_flag     | revenue_loss |
|----------|--------|-------------|------------------|------------------|----------------|--------------|
| OR5001   | Mumbai | Main-Course | 1350             | 1350             | Completed      | 0            |
| OR5002   | Mumbai | Dessert     | 600              | 600              | Completed      | 0            |
| OR5003   | Mumbai | Main-Course | 500              | 0                | Lost-Cancelled | 500          |
| OR5004   | Mumbai | Beverage    | 600              | 600              | Completed      | 0            |
| OR5005   | Mumbai | Main-Course | 700              | 0                | Lost-Failed    | 700          |
| OR5033   | Mumbai | Main-Course | 900              | 900              | Completed      | 0            |

### SAMPLE OUTPUT

| city   | meal_type   | order_flag     | total_orders | total_expected_revenue | total_realized_revenue | total_revenue_loss |
|--------|-------------|----------------|--------------|------------------------|------------------------|--------------------|
| Mumbai | Beverage    | Completed      | 1            | 600                    | 600                    | 0                  |
| Mumbai | Dessert     | Completed      | 1            | 600                    | 600                    | 0                  |
| Mumbai | Main-Course | Completed      | 2            | 2250                   | 2250                   | 0                  |
| Mumbai | Main-Course | Lost-Cancelled | 1            | 500                    | 0                      | 500                |
| Mumbai | Main-Course | Lost-Failed    | 1            | 700                    | 0                      |                    |

## Q4. Online Grocery Order Revenue and Delivery Performance Analytics

Part 2 · PySpark — Medium — question 4 of 16

A grocery delivery company operates an online ordering platform across multiple Indian cities. The business analytics team needs to monitor order revenue, identify losses from cancelled and failed deliveries, and understand performance across product categories.

Each grocery order contains an order identifier, customer identifier, store assignment, delivery date, quantity purchased, product category, unit price, and delivery status. Store reference data provides the store name, city, store type, and service rating.

The analytics team requires an automated PySpark pipeline that cleans order data, enriches orders with store information, calculates expected and realized revenue, classifies delivery outcomes, calculates revenue loss, and generates a city-level category summary.

In addition to the basic revenue analysis, the pipeline must also calculate the average revenue per order for each city/category/outcome group.

### OBJECTIVE

- Load grocery order and store datasets using explicitly defined schemas.
- Clean invalid order records and join them with store reference data.
- Calculate expected revenue using quantity and unit price.
- Calculate realized revenue only for successfully delivered orders.
- Classify orders into outcome categories.
- Calculate revenue loss for unsuccessful orders.
- Produce a city/category/outcome summary containing order counts and revenue metrics.
- Additionally calculate average realized revenue per order for each summary group.

### FILE STRUCTURE

```
Project
+-- data
|   +-- grocery_orders.csv
|   +-- stores.csv
+-- main.py
+-- solution.py
+-- tests.py
+-- requirements.txt
```

### PURPOSE OF FILES

- main.py - Driver script that executes the pipeline.
- solution.py - Contains six functions to be implemented.
- tests.py - Contains test cases.
- requirements.txt - Contains pyspark and pytest.
- data/grocery\_orders.csv - Raw grocery order records.
- data/stores.csv - Store reference dataset.

### DATASET DESCRIPTION

File: grocery\_orders.csv

### SCHEMA

| Column Name   | Data Type | Description               |
|---------------|-----------|---------------------------|
| order_id      | string    | Unique order identifier   |
| customer_id   | string    | Customer identifier       |
| store_id      | string    | Store linked to the order |
| delivery_date | date      | Scheduled delivery date   |

| Column Name      | Data Type | Description                |
|------------------|-----------|----------------------------|
| quantity         | integer   | Number of products ordered |
| product_category | string    | Category of products       |
| unit_price       | integer   | Price of one product       |
| delivery_status  | string    | Delivery status            |

## SAMPLE DATA

| order_id | customer_id | store_id | delivery_date | quantity | product_category | unit_price | delivery_status |
|----------|-------------|----------|---------------|----------|------------------|------------|-----------------|
| G06001   | C401        | S301     | 2025-12-10    | 3        | Grocery          | 250        | Delivered       |
| G06002   | C402        | S301     | 2025-12-11    | 2        | Dairy            | 180        | Delivered       |
| G06003   | C403        | S301     | 2025-12-12    | 1        | Grocery          | 500        | Cancelled       |
| G06004   | C404        | S302     | 2025-12-10    | 4        | Beverages        | 120        | Delivered       |
| G06005   | C405        | S302     | 2025-12-11    | 2        | Grocery          | 350        | Failed          |
| G06039   | C439        | null     | 2025-12-11    | 3        | Dairy            | 400        | Delivered       |

File: stores.csv

## SCHEMA

| Column Name | Data Type | Description             |
|-------------|-----------|-------------------------|
| store_id    | string    | Unique store identifier |
| store_name  | string    | Name of the store       |
| city        | string    | Store location          |
| store_type  | string    | Type of grocery store   |
| rating      | integer   | Store service rating    |

## SAMPLE DATA

| store_id | store_name   | city      | store_type  | rating |
|----------|--------------|-----------|-------------|--------|
| S301     | Fresh Mart   | Mumbai    | Supermarket | 5      |
| S302     | Daily Needs  | Mumbai    | Convenience | 3      |
| S303     | Green Basket | Delhi     | Supermarket | 5      |
| S304     | Value Store  | Delhi     | Hypermarket | 2      |
| S305     | Nature Foods | Bangalore | Organic     | 4      |

## TASK

### Function 1 — load\_orders(spark, path)

#### PURPOSE

Load the grocery orders CSV using an explicit schema.

#### SCHEMA

```
order_id      StringType
customer_id   StringType
store_id      StringType
delivery_date  DateType
```

```
quantity      IntegerType
product_category StringType
unit_price     IntegerType
delivery_status StringType
```

Read the CSV with:

```
header = true
```

## RETURNS

---

DataFrame containing all eight columns.

### Function 2 — load\_stores(spark, path)

Load the store dataset using an explicitly defined schema.

## SCHEMA

---

```
store_id  StringType
store_name StringType
city      StringType
store_type StringType
rating    IntegerType
```

## RETURNS

---

DataFrame containing five columns.

### Function 3 — clean\_and\_join(df\_orders, df\_stores)

## PURPOSE

---

Clean invalid grocery orders and enrich them with store information.

Remove records where:

- order\_id is null
- store\_id is null
- quantity <= 0
- unit\_price <= 0

Perform an inner join on:

- store\_id

Select:

- order\_id
- store\_name
- city
- quantity
- product\_category
- unit\_price
- delivery\_status

## RETURNS

---

DataFrame with seven columns.

### Function 4 — compute\_revenue(df\_joined)

Add:

```
expected_revenue = quantity * unit_price
```

Add:

- realized\_revenue =
- expected\_revenue if delivery\_status = "Delivered"
- 0 otherwise

## RETURNS

All existing columns plus:

- expected\_revenue
- realized\_revenue

## Function 5 — classify\_and\_compute\_loss(df\_revenue)

Create delivery\_flag using:

- Delivered → Completed
- Cancelled → Lost-Cancellation
- Any other status → Lost-Failed

Create:

- revenue\_loss =
- expected\_revenue when delivery\_status != "Delivered"
- 0 otherwise

## RETURNS

All existing columns plus:

- delivery\_flag
- revenue\_loss

## Function 6 — city\_category\_revenue\_summary(df\_classified)

### PURPOSE

Generate the final city-level performance summary.

Group by:

- city
- product\_category
- delivery\_flag

### CALCULATE

total\_orders

total\_expected\_revenue

total\_realized\_revenue

total\_revenue\_loss

Additional requirement

Also calculate:

- avg\_realized\_revenue

### WHERE

avg\_realized\_revenue =

total\_realized\_revenue / total\_orders

Round avg\_realized\_revenue to 2 decimal places.

Final output columns

city

product\_category

delivery\_flag

total\_orders

total\_expected\_revenue

total\_realized\_revenue

total\_revenue\_loss

avg\_realized\_revenue

## SAMPLE INPUT AND OUTPUT

### Function 1 — load\_orders(spark, path)

#### SAMPLE INPUT

| order_id | customer_id | store_id | delivery_date | quantity | product_category | unit_price | delivery_status |
|----------|-------------|----------|---------------|----------|------------------|------------|-----------------|
| G06001   | C401        | S301     | 2025-12-10    | 3        | Grocery          | 250        | Delivered       |
| G06002   | C402        | S301     | 2025-12-11    | 2        | Dairy            | 180        | Delivered       |
| G06003   | C403        | S301     | 2025-12-12    | 1        | Grocery          | 500        | Cancelled       |
| G06004   | C404        | S302     | 2025-12-10    | 4        | Beverages        | 120        | Delivered       |
| G06005   | C405        | S302     | 2025-12-11    | 2        | Grocery          | 350        | Failed          |
| G06033   | C433        | S301     | 2025-12-16    | 2        | Grocery          | 250        | Delivered       |
| null     | C437        | S301     | 2025-12-13    | 2        | Dairy            | 400        | Delivered       |
| G06040   | C440        | S302     | 2025-12-13    | 0        | Beverages        | 200        | Delivered       |
| G06041   | C441        | S302     | 2025-12-14    | 2        | Grocery          | -300       | Delivered       |

#### SAMPLE OUTPUT

Same data as the input because this function only loads the CSV; validation is performed later.

| order_id | customer_id | store_id | delivery_date | quantity | product_category | unit_price | delivery_status |
|----------|-------------|----------|---------------|----------|------------------|------------|-----------------|
| G06001   | C401        | S301     | 2025-12-10    | 3        | Grocery          | 250        | Delivered       |
| G06002   | C402        | S301     | 2025-12-11    | 2        | Dairy            | 180        | Delivered       |
| G06003   | C403        | S301     | 2025-12-12    | 1        | Grocery          | 500        | Cancelled       |
| G06004   | C404        | S302     | 2025-12-10    | 4        | Beverages        | 120        | Delivered       |
| G06005   | C405        | S302     | 2025-12-11    | 2        | Grocery          | 350        | Failed          |
| G06033   | C433        | S301     | 2025-12-16    | 2        | Grocery          | 250        | Delivered       |
| null     | C437        | S301     | 2025-12-13    | 2        | Dairy            | 400        | Delivered       |
| G06040   | C440        | S302     | 2025-12-13    | 0        | Beverages        | 200        | Delivered       |
| G06041   | C441        | S302     | 2025-12-14    | 2        | Grocery          | -300       | Delivered       |

### Function 2 — load\_stores(spark, path)

#### SAMPLE INPUT

| store_id | store_name | city   | store_type  | rating |
|----------|------------|--------|-------------|--------|
| S301     | Fresh Mart | Mumbai | Supermarket | 5      |

| store_id | store_name   | city   | store_type  | rating |
|----------|--------------|--------|-------------|--------|
| S302     | Daily Needs  | Mumbai | Convenience | 3      |
| S303     | Green Basket | Delhi  | Supermarket | 5      |
| S304     | Value Store  | Delhi  | Hypermarket | 2      |

### SAMPLE OUTPUT

| store_id | store_name   | city   | store_type  | rating |
|----------|--------------|--------|-------------|--------|
| S301     | Fresh Mart   | Mumbai | Supermarket | 5      |
| S302     | Daily Needs  | Mumbai | Convenience | 3      |
| S303     | Green Basket | Delhi  | Supermarket | 5      |
| S304     | Value Store  | Delhi  | Hypermarket | 2      |

### Function 3 — clean\_and\_join(df\_orders, df\_stores)

Sample Input — Orders

| order_id | store_id | quantity | product_category | unit_price | delivery_status |
|----------|----------|----------|------------------|------------|-----------------|
| G06001   | S301     | 3        | Grocery          | 250        | Delivered       |
| G06002   | S301     | 2        | Dairy            | 180        | Delivered       |
| G06003   | S301     | 1        | Grocery          | 500        | Cancelled       |
| G06004   | S302     | 4        | Beverages        | 120        | Delivered       |
| G06005   | S302     | 2        | Grocery          | 350        | Failed          |
| G06033   | S301     | 2        | Grocery          | 250        | Delivered       |
| null     | S301     | 2        | Dairy            | 400        | Delivered       |
| G06040   | S302     | 0        | Beverages        | 200        | Delivered       |
| G06041   | S302     | 2        | Grocery          | -300       | Delivered       |

Sample Input — Stores

| store_id | store_name   | city   | store_type  | rating |
|----------|--------------|--------|-------------|--------|
| S301     | Fresh Mart   | Mumbai | Supermarket | 5      |
| S302     | Daily Needs  | Mumbai | Convenience | 3      |
| S303     | Green Basket | Delhi  | Supermarket | 5      |

### SAMPLE OUTPUT

| order_id | store_name  | city   | quantity | product_category | unit_price | delivery_status |
|----------|-------------|--------|----------|------------------|------------|-----------------|
| G06001   | Fresh Mart  | Mumbai | 3        | Grocery          | 250        | Delivered       |
| G06002   | Fresh Mart  | Mumbai | 2        | Dairy            | 180        | Delivered       |
| G06003   | Fresh Mart  | Mumbai | 1        | Grocery          | 500        | Cancelled       |
| G06004   | Daily Needs | Mumbai | 4        | Beverages        | 120        | Delivered       |
| G06005   | Daily Needs | Mumbai | 2        | Grocery          | 350        | Failed          |
| G06033   | Fresh Mart  | Mumbai | 2        | Grocery          | 250        | Delivered       |

### Function 4 — compute\_revenue(df\_joined)

#### SAMPLE INPUT

| order_id | store_name  | city   | quantity | product_category | unit_price | delivery_status |
|----------|-------------|--------|----------|------------------|------------|-----------------|
| G06001   | Fresh Mart  | Mumbai | 3        | Grocery          | 250        | Delivered       |
| G06002   | Fresh Mart  | Mumbai | 2        | Dairy            | 180        | Delivered       |
| G06003   | Fresh Mart  | Mumbai | 1        | Grocery          | 500        | Cancelled       |
| G06004   | Daily Needs | Mumbai | 4        | Beverages        | 120        | Delivered       |
| G06005   | Daily Needs | Mumbai | 2        | Grocery          | 350        | Failed          |
| G06033   | Fresh Mart  | Mumbai | 2        | Grocery          | 250        | Delivered       |

## SAMPLE OUTPUT

| order_id | store_name  | city   | quantity | product_category | unit_price | delivery_status | expected_revenue | realized_revenue |
|----------|-------------|--------|----------|------------------|------------|-----------------|------------------|------------------|
| G06001   | Fresh Mart  | Mumbai | 3        | Grocery          | 250        | Delivered       | 750              | 750              |
| G06002   | Fresh Mart  | Mumbai | 2        | Dairy            | 180        | Delivered       | 360              | 360              |
| G06003   | Fresh Mart  | Mumbai | 1        | Grocery          | 500        | Cancelled       | 500              | 0                |
| G06004   | Daily Needs | Mumbai | 4        | Beverages        | 120        | Delivered       | 480              | 480              |
| G06005   | Daily Needs | Mumbai | 2        | Grocery          | 350        | Failed          | 700              | 0                |
| G06033   | Fresh Mart  | Mumbai | 2        | Grocery          | 250        | Delivered       | 500              | 500              |

## Function 5 — classify\_and\_compute\_loss(df\_revenue)

### SAMPLE INPUT

| order_id | store_name  | city   | quantity | product_category | unit_price | delivery_status | expected_revenue | realized_revenue |
|----------|-------------|--------|----------|------------------|------------|-----------------|------------------|------------------|
| G06001   | Fresh Mart  | Mumbai | 3        | Grocery          | 250        | Delivered       | 750              | 750              |
| G06002   | Fresh Mart  | Mumbai | 2        | Dairy            | 180        | Delivered       | 360              | 360              |
| G06003   | Fresh Mart  | Mumbai | 1        | Grocery          | 500        | Cancelled       | 500              | 0                |
| G06004   | Daily Needs | Mumbai | 4        | Beverages        | 120        | Delivered       | 480              | 480              |
| G06005   | Daily Needs | Mumbai | 2        | Grocery          | 350        | Failed          | 700              | 0                |
| G06033   | Fresh Mart  | Mumbai | 2        | Grocery          | 250        | Delivered       | 500              | 500              |

### SAMPLE OUTPUT

| order_id | store_name  | city   | quantity | product_category | unit_price | delivery_status | expected_revenue | realized_revenue |
|----------|-------------|--------|----------|------------------|------------|-----------------|------------------|------------------|
| G06001   | Fresh Mart  | Mumbai | 3        | Grocery          | 250        | Delivered       | 750              | 750              |
| G06002   | Fresh Mart  | Mumbai | 2        | Dairy            | 180        | Delivered       | 360              | 360              |
| G06003   | Fresh Mart  | Mumbai | 1        | Grocery          | 500        | Cancelled       | 500              | 0                |
| G06004   | Daily Needs | Mumbai | 4        | Beverages        | 120        | Delivered       | 480              | 480              |
| G06005   | Daily Needs | Mumbai | 2        | Grocery          | 350        | Failed          | 700              | 0                |
| G06033   | Fresh Mart  | Mumbai | 2        | Grocery          | 250        | Delivered       | 500              | 500              |

## Function 6 — city\_category\_revenue\_summary(df\_classified)

### SAMPLE INPUT

| order_id | city   | product_category | expected_revenue | realized_revenue | delivery_flag     | revenue_loss |
|----------|--------|------------------|------------------|------------------|-------------------|--------------|
| G06001   | Mumbai | Grocery          | 750              | 750              | Completed         | 0            |
| G06002   | Mumbai | Dairy            | 360              | 360              | Completed         | 0            |
| G06003   | Mumbai | Grocery          | 500              | 0                | Lost-Cancellation | 500          |
| G06004   | Mumbai | Beverages        | 480              | 480              | Completed         | 0            |
| G06005   | Mumbai | Grocery          | 700              | 0                | Lost-Failed       | 700          |
| G06033   | Mumbai | Grocery          | 500              | 500              | Completed         | 0            |

## SAMPLE OUTPUT

The additional requirement here is avg\_realized\_revenue.

| city   | product_category | delivery_flag     | total_orders | total_expected_revenue | total_realized_revenue | total_revenue_loss |
|--------|------------------|-------------------|--------------|------------------------|------------------------|--------------------|
| Mumbai | Beverages        | Completed         | 1            | 480                    | 480                    | 0                  |
| Mumbai | Dairy            | Completed         | 1            | 360                    | 360                    | 0                  |
| Mumbai | Grocery          | Completed         | 2            | 1250                   | 1250                   | 0                  |
| Mumbai | Grocery          | Lost-Cancellation | 1            | 500                    | 0                      | 500                |
| Mumbai | Grocery          | Lost-Failed       | 1            | 700                    | 0                      | 700                |

## Q5. Retail Store Inventory Consumption & Replenishment Forecasting System

Part 2 · PySpark — Medium — question 5 of 16

### PROBLEM STATEMENT

A large retail company operates hundreds of stores across multiple cities. Each store manages inventory through multiple billing terminals and records product movement throughout the day, including sales, stock checks, returns, and other inventory-related activities. The warehouse operations team also maintains records of every stock replenishment performed at each store, including when the replenishment occurred and how much stock was delivered.

Over time, the company observed operational inefficiencies — some stores frequently experience stock shortages before the next scheduled replenishment, while others receive more inventory than required. To improve inventory planning, the analytics team wants to implement a data pipeline that studies store-level consumption patterns and recommends the approximate daily inventory requirement for each store.

Your task is to design and implement a PySpark pipeline that reads and processes both store transaction data and inventory replenishment data. The system should calculate important metrics such as daily product consumption, store-level replenishment frequency, average replenishment quantity, and an indicative daily inventory requirement for each store.

### OBJECTIVE

- Ingest and normalize store transaction and replenishment data from CSV files.
- Identify valid product-sale transactions and derive date-based information.
- Calculate daily consumption metrics for each store.
- Aggregate replenishment statistics at store level.
- Combine consumption and replenishment summaries to generate a store-level inventory forecast report using SQL.

### FILE STRUCTURE

```
Project
├── data
├── main.py
├── requirements.txt
├── solution.py
└── tests.py
```

### MODULES

- main.py — contains the driver code / main pipeline invocation.
- solution.py — must contain the implementation of all required functions.

### DATASET DESCRIPTION

Dataset 1 — store\_transactions.csv

| Column Name           | Type   | Description                                                                             |
|-----------------------|--------|-----------------------------------------------------------------------------------------|
| terminal_id           | string | Unique billing terminal identifier                                                      |
| store_id              | string | Store identifier                                                                        |
| transaction_timestamp | string | Date and time of transaction (YYYY-MM-DD HH:MM:SS)                                      |
| transaction_type      | string | Transaction type such as Sale, Return, Stock Check, etc.                                |
| transaction_amount    | double | Quantity/value associated with the transaction; positive values represent product sales |

## SAMPLE DATA

| terminal_id | store_id | transaction_timestamp | transaction_type | transaction_amount |
|-------------|----------|-----------------------|------------------|--------------------|
| T101        | S001     | 2025-08-01 09:15:00   | Sale             | 2500               |
| T101        | S001     | 2025-08-01 10:40:00   | Stock Check      | 0                  |
| T102        | S001     | 2025-08-01 12:20:00   | Sale             | 4000               |
| T102        | S001     | 2025-08-01 15:30:00   | Return           | 500                |
| T101        | S001     | 2025-08-02 09:10:00   | Sale             | 3000               |
| T201        | S002     | 2025-08-02 10:30:00   | Sale             | 2000               |
| T301        | S003     | 2025-08-04 09:00:00   | Sale             | 4500               |

## Dataset 2 — store\_replenishments.csv

| Column Name        | Type   | Description                          |
|--------------------|--------|--------------------------------------|
| terminal_id        | string | Terminal receiving the replenishment |
| store_id           | string | Store identifier                     |
| replenishment_date | date   | Date on which stock was replenished  |
| quantity_loaded    | double | Quantity of inventory delivered      |

## SAMPLE DATA

| terminal_id | store_id | replenishment_date | quantity_loaded |
|-------------|----------|--------------------|-----------------|
| T101        | S001     | 2025-08-01         | 200000          |
| T102        | S001     | 2025-08-02         | 180000          |
| T101        | S001     | 2025-08-05         | 190000          |
| T201        | S002     | 2025-08-02         | 150000          |
| T202        | S002     | 2025-08-03         | 170000          |

## IMPLEMENTATION REQUIREMENTS

- 1) load\_store\_transactions(spark: SparkSession, csv\_path: str) -> DataFrame

## PURPOSE

Read and normalize the store transaction CSV.

## INPUTS

`spark` — SparkSession

`csv_path` — path to store\_transactions.csv

Source Columns

terminal\_id

store\_id

transaction\_timestamp

transaction\_type

transaction\_amount

## RETURN

| Column                | Type      |
|-----------------------|-----------|
| terminal_id           | string    |
| store_id              | string    |
| transaction_timestamp | timestamp |
| transaction_type      | string    |
| transaction_amount    | double    |

## LOGIC

Read the CSV with header.

Convert transaction\_timestamp into a proper timestamp.

Convert transaction\_amount into double.

Keep only the five required columns.

Preserve the specified column order.

2) filter\_sales(df\_tx: DataFrame) -> DataFrame

## PURPOSE

Keep only valid product-sale transactions and derive a calendar date.

## INPUT

df\_tx returned by load\_store\_transactions().

## RETURN

All original columns plus:

- transaction\_date
- where transaction\_date is of type date.

## LOGIC

Retain only records where:

- transaction\_type represents a Sale
- transaction\_amount is strictly greater than zero.
- Derive transaction\_date from transaction\_timestamp.
- The derived field must contain only the date portion.
- 3) aggregate\_daily\_by\_store(df\_sales: DataFrame) -> DataFrame

## PURPOSE

Generate daily product-consumption KPIs for each store.

## INPUT

df\_sales from filter\_sales().

## RETURN

| Column           | Type   |
|------------------|--------|
| store_id         | string |
| transaction_date | date   |
| total_consumed   | double |
| sale_count       | long   |
| avg_sale         | double |

## LOGIC

For every:

- store\_id + transaction\_date

## CALCULATE

Sum of transaction\_amount → total\_consumed

Number of sales → sale\_count

Average sale amount → avg\_sale

Round avg\_sale to 2 decimal places

4) load\_replenishments(spark: SparkSession, csv\_path: str) -> DataFrame

## PURPOSE

Read and normalize store replenishment records.

## INPUTS

spark

csv\_path pointing to store\_replenishments.csv

Source Columns

terminal\_id

store\_id

replenishment\_date

quantity\_loaded

## RETURN

| Column             | Type   |
|--------------------|--------|
| terminal_id        | string |
| store_id           | string |
| replenishment_date | date   |
| quantity_loaded    | double |

## LOGIC

Read CSV with header.

Convert replenishment\_date to proper date type.

Convert quantity\_loaded to double.

Select only the four required columns.

5) compute\_store\_replenishment\_summary(df\_replenishments: DataFrame) -> DataFrame

## PURPOSE

Summarize replenishment frequency and average replenishment quantity for each store.

## INPUT

df\_replenishments from load\_replenishments().

## RETURN

| Column               | Type   |
|----------------------|--------|
| store_id             | string |
| total_replenishments | long   |
| avg_quantity_loaded  | double |

## LOGIC

For each store\_id:

- Count replenishment records → total\_replenishments
- Calculate average quantity\_loaded → avg\_quantity\_loaded
- Order output by store\_id ascending.
- 6) forecast\_daily\_inventory\_sql(df\_daily\_store: DataFrame, df\_store\_replenishment: DataFrame, spark: SparkSession) -> DataFrame

## PURPOSE

Generate an indicative daily inventory requirement by combining daily consumption trends with store replenishment statistics using Spark SQL.

## INPUTS

df\_daily\_store from aggregate\_daily\_by\_store()

df\_store\_replenishment from compute\_store\_replenishment\_summary()

spark

Temporary Views

Register:

- df\_daily\_store → daily
- df\_store\_replenishment → replenishments

## RETURN

| Column                    | Type   |
|---------------------------|--------|
| store_id                  | string |
| suggested_daily_inventory | double |
| replenishments_in_period  | long   |
| avg_replenishment_qty     | double |

## LOGIC

For every store\_id:

- Calculate the average total\_consumed across all available transaction dates.
- Multiply the average by a fixed 1.30 uplift factor.
- Round the result to 0 decimal places.

Alias it as:

- suggested\_daily\_inventory
- Left join the result with the replenishment summary.

Bring:

- replenishment count as replenishments\_in\_period
- average replenishment amount as avg\_replenishment\_qty
- Replace missing replenishment values with zero.

- Round avg\_replenishment\_qty to 0 decimals.
- Sort the final result by suggested\_daily\_inventory in descending order.
- Sample Input / Output
- forecast\_daily\_inventory\_sql(...) → DataFrame
- Sample Input — Daily Store Summary

| store_id | transaction_date | total_consumed | sale_count | avg_sale |
|----------|------------------|----------------|------------|----------|
| S001     | 2025-08-01       | 6500           | 2          | 3250     |
| S001     | 2025-08-02       | 3000           | 1          | 3000     |
| S002     | 2025-08-02       | 2000           | 1          | 2000     |
| S003     | 2025-08-04       | 4500           | 1          | 4500     |

#### Sample Input — Replenishment Summary

| store_id | total_replenishments | avg_quantity_loaded |
|----------|----------------------|---------------------|
| S001     | 3                    | 190000              |
| S002     | 2                    | 160000              |
| S003     | 0                    | 0                   |

#### EXPECTED OUTPUT

For S001:

- Average consumption =  $(6500 + 3000) / 2$
- = 4750
- Suggested inventory =  $4750 \times 1.30$
- = 6175

#### SAMPLE OUTPUT

| store_id | suggested_daily_inventory | replenishments_in_period | avg_replenishment_qty |
|----------|---------------------------|--------------------------|-----------------------|
| S001     | 6175                      | 3                        | 190000                |
| S003     | 5850                      | 0                        | 0                     |
| S002     | 2600                      | 2                        | 160000                |

1) load\_store\_transactions(spark, csv\_path) → DataFrame

#### SAMPLE INPUT

Input file: store\_transactions.csv

| terminal_id | store_id | transaction_timestamp | transaction_type | transaction_amount |
|-------------|----------|-----------------------|------------------|--------------------|
| T101        | S001     | 2025-08-01 09:15:00   | Sale             | 2500               |
| T101        | S001     | 2025-08-01 10:40:00   | Stock Check      | 0                  |
| T102        | S001     | 2025-08-01 12:20:00   | Sale             | 4000               |
| T102        | S001     | 2025-08-01 15:30:00   | Return           | 500                |
| T101        | S001     | 2025-08-02 09:10:00   | Sale             | 3000               |
| T201        | S002     | 2025-08-02 10:30:00   | Sale             | 2000               |
| T301        | S003     | 2025-08-04 09:00:00   | Sale             | 4500               |

#### SAMPLE OUTPUT

| terminal_id | store_id | transaction_timestamp | transaction_type | transaction_amount |
|-------------|----------|-----------------------|------------------|--------------------|
| T101        | S001     | 2025-08-01 09:15:00   | Sale             | 2500.0             |
| T101        | S001     | 2025-08-01 10:40:00   | Stock Check      | 0.0                |
| T102        | S001     | 2025-08-01 12:20:00   | Sale             | 4000.0             |
| T102        | S001     | 2025-08-01 15:30:00   | Return           | 500.0              |
| T101        | S001     | 2025-08-02 09:10:00   | Sale             | 3000.0             |
| T201        | S002     | 2025-08-02 10:30:00   | Sale             | 2000.0             |
| T301        | S003     | 2025-08-04 09:00:00   | Sale             | 4500.0             |

2) filter\_sales(df\_tx) → DataFrame

#### SAMPLE INPUT

| terminal_id | store_id | transaction_timestamp | transaction_type | transaction_amount |
|-------------|----------|-----------------------|------------------|--------------------|
| T101        | S001     | 2025-08-01 09:15:00   | Sale             | 2500.0             |
| T101        | S001     | 2025-08-01 10:40:00   | Stock Check      | 0.0                |
| T102        | S001     | 2025-08-01 12:20:00   | Sale             | 4000.0             |
| T102        | S001     | 2025-08-01 15:30:00   | Return           | 500.0              |
| T101        | S001     | 2025-08-02 09:10:00   | Sale             | 3000.0             |
| T201        | S002     | 2025-08-02 10:30:00   | Sale             | 2000.0             |
| T301        | S003     | 2025-08-04 09:00:00   | Sale             | 4500.0             |

#### SAMPLE OUTPUT

| terminal_id | store_id | transaction_timestamp | transaction_type | transaction_amount | transaction_date |
|-------------|----------|-----------------------|------------------|--------------------|------------------|
| T101        | S001     | 2025-08-01 09:15:00   | Sale             | 2500.0             | 2025-08-01       |
| T102        | S001     | 2025-08-01 12:20:00   | Sale             | 4000.0             | 2025-08-01       |
| T101        | S001     | 2025-08-02 09:10:00   | Sale             | 3000.0             | 2025-08-02       |
| T201        | S002     | 2025-08-02 10:30:00   | Sale             | 2000.0             | 2025-08-02       |
| T301        | S003     | 2025-08-04 09:00:00   | Sale             | 4500.0             | 2025-08-04       |

3) aggregate\_daily\_by\_store(df\_sales) → DataFrame

#### SAMPLE INPUT

| terminal_id | store_id | transaction_timestamp | transaction_type | transaction_amount | transaction_date |
|-------------|----------|-----------------------|------------------|--------------------|------------------|
| T101        | S001     | 2025-08-01 09:15:00   | Sale             | 2500.0             | 2025-08-01       |
| T102        | S001     | 2025-08-01 12:20:00   | Sale             | 4000.0             | 2025-08-01       |
| T101        | S001     | 2025-08-02 09:10:00   | Sale             | 3000.0             | 2025-08-02       |
| T201        | S002     | 2025-08-02 10:30:00   | Sale             | 2000.0             | 2025-08-02       |
| T301        | S003     | 2025-08-04 09:00:00   | Sale             | 4500.0             | 2025-08-04       |

#### SAMPLE OUTPUT

| store_id | transaction_date | total_consumed | sale_count | avg_sale |
|----------|------------------|----------------|------------|----------|
| S001     | 2025-08-01       | 6500.0         | 2          | 3250.00  |
| S001     | 2025-08-02       | 3000.0         | 1          | 3000.00  |
| S002     | 2025-08-02       | 2000.0         | 1          | 2000.00  |
| S003     | 2025-08-04       | 4500.0         | 1          | 4500.00  |

4) load\_replenishments(spark, csv\_path) → DataFrame

#### SAMPLE INPUT

Input file: store\_replenishments.csv

| terminal_id | store_id | replenishment_date | quantity_loaded |
|-------------|----------|--------------------|-----------------|
| T101        | S001     | 2025-08-01         | 200000          |
| T102        | S001     | 2025-08-02         | 180000          |
| T101        | S001     | 2025-08-05         | 190000          |
| T201        | S002     | 2025-08-02         | 150000          |
| T202        | S002     | 2025-08-03         | 170000          |

#### SAMPLE OUTPUT

| terminal_id | store_id | replenishment_date | quantity_loaded |
|-------------|----------|--------------------|-----------------|
| T101        | S001     | 2025-08-01         | 200000.0        |
| T102        | S001     | 2025-08-02         | 180000.0        |
| T101        | S001     | 2025-08-05         | 190000.0        |
| T201        | S002     | 2025-08-02         | 150000.0        |
| T202        | S002     | 2025-08-03         | 170000.0        |

5) compute\_store\_replenishment\_summary(df\_replenishments) → DataFrame

#### SAMPLE INPUT

| terminal_id | store_id | replenishment_date | quantity_loaded |
|-------------|----------|--------------------|-----------------|
| T101        | S001     | 2025-08-01         | 200000.0        |
| T102        | S001     | 2025-08-02         | 180000.0        |
| T101        | S001     | 2025-08-05         | 190000.0        |
| T201        | S002     | 2025-08-02         | 150000.0        |
| T202        | S002     | 2025-08-03         | 170000.0        |

#### SAMPLE OUTPUT

| store_id | total_replenishments | avg_quantity_loaded |
|----------|----------------------|---------------------|
| S001     | 3                    | 190000.00           |
| S002     | 2                    | 160000.00           |

6) forecast\_daily\_inventory\_sql(df\_daily\_store, df\_store\_replenishment, spark) → DataFrame

#### SAMPLE INPUT

Daily Store Summary

| store_id | transaction_date | total_consumed | sale_count | avg_sale |
|----------|------------------|----------------|------------|----------|
| S001     | 2025-08-01       | 6500.0         | 2          | 3250.00  |
| S001     | 2025-08-02       | 3000.0         | 1          | 3000.00  |
| S002     | 2025-08-02       | 2000.0         | 1          | 2000.00  |
| S003     | 2025-08-04       | 4500.0         | 1          | 4500.00  |

## Store Replenishment Summary

| store_id | total_replenishments | avg_quantity_loaded |
|----------|----------------------|---------------------|
| S001     | 3                    | 190000.00           |
| S002     | 2                    | 160000.00           |

## Calculation

For S001:

- Average consumption =  $(6500 + 3000) / 2$
- = 4750
- Suggested inventory =  $4750 \times 1.30$
- = 6175

For S002:

- Average consumption = 2000
- Suggested inventory =  $2000 \times 1.30$
- = 2600

For S003:

- Average consumption = 4500
- Suggested inventory =  $4500 \times 1.30$
- = 5850

## SAMPLE OUTPUT

| store_id | suggested_daily_inventory | replenishments_in_period | avg_replenishment_qty |
|----------|---------------------------|--------------------------|-----------------------|
| S001     | 6175                      | 3                        | 190000                |
| S003     | 5850                      | 0                        | 0                     |
| S002     | 2600                      | 2                        | 160000                |

## Q6. Fuel Station Consumption & Tank Refill Forecasting System

Part 2 · PySpark — Medium — question 6 of 16

### PROBLEM STATEMENT

A nationwide fuel company operates multiple fuel stations across different regions. Each station records fuel sales and other customer transactions throughout the day through its dispensing terminals. Along with sales information, the operations team records every tank replenishment, including the date of replenishment and the amount of fuel delivered to the station.

The company wants to improve fuel availability and reduce unnecessary inventory. Some stations consume fuel rapidly and require frequent deliveries, while others have comparatively lower demand. The analytics team therefore wants to build a PySpark-based pipeline that studies historical fuel consumption and replenishment activity and estimates the daily fuel requirement for each station.

Your task is to implement a PySpark data-processing pipeline that reads fuel transaction data and tank replenishment data from CSV files. The pipeline must identify valid fuel-sale transactions, calculate daily consumption metrics, summarize station replenishment behavior, and produce a final station-level fuel requirement forecast.

The final forecast should combine historical consumption with replenishment information and provide an indicative daily quantity that can be used by the operations team for planning future deliveries.

### OBJECTIVE

- Read and normalize fuel transaction data.
- Filter valid fuel-sale records.
- Derive transaction dates from timestamps.
- Calculate daily fuel consumption statistics by station.
- Analyze station-level tank replenishment activity.
- Combine consumption and replenishment information using Spark SQL.
- Produce a sorted daily fuel requirement report.

### FILE STRUCTURE

```
Project
├── data
├── main.py
├── requirements.txt
├── solution.py
└── tests.py
```

### MODULES

- main.py — driver program that invokes the pipeline.
- solution.py — contains all functions that must be implemented.

### DATASET DESCRIPTION

Dataset 1 — fuel\_transactions.csv

| Column Name      | Type   | Description                                                                               |
|------------------|--------|-------------------------------------------------------------------------------------------|
| dispenser_id     | string | Unique fuel dispenser identifier                                                          |
| station_id       | string | Fuel station identifier                                                                   |
| transaction_time | string | Date and time of fuel transaction                                                         |
| transaction_type | string | Type such as Fuel Sale, Refund, Inspection, etc.                                          |
| fuel_amount      | double | Fuel quantity/value associated with the transaction; positive values represent fuel sales |

## SAMPLE DATA

| dispenser_id | station_id | transaction_time    | transaction_type | fuel_amount |
|--------------|------------|---------------------|------------------|-------------|
| D101         | ST001      | 2025-09-01 08:20:00 | Fuel Sale        | 3200        |
| D101         | ST001      | 2025-09-01 10:15:00 | Inspection       | 0           |
| D102         | ST001      | 2025-09-01 12:40:00 | Fuel Sale        | 4500        |
| D102         | ST001      | 2025-09-01 15:20:00 | Refund           | 300         |
| D101         | ST001      | 2025-09-02 09:10:00 | Fuel Sale        | 2800        |
| D201         | ST002      | 2025-09-02 11:00:00 | Fuel Sale        | 2200        |
| D301         | ST003      | 2025-09-04 09:30:00 | Fuel Sale        | 5000        |

## Dataset 2 — tank\_refills.csv

| Column Name  | Type   | Description                          |
|--------------|--------|--------------------------------------|
| dispenser_id | string | Dispenser associated with the refill |
| station_id   | string | Fuel station identifier              |
| refill_date  | date   | Date of fuel replenishment           |
| fuel_loaded  | double | Amount of fuel delivered             |

## SAMPLE DATA

| dispenser_id | station_id | refill_date | fuel_loaded |
|--------------|------------|-------------|-------------|
| D101         | ST001      | 2025-09-01  | 210000      |
| D102         | ST001      | 2025-09-02  | 175000      |
| D101         | ST001      | 2025-09-05  | 195000      |
| D201         | ST002      | 2025-09-02  | 155000      |
| D202         | ST002      | 2025-09-03  | 165000      |

## IMPLEMENTATION REQUIREMENTS

- 1) load\_fuel\_transactions(spark: SparkSession, csv\_path: str) -> DataFrame

## PURPOSE

Load and standardize the fuel transaction dataset.

## INPUT

spark

csv\_path

Columns to use

dispenser\_id

station\_id

transaction\_time

transaction\_type

fuel\_amount

## RETURN

| Column           | Type      |
|------------------|-----------|
| dispenser_id     | string    |
| station_id       | string    |
| transaction_time | timestamp |
| transaction_type | string    |
| fuel_amount      | double    |

## LOGIC

Read CSV with header.

Convert transaction\_time to timestamp.

Convert fuel\_amount to double.

Select only the five required columns.

Maintain the specified order.

2) filter\_fuel\_sales(df\_tx: DataFrame) -> DataFrame

## PURPOSE

Keep only valid fuel-sale transactions and derive a transaction date.

## INPUT

df\_tx from load\_fuel\_transactions().

## RETURN

All input columns plus:

- sale\_date

## LOGIC

Retain records where:

```
transaction_type = "Fuel Sale"
```

and:

- fuel\_amount > 0

Derive:

- sale\_date

```
from transaction_time.
```

The new field must contain only the date component.

3) aggregate\_daily\_by\_station(df\_sales: DataFrame) -> DataFrame

## PURPOSE

Calculate daily fuel-consumption metrics for each station.

## INPUT

df\_sales from filter\_fuel\_sales().

## RETURN

| Column     | Type   |
|------------|--------|
| station_id | string |
| sale_date  | date   |

| Column          | Type   |
|-----------------|--------|
| total_fuel_sold | double |
| sale_count      | long   |
| avg_sale_amount | double |

## LOGIC

For every:

- station\_id + sale\_date

## CALCULATE

Sum of fuel\_amount → total\_fuel\_sold

Number of valid sales → sale\_count

Average fuel\_amount → avg\_sale\_amount

Round average sale amount to 2 decimal places

4) load\_tank\_refills(spark: SparkSession, csv\_path: str) -> DataFrame

## PURPOSE

Load and normalize fuel tank replenishment records.

## INPUT

spark

csv\_path

Columns

dispenser\_id

station\_id

refill\_date

fuel\_loaded

## RETURN

| Column       | Type   |
|--------------|--------|
| dispenser_id | string |
| station_id   | string |
| refill_date  | date   |
| fuel_loaded  | double |

## LOGIC

Read CSV with header.

Convert refill\_date to date.

Convert fuel\_loaded to double.

Select only the four required columns.

5) compute\_station\_refill\_summary(df\_refills: DataFrame) -> DataFrame

## PURPOSE

Summarize fuel-delivery frequency and average delivery quantity for each station.

## INPUT

df\_refills from load\_tank\_refills().

## RETURN

| Column          | Type   |
|-----------------|--------|
| station_id      | string |
| total_refills   | long   |
| avg_fuel_loaded | double |

## LOGIC

For each station\_id:

- Count refill records → total\_refills
- Calculate average fuel\_loaded → avg\_fuel\_loaded
- Sort output by station\_id ascending.
- 6) forecast\_daily\_fuel\_sql(df\_daily\_station: DataFrame, df\_station\_refill: DataFrame, spark: SparkSession) -> DataFrame

## PURPOSE

Create an indicative daily fuel requirement for each station by combining historical fuel consumption and refill statistics through Spark SQL.

## INPUTS

df\_daily\_station from aggregate\_daily\_by\_station()

df\_station\_refill from compute\_station\_refill\_summary()

spark

Temporary View Names

Register:

- df\_daily\_station → daily\_sales
- df\_station\_refill → station\_refills

## RETURN

| Column                 | Type   |
|------------------------|--------|
| station_id             | string |
| recommended_daily_fuel | double |
| refills_in_period      | long   |
| avg_refill_quantity    | double |

## LOGIC

For each station\_id:

- Calculate the average of total\_fuel\_sold over the available dates.
- Apply a fixed 1.30 safety factor.
- Round the result to 0 decimal places.

Name the result:

- recommended\_daily\_fuel
- Left join this result with the station refill summary.

Add:

- refill count as refills\_in\_period
- average refill quantity as avg\_refill\_quantity
- Replace missing refill values with zero.
- Round avg\_refill\_quantity to 0 decimal places.
- Sort final output by recommended\_daily\_fuel in descending order.
- Sample Input / Output
- forecast\_daily\_fuel\_sql(...) → DataFrame
- Sample Input — Daily Station Consumption

| station_id | sale_date  | total_fuel_sold | sale_count | avg_sale_amount |
|------------|------------|-----------------|------------|-----------------|
| ST001      | 2025-09-01 | 7700            | 2          | 3850            |
| ST001      | 2025-09-02 | 2800            | 1          | 2800            |
| ST002      | 2025-09-02 | 2200            | 1          | 2200            |
| ST003      | 2025-09-04 | 5000            | 1          | 5000            |

#### Sample Input — Station Refill Summary

| station_id | total_refills | avg_fuel_loaded |
|------------|---------------|-----------------|
| ST001      | 3             | 193333.33       |
| ST002      | 2             | 160000          |
| ST003      | 0             | 0               |

#### Forecast Calculation

For ST001:

- Average daily fuel sold
- =  $(7700 + 2800) / 2$
- = 5250
- Recommended daily fuel
- =  $5250 \times 1.30$
- = 6825

#### SAMPLE OUTPUT

| station_id | recommended_daily_fuel | refills_in_period | avg_refill_quantity |
|------------|------------------------|-------------------|---------------------|
| ST001      | 6825                   | 3                 | 193333              |
| ST003      | 6500                   | 0                 | 0                   |
| ST002      | 2860                   | 2                 | 160000              |

1) load\_fuel\_transactions(spark, csv\_path) → DataFrame

#### SAMPLE INPUT

Input file: fuel\_transactions.csv

| dispenser_id | station_id | transaction_time    | transaction_type | fuel_amount |
|--------------|------------|---------------------|------------------|-------------|
| D101         | ST001      | 2025-09-01 08:20:00 | Fuel Sale        | 3200        |
| D101         | ST001      | 2025-09-01 10:15:00 | Inspection       | 0           |
| D102         | ST001      | 2025-09-01 12:40:00 | Fuel Sale        | 4500        |
| D102         | ST001      | 2025-09-01 15:20:00 | Refund           | 300         |
| D101         | ST001      | 2025-09-02 09:10:00 | Fuel Sale        | 2800        |
| D201         | ST002      | 2025-09-02 11:00:00 | Fuel Sale        | 2200        |

| dispenser_id | station_id | transaction_time    | transaction_type | fuel_amount |
|--------------|------------|---------------------|------------------|-------------|
| D301         | ST003      | 2025-09-04 09:30:00 | Fuel Sale        | 5000        |

## SAMPLE OUTPUT

| dispenser_id | station_id | transaction_time    | transaction_type | fuel_amount |
|--------------|------------|---------------------|------------------|-------------|
| D101         | ST001      | 2025-09-01 08:20:00 | Fuel Sale        | 3200.0      |
| D101         | ST001      | 2025-09-01 10:15:00 | Inspection       | 0.0         |
| D102         | ST001      | 2025-09-01 12:40:00 | Fuel Sale        | 4500.0      |
| D102         | ST001      | 2025-09-01 15:20:00 | Refund           | 300.0       |
| D101         | ST001      | 2025-09-02 09:10:00 | Fuel Sale        | 2800.0      |
| D201         | ST002      | 2025-09-02 11:00:00 | Fuel Sale        | 2200.0      |
| D301         | ST003      | 2025-09-04 09:30:00 | Fuel Sale        | 5000.0      |

2) filter\_fuel\_sales(df\_tx) → DataFrame

## SAMPLE INPUT

| dispenser_id | station_id | transaction_time    | transaction_type | fuel_amount |
|--------------|------------|---------------------|------------------|-------------|
| D101         | ST001      | 2025-09-01 08:20:00 | Fuel Sale        | 3200.0      |
| D101         | ST001      | 2025-09-01 10:15:00 | Inspection       | 0.0         |
| D102         | ST001      | 2025-09-01 12:40:00 | Fuel Sale        | 4500.0      |
| D102         | ST001      | 2025-09-01 15:20:00 | Refund           | 300.0       |
| D101         | ST001      | 2025-09-02 09:10:00 | Fuel Sale        | 2800.0      |
| D201         | ST002      | 2025-09-02 11:00:00 | Fuel Sale        | 2200.0      |
| D301         | ST003      | 2025-09-04 09:30:00 | Fuel Sale        | 5000.0      |

## SAMPLE OUTPUT

| dispenser_id | station_id | transaction_time    | transaction_type | fuel_amount | sale_date  |
|--------------|------------|---------------------|------------------|-------------|------------|
| D101         | ST001      | 2025-09-01 08:20:00 | Fuel Sale        | 3200.0      | 2025-09-01 |
| D102         | ST001      | 2025-09-01 12:40:00 | Fuel Sale        | 4500.0      | 2025-09-01 |
| D101         | ST001      | 2025-09-02 09:10:00 | Fuel Sale        | 2800.0      | 2025-09-02 |
| D201         | ST002      | 2025-09-02 11:00:00 | Fuel Sale        | 2200.0      | 2025-09-02 |
| D301         | ST003      | 2025-09-04 09:30:00 | Fuel Sale        | 5000.0      | 2025-09-04 |

3) aggregate\_daily\_by\_station(df\_sales) → DataFrame

## SAMPLE INPUT

| dispenser_id | station_id | transaction_time    | transaction_type | fuel_amount | sale_date  |
|--------------|------------|---------------------|------------------|-------------|------------|
| D101         | ST001      | 2025-09-01 08:20:00 | Fuel Sale        | 3200.0      | 2025-09-01 |
| D102         | ST001      | 2025-09-01 12:40:00 | Fuel Sale        | 4500.0      | 2025-09-01 |
| D101         | ST001      | 2025-09-02 09:10:00 | Fuel Sale        | 2800.0      | 2025-09-02 |

| dispenser_id | station_id | transaction_time       | transaction_type | fuel_amount | sale_date  |
|--------------|------------|------------------------|------------------|-------------|------------|
| D201         | ST002      | 2025-09-02<br>11:00:00 | Fuel Sale        | 2200.0      | 2025-09-02 |
| D301         | ST003      | 2025-09-04<br>09:30:00 | Fuel Sale        | 5000.0      | 2025-09-04 |

### SAMPLE OUTPUT

| station_id | sale_date  | total_fuel_sold | sale_count | avg_sale_amount |
|------------|------------|-----------------|------------|-----------------|
| ST001      | 2025-09-01 | 7700.0          | 2          | 3850.00         |
| ST001      | 2025-09-02 | 2800.0          | 1          | 2800.00         |
| ST002      | 2025-09-02 | 2200.0          | 1          | 2200.00         |
| ST003      | 2025-09-04 | 5000.0          | 1          | 5000.00         |

4) load\_tank\_refills(spark, csv\_path) → DataFrame

### SAMPLE INPUT

Input file: tank\_refills.csv

| dispenser_id | station_id | refill_date | fuel_loaded |
|--------------|------------|-------------|-------------|
| D101         | ST001      | 2025-09-01  | 210000      |
| D102         | ST001      | 2025-09-02  | 175000      |
| D101         | ST001      | 2025-09-05  | 195000      |
| D201         | ST002      | 2025-09-02  | 155000      |
| D202         | ST002      | 2025-09-03  | 165000      |

### SAMPLE OUTPUT

| dispenser_id | station_id | refill_date | fuel_loaded |
|--------------|------------|-------------|-------------|
| D101         | ST001      | 2025-09-01  | 210000.0    |
| D102         | ST001      | 2025-09-02  | 175000.0    |
| D101         | ST001      | 2025-09-05  | 195000.0    |
| D201         | ST002      | 2025-09-02  | 155000.0    |
| D202         | ST002      | 2025-09-03  | 165000.0    |

5) compute\_station\_refill\_summary(df\_refills) → DataFrame

### SAMPLE INPUT

| dispenser_id | station_id | refill_date | fuel_loaded |
|--------------|------------|-------------|-------------|
| D101         | ST001      | 2025-09-01  | 210000.0    |
| D102         | ST001      | 2025-09-02  | 175000.0    |
| D101         | ST001      | 2025-09-05  | 195000.0    |
| D201         | ST002      | 2025-09-02  | 155000.0    |
| D202         | ST002      | 2025-09-03  | 165000.0    |

### SAMPLE OUTPUT

| station_id | total_refills | avg_fuel_loaded |
|------------|---------------|-----------------|
| ST001      | 3             | 193333.33       |

| station_id | total_refills | avg_fuel_loaded |
|------------|---------------|-----------------|
| ST002      | 2             | 160000.00       |

6) forecast\_daily\_fuel\_sql(df\_daily\_station, df\_station\_refill, spark) → DataFrame

### SAMPLE INPUT

Daily Station Consumption

| station_id | sale_date  | total_fuel_sold | sale_count | avg_sale_amount |
|------------|------------|-----------------|------------|-----------------|
| ST001      | 2025-09-01 | 7700.0          | 2          | 3850.00         |
| ST001      | 2025-09-02 | 2800.0          | 1          | 2800.00         |
| ST002      | 2025-09-02 | 2200.0          | 1          | 2200.00         |
| ST003      | 2025-09-04 | 5000.0          | 1          | 5000.00         |

Station Refill Summary

| station_id | total_refills | avg_fuel_loaded |
|------------|---------------|-----------------|
| ST001      | 3             | 193333.33       |
| ST002      | 2             | 160000.00       |

Calculation

For ST001:

- Average fuel sold =  $(7700 + 2800) / 2$
- = 5250
- Recommended daily fuel =  $5250 \times 1.30$
- = 6825

For ST002:

- Average fuel sold = 2200
- Recommended daily fuel =  $2200 \times 1.30$
- = 2860

For ST003:

- Average fuel sold = 5000
- Recommended daily fuel =  $5000 \times 1.30$
- = 6500

### SAMPLE OUTPUT

| station_id | recommended_daily_fuel | refills_in_period | avg_refill_quantity |
|------------|------------------------|-------------------|---------------------|
| ST001      | 6825                   | 3                 | 193333              |
| ST003      | 6500                   | 0                 | 0                   |
| ST002      | 2860                   | 2                 | 160000              |

## Q7. E-Commerce Customer Purchase Analytics & Customer Segmentation

Part 2 · PySpark — Medium — question 7 of 16

### PROBLEM STATEMENT

A large e-commerce company maintains detailed records of customer purchases made through its online platform. The purchase database stores information such as transaction dates, product categories, quantities, prices, and payment methods. Meanwhile, the customer database contains customer profiles including names, locations, registration years, and customer types.

The company wants to understand customer purchasing behavior and identify its most valuable customers. The analytics team needs to develop a PySpark-based data pipeline that integrates purchase and customer data, removes invalid transactions, computes customer-level purchase metrics, and summarizes sales based on customer registration year.

The final output should categorize customers into contribution segments such as Platinum, Gold, Silver, or Bronze based on their total lifetime purchase value.

### OBJECTIVE

- Load and normalize customer and purchase datasets.
- Join customer and purchase data while filtering invalid or zero-value purchases.
- Compute key customer-level metrics: total spending, number of purchases, and average purchase amount.
- Aggregate total and average purchase amounts by customer registration year.
- Categorize customers into contribution segments using SQL CASE WHEN logic.
- Produce clean DataFrames suitable for further analysis.

### FILE STRUCTURE

```
Project
├── data
│   ├── purchases.csv
│   └── customers.csv
├── main.py
├── requirements.txt
├── solution.py
└── tests.py
```

### MODULES

- main.py – contains the driver code / main invocation pipeline.
- solution.py – Must complete all the required functions.

### DATASET DESCRIPTION

Dataset 1 – purchases.csv

| Column Name    | Type   | Description                            |
|----------------|--------|----------------------------------------|
| purchase_id    | string | Unique purchase transaction identifier |
| customer_id    | string | Unique customer identifier             |
| purchase_date  | date   | Date of purchase (YYYY-MM-DD)          |
| amount         | double | Purchase amount                        |
| payment_method | string | Payment method used                    |
| category       | string | Product category                       |

### SAMPLE DATA

| purchase_id | customer_id | purchase_date | amount | payment_method | category    |
|-------------|-------------|---------------|--------|----------------|-------------|
| P001        | C001        | 2024-01-15    | 5000   | Card           | Electronics |
| P002        | C002        | 2024-02-10    | 15000  | Bank           | Furniture   |
| P003        | C003        | 2024-03-05    | 2500   | UPI            | Clothing    |
| P004        | C004        | 2024-03-12    | 50000  | Card           | Electronics |
| P005        | C005        | 2024-03-20    | 0      | UPI            | Grocery     |
| P006        | C006        | 2024-04-02    | 12000  | Bank           | Electronics |
| P007        | C007        | 2024-04-18    | 3000   | Card           | Clothing    |
| P010        | C010        | 2024-05-21    | 25000  | Card           | Furniture   |
| P016        | C001        | 2024-08-10    | 3000   | Card           | Clothing    |
| P019        | C010        | 2024-09-15    | 50000  | Card           | Electronics |
| P022        | C006        | 2024-10-20    | 6000   | Card           | Grocery     |

## Dataset 2 – customers.csv

| Column Name       | Type    | Description                |
|-------------------|---------|----------------------------|
| customer_id       | string  | Unique customer identifier |
| full_name         | string  | Full name of the customer  |
| email             | string  | Registered customer email  |
| registration_year | integer | Year customer registered   |
| customer_type     | string  | Type of customer           |
| city              | string  | Current city               |
| country           | string  | Country of residence       |

## SAMPLE DATA

| customer_id | full_name     | email              | registration_year | customer_type | city      | country |
|-------------|---------------|--------------------|-------------------|---------------|-----------|---------|
| C001        | Ananya Rao    | ananya@example.com | 2018              | Premium       | Bengaluru | India   |
| C002        | Rahul Mehta   | rahul@example.com  | 2020              | Regular       | Mumbai    | India   |
| C003        | Kavita Shah   | kavita@example.com | 2019              | Regular       | Delhi     | India   |
| C004        | Daniel Thomas | daniel@example.com | 2016              | Premium       | Kochi     | India   |
| C005        | Arjun Reddy   | arjun@example.com  | 2021              | Regular       | Hyderabad | India   |
| C006        | Meera Nair    | meera@example.com  | 2017              | Premium       | Chennai   | India   |
| C007        | Sneha Kapoor  | sneha@example.com  | 2020              | Regular       | Pune      | India   |
| C010        | Suresh Pillai | suresh@example.com | 2015              | Premium       | Kochi     | India   |

## IMPLEMENTATION REQUIREMENTS

### FUNCTIONS TO IMPLEMENT IN SOLUTION.PY

- 1) load\_purchases(spark: SparkSession, purchases\_csv\_path: str) → DataFrame

### PURPOSE

Load purchase transactions and ensure that purchase amounts are properly typed as numeric values.

### INPUTS

SparkSession object

Path to the purchases.csv file

## RETURNS

A DataFrame containing:

- purchase\_id, customer\_id, purchase\_date, amount, payment\_method, category

## LOGIC

Read the CSV file with headers.

Convert amount to double type.

Retain only relevant columns required for analysis.

## SAMPLE I/O

### SAMPLE INPUT

| purchase_id | customer_id | purchase_date | amount | payment_method | category    |
|-------------|-------------|---------------|--------|----------------|-------------|
| P001        | C001        | 2024-01-15    | 5000   | Card           | Electronics |
| P003        | C003        | 2024-03-05    | 2500   | UPI            | Clothing    |
| P005        | C005        | 2024-03-20    | 0      | UPI            | Grocery     |
| P006        | C006        | 2024-04-02    | 12000  | Bank           | Electronics |
| P010        | C010        | 2024-05-21    | 25000  | Card           | Furniture   |
| P016        | C001        | 2024-08-10    | 3000   | Card           | Clothing    |

### SAMPLE OUTPUT

| purchase_id | customer_id | purchase_date | amount  | payment_method | category    |
|-------------|-------------|---------------|---------|----------------|-------------|
| P001        | C001        | 2024-01-15    | 5000.0  | Card           | Electronics |
| P003        | C003        | 2024-03-05    | 2500.0  | UPI            | Clothing    |
| P005        | C005        | 2024-03-20    | 0.0     | UPI            | Grocery     |
| P006        | C006        | 2024-04-02    | 12000.0 | Bank           | Electronics |
| P010        | C010        | 2024-05-21    | 25000.0 | Card           | Furniture   |
| P016        | C001        | 2024-08-10    | 3000.0  | Card           | Clothing    |

2) load\_customers(spark: SparkSession, customers\_csv\_path: str) → DataFrame

## PURPOSE

Load customer profile data and ensure that numeric columns have the correct data types.

## INPUTS

SparkSession object

Path to the customers.csv file

## RETURNS

A DataFrame containing:

- customer\_id, full\_name, email, registration\_year, customer\_type, city, country

## LOGIC

Read the CSV file with headers.

Convert registration\_year to integer.

Return the cleaned DataFrame.

### SAMPLE I/O

#### SAMPLE INPUT

| customer_id | full_name     | email              | registration_year | customer_type | city      | country |
|-------------|---------------|--------------------|-------------------|---------------|-----------|---------|
| C001        | Ananya Rao    | ananya@example.com | 2018              | Premium       | Bengaluru | India   |
| C003        | Kavita Shah   | kavita@example.com | 2019              | Regular       | Delhi     | India   |
| C004        | Daniel Thomas | daniel@example.com | 2016              | Premium       | Kochi     | India   |
| C006        | Meera Nair    | meera@example.com  | 2017              | Premium       | Chennai   | India   |
| C010        | Suresh Pillai | suresh@example.com | 2015              | Premium       | Kochi     | India   |

#### SAMPLE OUTPUT

| customer_id | full_name     | email              | registration_year | customer_type | city      | country |
|-------------|---------------|--------------------|-------------------|---------------|-----------|---------|
| C001        | Ananya Rao    | ananya@example.com | 2018              | Premium       | Bengaluru | India   |
| C003        | Kavita Shah   | kavita@example.com | 2019              | Regular       | Delhi     | India   |
| C004        | Daniel Thomas | daniel@example.com | 2016              | Premium       | Kochi     | India   |
| C006        | Meera Nair    | meera@example.com  | 2017              | Premium       | Chennai   | India   |
| C010        | Suresh Pillai | suresh@example.com | 2015              | Premium       | Kochi     | India   |

3) join\_and\_clean(purchases\_df: DataFrame, customers\_df: DataFrame) → DataFrame

### PURPOSE

Combine customer and purchase datasets and remove invalid or zero-value purchases.

### INPUTS

purchases\_df: Purchase DataFrame from load\_purchases

customers\_df: Customer DataFrame from load\_customers

### RETURNS

A DataFrame containing merged and validated records.

### LOGIC

Perform an inner join using customer\_id.

Filter out records where amount <= 0.

Keep the combined attributes required for further analysis.

### SAMPLE I/O

Sample Input — Purchases

| purchase_id | customer_id | purchase_date | amount  | payment_method | category    |
|-------------|-------------|---------------|---------|----------------|-------------|
| P001        | C001        | 2024-01-15    | 5000.0  | Card           | Electronics |
| P002        | C002        | 2024-02-10    | 15000.0 | Bank           | Furniture   |
| P005        | C005        | 2024-03-20    | 0.0     | UPI            | Grocery     |
| P006        | C006        | 2024-04-02    | 12000.0 | Bank           | Electronics |
| P010        | C010        | 2024-05-21    | 25000.0 | Card           | Furniture   |

| purchase_id | customer_id | purchase_date | amount  | payment_method | category    |
|-------------|-------------|---------------|---------|----------------|-------------|
| P016        | C001        | 2024-08-10    | 3000.0  | Card           | Clothing    |
| P019        | C010        | 2024-09-15    | 50000.0 | Card           | Electronics |
| P022        | C006        | 2024-10-20    | 6000.0  | Card           | Grocery     |

#### Sample Input — Customers

| customer_id | full_name     | registration_year | customer_type | city      |
|-------------|---------------|-------------------|---------------|-----------|
| C001        | Ananya Rao    | 2018              | Premium       | Bengaluru |
| C002        | Rahul Mehta   | 2020              | Regular       | Mumbai    |
| C005        | Arjun Reddy   | 2021              | Regular       | Hyderabad |
| C006        | Meera Nair    | 2017              | Premium       | Chennai   |
| C010        | Suresh Pillai | 2015              | Premium       | Kochi     |

#### SAMPLE OUTPUT

The zero-value purchase P005 is removed.

| customer_id | purchase_id | purchase_date | amount  | payment_method | category    | full_name     | registration_year | customer_type |
|-------------|-------------|---------------|---------|----------------|-------------|---------------|-------------------|---------------|
| C001        | P001        | 2024-01-15    | 5000.0  | Card           | Electronics | Ananya Rao    | 2018              | Premium       |
| C001        | P016        | 2024-08-10    | 3000.0  | Card           | Clothing    | Ananya Rao    | 2018              | Premium       |
| C002        | P002        | 2024-02-10    | 15000.0 | Bank           | Furniture   | Rahul Mehta   | 2020              | Regular       |
| C006        | P006        | 2024-04-02    | 12000.0 | Bank           | Electronics | Meera Nair    | 2017              | Premium       |
| C006        | P022        | 2024-10-20    | 6000.0  | Card           | Grocery     | Meera Nair    | 2017              | Premium       |
| C010        | P010        | 2024-05-21    | 25000.0 | Card           | Furniture   | Suresh Pillai | 2015              | Premium       |
| C010        | P019        | 2024-09-15    | 50000.0 | Card           | Electronics | Suresh Pillai | 2015              | Premium       |

4) compute\_customer\_metrics(joined\_df: DataFrame) → DataFrame

#### PURPOSE

Generate customer-level purchase summary metrics.

#### INPUTS

joined\_df: Cleaned joined DataFrame from join\_and\_clean

#### RETURNS

A DataFrame containing:

- customer\_id, total\_spent, purchase\_count, avg\_purchase

#### LOGIC

Group records by customer\_id.

Calculate total purchase amount as total\_spent.

Count purchases as purchase\_count.

Calculate average purchase as avg\_purchase.

**SAMPLE I/O****SAMPLE INPUT**

| customer_id | purchase_id | amount  |
|-------------|-------------|---------|
| C001        | P001        | 5000.0  |
| C001        | P016        | 3000.0  |
| C002        | P002        | 15000.0 |
| C006        | P006        | 12000.0 |
| C006        | P022        | 6000.0  |
| C010        | P010        | 25000.0 |
| C010        | P019        | 50000.0 |

**SAMPLE OUTPUT**

| customer_id | total_spent | purchase_count | avg_purchase |
|-------------|-------------|----------------|--------------|
| C001        | 8000.0      | 2              | 4000.0       |
| C002        | 15000.0     | 1              | 15000.0      |
| C006        | 18000.0     | 2              | 9000.0       |
| C010        | 75000.0     | 2              | 37500.0      |

5) aggregate\_by\_registration\_year(joined\_df: DataFrame) → DataFrame

**PURPOSE**

Summarize purchase metrics grouped by customer registration year.

**INPUTS**

joined\_df: DataFrame from join\_and\_clean

**RETURNS**

A DataFrame containing:

- registration\_year, total\_amount, average\_amount

**LOGIC**

Group records by registration\_year.

Compute total purchase amount as total\_amount.

Compute average purchase amount as average\_amount.

Order results by registration\_year in ascending order.

**SAMPLE I/O****SAMPLE INPUT**

| customer_id | amount  | registration_year |
|-------------|---------|-------------------|
| C001        | 5000.0  | 2018              |
| C001        | 3000.0  | 2018              |
| C002        | 15000.0 | 2020              |
| C006        | 12000.0 | 2017              |
| C006        | 6000.0  | 2017              |
| C010        | 25000.0 | 2015              |

| customer_id | amount  | registration_year |
|-------------|---------|-------------------|
| C010        | 50000.0 | 2015              |

## SAMPLE OUTPUT

| registration_year | total_amount | average_amount |
|-------------------|--------------|----------------|
| 2015              | 75000.0      | 37500.0        |
| 2017              | 18000.0      | 9000.0         |
| 2018              | 8000.0       | 4000.0         |
| 2020              | 15000.0      | 15000.0        |

6) segment\_customers\_sql(customer\_metrics\_df: DataFrame, spark: SparkSession) → DataFrame

## PURPOSE

Categorize customers into segments based on their total purchase value using SQL logic.

## INPUTS

customer\_metrics\_df: DataFrame from compute\_customer\_metrics

spark: SparkSession for SQL operations

## RETURNS

A DataFrame containing:

- customer\_id, total\_spent, purchase\_count, avg\_purchase, segment

## LOGIC

Register the DataFrame as a temporary SQL view named customer\_metrics.

Apply the following CASE WHEN logic:

- Platinum: total\_spent >= 100000
- Gold: total\_spent >= 25000 AND total\_spent < 100000
- Silver: total\_spent >= 5000 AND total\_spent < 25000
- Bronze: total\_spent < 5000

## SAMPLE I/O

### SAMPLE INPUT

| customer_id | total_spent | purchase_count | avg_purchase |
|-------------|-------------|----------------|--------------|
| C001        | 8000.0      | 2              | 4000.0       |
| C002        | 15000.0     | 1              | 15000.0      |
| C006        | 18000.0     | 2              | 9000.0       |
| C010        | 75000.0     | 2              | 37500.0      |

## SAMPLE OUTPUT

| customer_id | total_spent | purchase_count | avg_purchase | segment |
|-------------|-------------|----------------|--------------|---------|
| C001        | 8000.0      | 2              | 4000.0       | Silver  |
| C002        | 15000.0     | 1              | 15000.0      | Silver  |
| C006        | 18000.0     | 2              | 9000.0       | Silver  |
| C010        | 75000.0     | 2              | 37500.0      | Gold    |



## Q8. Hospital Patient Billing Analytics & Patient Segmentation

Part 2 · PySpark — Medium — question 8 of 16

### PROBLEM STATEMENT

A multi-specialty hospital maintains records of patient billing transactions for consultations, diagnostic tests, procedures, and other medical services. Each billing record contains the service date, billed amount, payment method, and service category. A separate patient database contains patient information such as registration year, department, city, and patient type.

Hospital management wants to analyze patient spending patterns and identify patients who contribute significantly to the hospital's overall revenue. The analytics team needs to build a PySpark pipeline that integrates billing and patient data, removes invalid billing records, calculates patient-level billing metrics, and summarizes billing amounts by patient registration year.

The final output should categorize patients into contribution segments such as Platinum, Gold, Silver, or Bronze based on their cumulative billing amount.

### OBJECTIVE

- Load and normalize billing and patient datasets.
- Join patient and billing data.
- Remove invalid or zero-value billing records.
- Calculate total billing, billing count, and average billing amount for each patient.
- Aggregate billing information by patient registration year.
- Categorize patients into contribution segments using SQL CASE WHEN.

### FILE STRUCTURE

```
Project
├── data
│   ├── billing.csv
│   └── patients.csv
├── main.py
├── requirements.txt
├── solution.py
└── tests.py
```

### MODULES

- main.py – contains the driver code / main invocation pipeline.
- solution.py – Must complete all required functions.

### DATASET DESCRIPTION

Dataset 1 – billing.csv

| Column Name    | Type   | Description                           |
|----------------|--------|---------------------------------------|
| bill_id        | string | Unique billing transaction identifier |
| patient_id     | string | Unique patient identifier             |
| service_date   | date   | Date of hospital service              |
| amount         | double | Billing amount                        |
| payment_method | string | Payment method                        |
| service_type   | string | Type of medical service               |

### SAMPLE DATA

| bill_id | patient_id | service_date | amount | payment_method | service_type |
|---------|------------|--------------|--------|----------------|--------------|
| B001    | P001       | 2024-01-15   | 5000   | Card           | Consultation |
| B002    | P002       | 2024-02-10   | 15000  | Bank           | Surgery      |
| B003    | P003       | 2024-03-05   | 2500   | UPI            | Diagnostics  |
| B004    | P004       | 2024-03-12   | 50000  | Card           | Surgery      |
| B005    | P005       | 2024-03-20   | 0      | UPI            | Pharmacy     |
| B006    | P006       | 2024-04-02   | 12000  | Bank           | Diagnostics  |
| B007    | P007       | 2024-04-18   | 3000   | Card           | Consultation |
| B010    | P010       | 2024-05-21   | 25000  | Card           | Surgery      |
| B016    | P001       | 2024-08-10   | 3000   | Card           | Diagnostics  |
| B019    | P010       | 2024-09-15   | 50000  | Card           | Surgery      |
| B022    | P006       | 2024-10-20   | 6000   | Card           | Pharmacy     |

## Dataset 2 – patients.csv

| Column Name       | Type    | Description                 |
|-------------------|---------|-----------------------------|
| patient_id        | string  | Unique patient identifier   |
| full_name         | string  | Full name of patient        |
| email             | string  | Registered patient email    |
| registration_year | integer | Year patient registered     |
| patient_type      | string  | Patient category            |
| department        | string  | Primary hospital department |
| city              | string  | Patient city                |
| country           | string  | Country                     |

## SAMPLE DATA

| patient_id | full_name     | email              | registration_year | patient_type | department       | city      | country |
|------------|---------------|--------------------|-------------------|--------------|------------------|-----------|---------|
| P001       | Amit Sharma   | amit@example.com   | 2018              | Regular      | Cardiology       | Bengaluru | India   |
| P002       | Rahul Mehta   | rahul@example.com  | 2020              | Regular      | Orthopedics      | Mumbai    | India   |
| P003       | Kavita Shah   | kavita@example.com | 2019              | Regular      | Diagnostics      | Delhi     | India   |
| P004       | Daniel Thomas | daniel@example.com | 2016              | Premium      | Surgery          | Kochi     | India   |
| P005       | Arjun Reddy   | arjun@example.com  | 2021              | Regular      | General Medicine | Hyderabad | India   |
| P006       | Meera Nair    | meera@example.com  | 2017              | Premium      | Diagnostics      | Chennai   | India   |
| P007       | Sneha Kapoor  | sneha@example.com  | 2020              | Regular      | Cardiology       | Pune      | India   |
| P010       | Suresh Pillai | suresh@example.com | 2015              | Premium      | Surgery          | Kochi     | India   |

## IMPLEMENTATION REQUIREMENTS

### FUNCTIONS TO IMPLEMENT IN SOLUTION.PY

- 1) load\_billing(spark: SparkSession, billing\_csv\_path: str) → DataFrame

### PURPOSE

Load hospital billing transactions and ensure that billing amounts are properly typed as numeric values.

## INPUTS

SparkSession object

Path to billing.csv

## RETURNS

A DataFrame containing:

- bill\_id, patient\_id, service\_date, amount, payment\_method, service\_type

## LOGIC

Read CSV with headers.

Convert amount to double.

Retain relevant columns.

## SAMPLE I/O

### SAMPLE INPUT

| bill_id | patient_id | service_date | amount | payment_method | service_type |
|---------|------------|--------------|--------|----------------|--------------|
| B001    | P001       | 2024-01-15   | 5000   | Card           | Consultation |
| B003    | P003       | 2024-03-05   | 2500   | UPI            | Diagnostics  |
| B005    | P005       | 2024-03-20   | 0      | UPI            | Pharmacy     |
| B006    | P006       | 2024-04-02   | 12000  | Bank           | Diagnostics  |
| B010    | P010       | 2024-05-21   | 25000  | Card           | Surgery      |
| B016    | P001       | 2024-08-10   | 3000   | Card           | Diagnostics  |

### SAMPLE OUTPUT

| bill_id | patient_id | service_date | amount  | payment_method | service_type |
|---------|------------|--------------|---------|----------------|--------------|
| B001    | P001       | 2024-01-15   | 5000.0  | Card           | Consultation |
| B003    | P003       | 2024-03-05   | 2500.0  | UPI            | Diagnostics  |
| B005    | P005       | 2024-03-20   | 0.0     | UPI            | Pharmacy     |
| B006    | P006       | 2024-04-02   | 12000.0 | Bank           | Diagnostics  |
| B010    | P010       | 2024-05-21   | 25000.0 | Card           | Surgery      |
| B016    | P001       | 2024-08-10   | 3000.0  | Card           | Diagnostics  |

2) load\_patients(spark: SparkSession, patients\_csv\_path: str) → DataFrame

## PURPOSE

Load patient profile data and ensure numeric columns have the correct types.

## INPUTS

SparkSession object

Path to patients.csv

## RETURNS

A DataFrame containing:

- patient\_id, full\_name, email, registration\_year, patient\_type, department, city, country

## LOGIC

Read CSV with headers.

Convert registration\_year to integer.

Return cleaned DataFrame.

## SAMPLE I/O

### SAMPLE INPUT

| patient_id | full_name     | email              | registration_year | patient_type | department  | city      | country |
|------------|---------------|--------------------|-------------------|--------------|-------------|-----------|---------|
| P001       | Amit Sharma   | amit@example.com   | 2018              | Regular      | Cardiology  | Bengaluru | India   |
| P003       | Kavita Shah   | kavita@example.com | 2019              | Regular      | Diagnostics | Delhi     | India   |
| P004       | Daniel Thomas | daniel@example.com | 2016              | Premium      | Surgery     | Kochi     | India   |
| P006       | Meera Nair    | meera@example.com  | 2017              | Premium      | Diagnostics | Chennai   | India   |
| P010       | Suresh Pillai | suresh@example.com | 2015              | Premium      | Surgery     | Kochi     | India   |

### SAMPLE OUTPUT

| patient_id | full_name     | email              | registration_year | patient_type | department  | city      | country |
|------------|---------------|--------------------|-------------------|--------------|-------------|-----------|---------|
| P001       | Amit Sharma   | amit@example.com   | 2018              | Regular      | Cardiology  | Bengaluru | India   |
| P003       | Kavita Shah   | kavita@example.com | 2019              | Regular      | Diagnostics | Delhi     | India   |
| P004       | Daniel Thomas | daniel@example.com | 2016              | Premium      | Surgery     | Kochi     | India   |
| P006       | Meera Nair    | meera@example.com  | 2017              | Premium      | Diagnostics | Chennai   | India   |
| P010       | Suresh Pillai | suresh@example.com | 2015              | Premium      | Surgery     | Kochi     | India   |

3) join\_and\_clean(billing\_df: DataFrame, patients\_df: DataFrame) → DataFrame

## PURPOSE

Combine patient and billing datasets and remove invalid or zero-value billing records.

## INPUTS

billing\_df: Billing DataFrame from load\_billing

patients\_df: Patient DataFrame from load\_patients

## RETURNS

A DataFrame containing merged and validated records.

## LOGIC

Perform an inner join on patient\_id.

Filter records where amount <= 0.

Keep combined attributes required for analysis.

## SAMPLE I/O

## Sample Input — Billing

| bill_id | patient_id | service_date | amount  | payment_method | service_type |
|---------|------------|--------------|---------|----------------|--------------|
| B001    | P001       | 2024-01-15   | 5000.0  | Card           | Consultation |
| B002    | P002       | 2024-02-10   | 15000.0 | Bank           | Surgery      |
| B005    | P005       | 2024-03-20   | 0.0     | UPI            | Pharmacy     |
| B006    | P006       | 2024-04-02   | 12000.0 | Bank           | Diagnostics  |
| B010    | P010       | 2024-05-21   | 25000.0 | Card           | Surgery      |
| B016    | P001       | 2024-08-10   | 3000.0  | Card           | Diagnostics  |
| B019    | P010       | 2024-09-15   | 50000.0 | Card           | Surgery      |
| B022    | P006       | 2024-10-20   | 6000.0  | Card           | Pharmacy     |

## Sample Input — Patients

| patient_id | full_name     | registration_year | patient_type | department       | city      |
|------------|---------------|-------------------|--------------|------------------|-----------|
| P001       | Amit Sharma   | 2018              | Regular      | Cardiology       | Bengaluru |
| P002       | Rahul Mehta   | 2020              | Regular      | Orthopedics      | Mumbai    |
| P005       | Arjun Reddy   | 2021              | Regular      | General Medicine | Hyderabad |
| P006       | Meera Nair    | 2017              | Premium      | Diagnostics      | Chennai   |
| P010       | Suresh Pillai | 2015              | Premium      | Surgery          | Kochi     |

## SAMPLE OUTPUT

The zero-value bill B005 is removed.

| patient_id | bill_id | service_date | amount  | payment_method | service_type | full_name     | registration_year | patient_type | department  |
|------------|---------|--------------|---------|----------------|--------------|---------------|-------------------|--------------|-------------|
| P001       | B001    | 2024-01-15   | 5000.0  | Card           | Consultation | Amit Sharma   | 2018              | Regular      | Cardiology  |
| P001       | B016    | 2024-08-10   | 3000.0  | Card           | Diagnostics  | Amit Sharma   | 2018              | Regular      | Cardiology  |
| P002       | B002    | 2024-02-10   | 15000.0 | Bank           | Surgery      | Rahul Mehta   | 2020              | Regular      | Orthopedics |
| P006       | B006    | 2024-04-02   | 12000.0 | Bank           | Diagnostics  | Meera Nair    | 2017              | Premium      | Diagnostics |
| P006       | B022    | 2024-10-20   | 6000.0  | Card           | Pharmacy     | Meera Nair    | 2017              | Premium      | Diagnostics |
| P010       | B010    | 2024-05-21   | 25000.0 | Card           | Surgery      | Suresh Pillai | 2015              | Premium      | Surgery     |
| P010       | B019    | 2024-09-15   | 50000.0 | Card           | Surgery      | Suresh Pillai | 2015              | Premium      | Surgery     |

4) compute\_patient\_metrics(joined\_df: DataFrame) → DataFrame

## PURPOSE

Generate patient-level billing summary metrics.

## INPUTS

joined\_df: Cleaned joined DataFrame from join\_and\_clean

## RETURNS

A DataFrame containing:

- patient\_id, total\_billed, bill\_count, avg\_bill

## LOGIC

Group records by patient\_id.

Calculate total billing amount as total\_billed.

Count billing transactions as bill\_count.

Calculate average billing amount as avg\_bill.

## SAMPLE I/O

### SAMPLE INPUT

| patient_id | bill_id | amount  |
|------------|---------|---------|
| P001       | B001    | 5000.0  |
| P001       | B016    | 3000.0  |
| P002       | B002    | 15000.0 |
| P006       | B006    | 12000.0 |
| P006       | B022    | 6000.0  |
| P010       | B010    | 25000.0 |
| P010       | B019    | 50000.0 |

### SAMPLE OUTPUT

| patient_id | total_billed | bill_count | avg_bill |
|------------|--------------|------------|----------|
| P001       | 8000.0       | 2          | 4000.0   |
| P002       | 15000.0      | 1          | 15000.0  |
| P006       | 18000.0      | 2          | 9000.0   |
| P010       | 75000.0      | 2          | 37500.0  |

5) aggregate\_by\_registration\_year(joined\_df: DataFrame) → DataFrame

## PURPOSE

Summarize billing metrics grouped by patient registration year.

## INPUTS

joined\_df: DataFrame from join\_and\_clean

## RETURNS

A DataFrame containing:

- registration\_year, total\_amount, average\_amount

## LOGIC

Group by registration\_year.

Compute total billing as total\_amount.

Compute average billing as average\_amount.

Order results by registration\_year in ascending order.

## SAMPLE I/O

### SAMPLE INPUT

| patient_id | amount  | registration_year |
|------------|---------|-------------------|
| P001       | 5000.0  | 2018              |
| P001       | 3000.0  | 2018              |
| P002       | 15000.0 | 2020              |
| P006       | 12000.0 | 2017              |
| P006       | 6000.0  | 2017              |
| P010       | 25000.0 | 2015              |
| P010       | 50000.0 | 2015              |

## SAMPLE OUTPUT

| registration_year | total_amount | average_amount |
|-------------------|--------------|----------------|
| 2015              | 75000.0      | 37500.0        |
| 2017              | 18000.0      | 9000.0         |
| 2018              | 8000.0       | 4000.0         |
| 2020              | 15000.0      | 15000.0        |

6) `segment_patients_sql(patient_metrics_df: DataFrame, spark: SparkSession) → DataFrame`

## PURPOSE

Categorize patients into contribution segments based on total billing amount using SQL logic.

## INPUTS

`patient_metrics_df`: DataFrame from `compute_patient_metrics`

`spark`: SparkSession for SQL operations

## RETURNS

A DataFrame containing:

- `patient_id`, `total_billed`, `bill_count`, `avg_bill`, `segment`

## LOGIC

Register the DataFrame as a temporary SQL view named `patient_metrics`.

Apply the following CASE WHEN logic:

- Platinum: `total_billed >= 100000`
- Gold: `total_billed >= 25000 AND total_billed < 100000`
- Silver: `total_billed >= 5000 AND total_billed < 25000`
- Bronze: `total_billed < 5000`

## SAMPLE I/O

### SAMPLE INPUT

| patient_id | total_billed | bill_count | avg_bill |
|------------|--------------|------------|----------|
| P001       | 8000.0       | 2          | 4000.0   |
| P002       | 15000.0      | 1          | 15000.0  |
| P006       | 18000.0      | 2          | 9000.0   |
| P010       | 75000.0      | 2          | 37500.0  |

## SAMPLE OUTPUT

| patient_id | total_billed | bill_count | avg_bill | segment |
|------------|--------------|------------|----------|---------|
| P001       | 8000.0       | 2          | 4000.0   | Silver  |
| P002       | 15000.0      | 1          | 15000.0  | Silver  |
| P006       | 18000.0      | 2          | 9000.0   | Silver  |
| P010       | 75000.0      | 2          | 37500.0  | Gold    |

## Q9. E-Commerce Order Fulfillment Delay Analytics

Part 2 · PySpark — Medium — question 9 of 16

### PROBLEM STATEMENT

A nationwide e-commerce company manages customer orders through multiple regional fulfillment warehouses. Orders are processed and dispatched from different warehouses and delivered to customers across India.

Deliveries are handled by third-party logistics partners such as FastTrack, QuickShip, RapidGo, and ParcelPro — each with different service levels and operating regions.

Recently, the operations team has identified inconsistencies in order delivery timelines, where actual delivery dates exceeded the promised delivery dates. To ensure fulfillment accountability and logistics partner performance transparency, the operations team wants a Spark-based data pipeline that analyzes delivery delays, quantifies delay rates per warehouse-partner combination, and produces an overall delivery partner performance summary.

Your task is to design and implement a PySpark-based ETL and analytics workflow that:

- Reads and cleans order data.
- Loads delivery partner metadata.
- Computes delivery delay features.
- Joins order information with delivery partner metadata.
- Produces aggregated fulfillment performance reports for business stakeholders.
- Produces an overall partner-level performance summary using Spark SQL.

### OBJECTIVE

- Load and normalize order data, including all relevant date columns.
- Load delivery partner metadata and standardize column types.
- Compute delivery delay days and identify delayed orders.
- Enrich orders with delivery partner details such as service level and operating region.
- Aggregate and compute delay performance by warehouse and delivery partner.
- Produce a delivery partner-level summary using Spark SQL to calculate delay percentages, average delays, and total shipment volume.

### FILE STRUCTURE

```
Project
├── data
│   ├── orders.csv
│   └── partners.csv
├── main.py
├── requirements.txt
├── solution.py
└── tests.py
```

### MODULES

- main.py - contains the driver code/main invocation pipeline.
- solution.py - Must complete the functions.

### DATASET DESCRIPTION

Dataset 1 – orders.csv

| Column Name | Type   | Description             |
|-------------|--------|-------------------------|
| order_id    | string | Unique order identifier |
| customer_id | string | Customer identifier     |

| Column Name      | Type    | Description                              |
|------------------|---------|------------------------------------------|
| warehouse_id     | string  | Fulfillment warehouse identifier         |
| partner_id       | string  | Third-party delivery partner identifier  |
| order_date       | date    | Date when the order was placed           |
| promised_date    | date    | Date committed for customer delivery     |
| delivered_date   | date    | Actual date when the order was delivered |
| destination_city | string  | Destination city for delivery            |
| order_value      | integer | Total value of the order                 |
| priority         | string  | Priority level of order (Normal/High)    |

## SAMPLE DATA

| order_id | customer_id | warehouse_id | partner_id | order_date | promised_date | delivered_date | destination_city | order_value |
|----------|-------------|--------------|------------|------------|---------------|----------------|------------------|-------------|
| 0001     | C101        | W01          | P01        | 2025-08-01 | 2025-08-04    | 2025-08-04     | Mumbai           | 2500        |
| 0002     | C102        | W02          | P02        | 2025-08-02 | 2025-08-06    | 2025-08-08     | Delhi            | 4200        |
| 0003     | C103        | W01          | P03        | 2025-08-03 | 2025-08-07    | 2025-08-06     | Pune             | 1800        |
| 0004     | C104        | W03          | P01        | 2025-08-04 | 2025-08-09    | 2025-08-11     | Kolkata          | 3600        |
| 0005     | C105        | W02          | P02        | 2025-08-05 | 2025-08-08    | 2025-08-08     | Chennai          | 2200        |

## Dataset 2 – partners.csv

| Column Name      | Type   | Description                                            |
|------------------|--------|--------------------------------------------------------|
| partner_id       | string | Delivery partner identifier                            |
| partner_name     | string | Name of the logistics partner                          |
| service_level    | string | Level of service (Economy, Standard, Premium, Express) |
| operating_region | string | Primary operating region                               |

## SAMPLE DATA

| partner_id | partner_name | service_level | operating_region |
|------------|--------------|---------------|------------------|
| P01        | FastTrack    | Express       | West             |
| P02        | QuickShip    | Premium       | North            |
| P03        | RapidGo      | Standard      | South            |
| P04        | ParcelPro    | Economy       | East             |

## IMPLEMENTATION REQUIREMENTS

### FUNCTIONS TO IMPLEMENT IN SOLUTION.PY

- 1. load\_orders(spark: SparkSession, csv\_path: str) → DataFrame

### PURPOSE

Load order records from the CSV file and standardize their column types so that the data can be reliably used for downstream delivery-delay analysis.

## INPUTS

---

SparkSession

Path to orders.csv

## COLUMNS USED

---

- order\_id
- customer\_id
- warehouse\_id
- partner\_id
- order\_date
- promised\_date
- delivered\_date
- destination\_city
- order\_value
- priority

## RETURNS

---

A DataFrame with the following columns, in this order:

- order\_id
- customer\_id
- warehouse\_id
- partner\_id
- order\_date
- promised\_date
- delivered\_date
- destination\_city
- order\_value
- priority

The analytical types should be:

- order\_id, customer\_id, warehouse\_id, partner\_id, destination\_city, and priority as text.
- order\_date, promised\_date, and delivered\_date as dates.
- order\_value as an integer.

## LOGIC

---

Read the order dataset with its header row.

Return exactly the listed columns in the stated order.

Convert all three date fields into proper date columns.

Convert order\_value into an integer.

Ensure all textual columns remain string/text columns.

The resulting DataFrame must be suitable for downstream date arithmetic and numerical aggregation.

2. load\_partners(spark: SparkSession, csv\_path: str) → DataFrame

## PURPOSE

---

Load delivery partner reference metadata to support enrichment of order data.

## INPUTS

---

SparkSession

Path to partners.csv

## COLUMNS USED

---

- partner\_id
- partner\_name
- service\_level
- operating\_region

## RETURNS

---

A DataFrame containing:

- partner\_id
- partner\_name
- service\_level
- operating\_region
- All four columns must be text/string columns.

## LOGIC

---

Read the partner dataset with its header row.

Return exactly the four partner attributes.

Standardize all columns as text.

This DataFrame acts as a reference lookup that maps each partner\_id to its descriptive partner information for later use.

3. compute\_delivery\_features(orders\_df: DataFrame) → DataFrame

## PURPOSE

---

Derive per-order delivery-delay measurements.

## INPUTS

---

The cleaned order DataFrame returned by load\_orders.

## COLUMNS USED

---

- order\_id
- warehouse\_id
- partner\_id
- promised\_date
- delivered\_date

## RETURNS

---

A DataFrame with:

- order\_id
- warehouse\_id
- partner\_id
- delivery\_delay\_days
- delivery\_delayed\_flag

## WHERE

---

delivery\_delay\_days is an integer.

delivery\_delayed\_flag is an integer containing either 0 or 1.

## LOGIC

---

For each order:

- Calculate delivery\_delay\_days as the number of days between the actual delivery date and promised delivery date.
- delivery\_delay\_days =  
delivered\_date - promised\_date
- A positive value indicates a late delivery.
- A zero value indicates delivery on the promised date.
- A negative value indicates delivery before the promised date.

Derive delivery\_delayed\_flag:

- 1 when the order was delivered after its promised date.
- 0 for all other cases.
- Return only the order identifiers together with the two derived measures.
- 4. join\_orders\_with\_partners(orders\_df: DataFrame, partners\_df: DataFrame) → DataFrame

## PURPOSE

---

Enrich order delay information with delivery partner attributes for combined performance reporting.

## INPUTS

---

Orders DataFrame containing delivery-delay features.

Partner metadata DataFrame from load\_partners.

Join Key:

- partner\_id

## RETURNS

---

A DataFrame containing:

- order\_id
- warehouse\_id
- partner\_name
- service\_level
- operating\_region
- delivery\_delay\_days
- delivery\_delayed\_flag

## LOGIC

---

Join the order delay DataFrame with the partner metadata using partner\_id.

Use a LEFT JOIN so that every order record is preserved even if a matching partner reference is unavailable.

Enrich each order with:

- partner\_name
- service\_level
- operating\_region
- The raw partner\_id should not be retained in the final result.
- Return only the columns listed above.
- 5. aggregate\_fulfillment\_by\_warehouse\_and\_partner(enriched\_df: DataFrame) → DataFrame

---

## PURPOSE

Summarize delivery-delay performance for each warehouse and delivery-partner combination.

---

## INPUTS

The enriched order DataFrame from `join_orders_with_partners`.

---

## COLUMNS USED

- `warehouse_id`
- `partner_name`
- `delivery_delayed_flag`

---

## RETURNS

A DataFrame containing:

- `warehouse_id`
- `partner_name`
- `delayed_orders`
- `total_orders`
- `delay_percentage`

---

## WHERE

`delayed_orders` is a long/integer count.

`total_orders` is a long/integer count.

`delay_percentage` is a double rounded to two decimal places.

---

## LOGIC

For each distinct combination of `warehouse_id` and `partner_name`, calculate:

- `delayed_orders` — number of orders that were delivered late.
- `total_orders` — total number of orders handled.
- `delay_percentage` — percentage of orders that were delayed.

Formula:

- `delay_percentage` =
- $(\text{delayed\_orders} / \text{total\_orders}) \times 100$
- Round the result to two decimal places.
- 6. `partner_performance_sql(enriched_df: DataFrame, spark: SparkSession) → DataFrame`

---

## PURPOSE

Produce an overall delivery-partner performance summary using Spark SQL.

---

## INPUTS

Enriched order DataFrame from `join_orders_with_partners`.

`SparkSession`

---

## COLUMNS USED

- `partner_name`
- `delivery_delayed_flag`
- `delivery_delay_days`

---

## RETURNS

---

A DataFrame containing:

- partner\_name
- delay\_percentage
- average\_delay\_days
- order\_volume

## WHERE

delay\_percentage is a double.

average\_delay\_days is a double.

order\_volume is a long.

## LOGIC

Using Spark SQL, summarize performance for each partner\_name and report:

- delay\_percentage — percentage of the partner's orders that were delayed, expressed out of 100 and rounded to two decimals.
- average\_delay\_days — average delivery delay in days, rounded to two decimals.
- order\_volume — total number of orders handled by the partner.
- The result should be ordered with the highest delay percentage first. If two partners have the same delay percentage, the higher average delay should appear first.
- Sample Input / Output

### Function 1 — load\_orders(spark, csv\_path)

#### SAMPLE INPUT

order\_id,customer\_id,warehouse\_id,partner\_id,order\_date,promised\_date,delivered\_date,destination\_city,order\_value,priority

O001,C101,W01,P01,2025-08-01,2025-08-04,2025-08-04,Mumbai,2500,Normal

O002,C102,W02,P02,2025-08-02,2025-08-06,2025-08-08,Delhi,4200,High

O003,C103,W01,P03,2025-08-03,2025-08-07,2025-08-06,Pune,1800,Normal

O004,C104,W03,P01,2025-08-04,2025-08-09,2025-08-11,Kolkata,3600,High

O005,C105,W02,P02,2025-08-05,2025-08-08,2025-08-08,Chennai,2200,Normal

#### SAMPLE OUTPUT

| order_id | customer_id | warehouse_id | partner_id | order_date | promised_date | delivered_date | destination_city | order_value |
|----------|-------------|--------------|------------|------------|---------------|----------------|------------------|-------------|
| O001     | C101        | W01          | P01        | 2025-08-01 | 2025-08-04    | 2025-08-04     | Mumbai           | 2500        |
| O002     | C102        | W02          | P02        | 2025-08-02 | 2025-08-06    | 2025-08-08     | Delhi            | 4200        |
| O003     | C103        | W01          | P03        | 2025-08-03 | 2025-08-07    | 2025-08-06     | Pune             | 1800        |
| O004     | C104        | W03          | P01        | 2025-08-04 | 2025-08-09    | 2025-08-11     | Kolkata          | 3600        |
| O005     | C105        | W02          | P02        | 2025-08-05 | 2025-08-08    | 2025-08-08     | Chennai          | 2200        |

### Function 2 — load\_partners(spark, csv\_path)

#### SAMPLE INPUT

| partner_id | partner_name | service_level | operating_region |
|------------|--------------|---------------|------------------|
| P01        | FastTrack    | Express       | West             |
| P02        | QuickShip    | Premium       | North            |
| P03        | RapidGo      | Standard      | South            |
| P04        | ParcelPro    | Economy       | East             |

### SAMPLE OUTPUT

| partner_id | partner_name | service_level | operating_region |
|------------|--------------|---------------|------------------|
| P01        | FastTrack    | Express       | West             |
| P02        | QuickShip    | Premium       | North            |
| P03        | RapidGo      | Standard      | South            |
| P04        | ParcelPro    | Economy       | East             |

### Function 3 — compute\_delivery\_features(orders\_df)

#### SAMPLE INPUT

| order_id | warehouse_id | partner_id | promised_date | delivered_date |
|----------|--------------|------------|---------------|----------------|
| 0001     | W01          | P01        | 2025-08-04    | 2025-08-04     |
| 0002     | W02          | P02        | 2025-08-06    | 2025-08-08     |
| 0003     | W01          | P03        | 2025-08-07    | 2025-08-06     |
| 0004     | W03          | P01        | 2025-08-09    | 2025-08-11     |
| 0005     | W02          | P02        | 2025-08-08    | 2025-08-08     |

#### SAMPLE OUTPUT

| order_id | warehouse_id | partner_id | delivery_delay_days | delivery_delayed_flag |
|----------|--------------|------------|---------------------|-----------------------|
| 0001     | W01          | P01        | 0                   | 0                     |
| 0002     | W02          | P02        | 2                   | 1                     |
| 0003     | W01          | P03        | -1                  | 0                     |
| 0004     | W03          | P01        | 2                   | 1                     |
| 0005     | W02          | P02        | 0                   | 0                     |

### Function 4 — join\_orders\_with\_partners(orders\_df, partners\_df)

#### SAMPLE INPUT

Orders Delay Data

| order_id | warehouse_id | partner_id | delivery_delay_days | delivery_delayed_flag |
|----------|--------------|------------|---------------------|-----------------------|
| 0001     | W01          | P01        | 0                   | 0                     |
| 0002     | W02          | P02        | 2                   | 1                     |
| 0003     | W01          | P03        | -1                  | 0                     |
| 0004     | W03          | P01        | 2                   | 1                     |
| 0005     | W02          | P02        | 0                   | 0                     |

Partner Data

| partner_id | partner_name | service_level | operating_region |
|------------|--------------|---------------|------------------|
| P01        | FastTrack    | Express       | West             |
| P02        | QuickShip    | Premium       | North            |
| P03        | RapidGo      | Standard      | South            |
| P04        | ParcelPro    | Economy       | East             |

## SAMPLE OUTPUT

| order_id | warehouse_id | partner_name | service_level | operating_region | delivery_delay_days | delivery_delayed_flag |
|----------|--------------|--------------|---------------|------------------|---------------------|-----------------------|
| 0001     | W01          | FastTrack    | Express       | West             | 0                   | 0                     |
| 0002     | W02          | QuickShip    | Premium       | North            | 2                   | 1                     |
| 0003     | W01          | RapidGo      | Standard      | South            | -1                  | 0                     |
| 0004     | W03          | FastTrack    | Express       | West             | 2                   | 1                     |
| 0005     | W02          | QuickShip    | Premium       | North            | 0                   | 0                     |

## Function 5 — aggregate\_fulfillment\_by\_warehouse\_and\_partner(enriched\_df)

### SAMPLE INPUT

| order_id | warehouse_id | partner_name | service_level | operating_region | delivery_delay_days | delivery_delayed_flag |
|----------|--------------|--------------|---------------|------------------|---------------------|-----------------------|
| 0001     | W01          | FastTrack    | Express       | West             | 0                   | 0                     |
| 0002     | W02          | QuickShip    | Premium       | North            | 2                   | 1                     |
| 0003     | W01          | RapidGo      | Standard      | South            | -1                  | 0                     |
| 0004     | W03          | FastTrack    | Express       | West             | 2                   | 1                     |
| 0005     | W02          | QuickShip    | Premium       | North            | 0                   | 0                     |

### SAMPLE OUTPUT

| warehouse_id | partner_name | delayed_orders | total_orders | delay_percentage |
|--------------|--------------|----------------|--------------|------------------|
| W01          | FastTrack    | 0              | 1            | 0.00             |
| W01          | RapidGo      | 0              | 1            | 0.00             |
| W02          | QuickShip    | 1              | 2            | 50.00            |
| W03          | FastTrack    | 1              | 1            | 100.00           |

## Function 6 — partner\_performance\_sql(enriched\_df, spark)

### SAMPLE INPUT

| order_id | partner_name | delivery_delay_days | delivery_delayed_flag |
|----------|--------------|---------------------|-----------------------|
| 0001     | FastTrack    | 0                   | 0                     |
| 0002     | QuickShip    | 2                   | 1                     |
| 0003     | RapidGo      | -1                  | 0                     |
| 0004     | FastTrack    | 2                   | 1                     |
| 0005     | QuickShip    | 0                   | 0                     |

### SAMPLE OUTPUT

| partner_name | delay_percentage | average_delay_days | order_volume |
|--------------|------------------|--------------------|--------------|
| FastTrack    | 50.00            | 1.00               | 2            |

| partner_name | delay_percentage | average_delay_days | order_volume |
|--------------|------------------|--------------------|--------------|
| QuickShip    | 50.00            | 1.00               | 2            |
| RapidGo      | 0.00             | -1.00              | 1            |

## Q10. Airline Baggage Handling Delay Analytics

Part 2 · PySpark — Medium — question 10 of 16

### PROBLEM STATEMENT

A nationwide airline operates flights between multiple airports across India. Baggage handling is performed by third-party baggage service providers at different airports, with each provider operating under different service types and regional responsibilities.

Recently, the airline's operations team has reported inconsistencies in baggage availability timelines, where passengers sometimes receive their baggage later than the scheduled baggage availability time. To improve baggage handling transparency and service quality, the airline wants a Spark-based data pipeline that analyzes baggage delays, identifies delayed baggage, classifies the severity of delays, and produces performance reports for baggage service providers.

Your task is to design and implement a PySpark-based ETL and analytics workflow that:

- Reads and cleans baggage handling data.
- Loads baggage service provider metadata.
- Computes baggage delay features.
- Classifies baggage delays based on the number of delayed days.
- Joins baggage records with provider metadata.
- Produces aggregated baggage handling performance reports.
- Produces an overall provider-level performance summary using Spark SQL.

### OBJECTIVE

- Load and normalize baggage data, including all relevant date columns.
- Load baggage provider metadata and standardize column types.
- Compute baggage delay days and identify delayed baggage.
- Classify delays into On Time, Minor, Moderate, and Severe categories.
- Enrich baggage records with provider details such as service type and region.
- Aggregate and compute delay performance by airport and baggage provider.
- Produce a provider-level summary using Spark SQL to calculate delay percentages, average delays, and total baggage volume.

### FILE STRUCTURE

```
Project
├── data
│   ├── baggage.csv
│   └── baggage_providers.csv
├── main.py
├── requirements.txt
├── solution.py
└── tests.py
```

### MODULES

- main.py - contains the driver code/main invocation pipeline.
- solution.py - Must complete the functions.

### DATASET DESCRIPTION

Dataset 1 – baggage.csv

| Column Name | Type   | Description               |
|-------------|--------|---------------------------|
| bag_id      | string | Unique baggage identifier |
| flight_id   | string | Flight identifier         |

| Column Name    | Type    | Description                                    |
|----------------|---------|------------------------------------------------|
| airport_code   | string  | Arrival airport code                           |
| provider_id    | string  | Baggage service provider identifier            |
| scheduled_time | date    | Scheduled baggage availability date            |
| actual_time    | date    | Actual baggage availability date               |
| passenger_type | string  | Passenger class (Economy/Business)             |
| bag_weight     | integer | Baggage weight in kilograms                    |
| priority_bag   | string  | Whether baggage has priority handling (Yes/No) |

## SAMPLE DATA

| bag_id | flight_id | airport_code | provider_id | scheduled_time | actual_time | passenger_type | bag_weight | priority_bag |
|--------|-----------|--------------|-------------|----------------|-------------|----------------|------------|--------------|
| B001   | F101      | DEL          | BP01        | 2025-09-01     | 2025-09-01  | Economy        | 18         | No           |
| B002   | F102      | BOM          | BP02        | 2025-09-02     | 2025-09-04  | Business       | 25         | Yes          |
| B003   | F103      | BLR          | BP01        | 2025-09-03     | 2025-09-04  | Economy        | 20         | No           |
| B004   | F104      | DEL          | BP03        | 2025-09-04     | 2025-09-04  | Business       | 28         | Yes          |
| B005   | F105      | BOM          | BP02        | 2025-09-05     | 2025-09-07  | Economy        | 16         | No           |
| B006   | F106      | BOM          | BP02        | 2025-09-06     | 2025-09-06  | Business       | 30         | Yes          |
| B007   | F107      | BLR          | BP01        | 2025-09-07     | 2025-09-10  | Economy        | 22         | No           |
| B008   | F108      | DEL          | BP03        | 2025-09-08     | 2025-09-09  | Business       | 27         | Yes          |

## Dataset 2 – baggage\_providers.csv

| Column Name   | Type   | Description                      |
|---------------|--------|----------------------------------|
| provider_id   | string | Baggage provider identifier      |
| provider_name | string | Name of baggage service provider |
| service_type  | string | Type of baggage service          |
| region        | string | Primary operating region         |

## SAMPLE DATA

| provider_id | provider_name | service_type | region |
|-------------|---------------|--------------|--------|
| BP01        | BagAssist     | Standard     | South  |
| BP02        | AirSupport    | Premium      | West   |
| BP03        | SkyBaggage    | Standard     | North  |
| BP04        | FlyCare       | Premium      | East   |

## IMPLEMENTATION REQUIREMENTS

### FUNCTIONS TO IMPLEMENT IN SOLUTION.PY

- 1. load\_baggage(spark: SparkSession, csv\_path: str) → DataFrame

### PURPOSE

Load baggage records from the CSV file and standardize their column types for reliable downstream delay analysis.

### INPUTS

SparkSession

Path to baggage.csv

---

## COLUMNS USED

- bag\_id
- flight\_id
- airport\_code
- provider\_id
- scheduled\_time
- actual\_time
- passenger\_type
- bag\_weight
- priority\_bag

---

## RETURNS

A DataFrame with the following columns, in this order:

- bag\_id
- flight\_id
- airport\_code
- provider\_id
- scheduled\_time
- actual\_time
- passenger\_type
- bag\_weight
- priority\_bag

The analytical types should be:

- bag\_id, flight\_id, airport\_code, provider\_id, passenger\_type, and priority\_bag as text.
- scheduled\_time and actual\_time as dates.
- bag\_weight as an integer.

---

## LOGIC

Read the baggage dataset with its header row.

Return exactly the listed columns in the stated order.

Convert the scheduled and actual availability fields into proper date columns.

Convert bag\_weight into an integer.

Ensure all textual columns remain string/text columns.

The resulting DataFrame must support downstream date arithmetic and numerical analysis.

2. load\_baggage\_providers(spark: SparkSession, csv\_path: str) → DataFrame

---

## PURPOSE

Load baggage service provider reference metadata to support enrichment of baggage records.

---

## INPUTS

SparkSession

Path to baggage\_providers.csv

---

## COLUMNS USED

- provider\_id
- provider\_name
- service\_type
- region

## RETURNS

---

A DataFrame containing:

- provider\_id
- provider\_name
- service\_type
- region
- All columns must be text/string columns.

## LOGIC

---

Read the baggage provider dataset with its header row.

Return exactly the four provider attributes.

Standardize all columns as text.

This DataFrame acts as a reference lookup that maps each provider\_id to its descriptive service information.

3. compute\_baggage\_delay(baggage\_df: DataFrame) → DataFrame

## PURPOSE

---

Derive per-baggage delay measurements.

## INPUTS

---

The cleaned baggage DataFrame returned by load\_baggage.

## COLUMNS USED

---

- bag\_id
- airport\_code
- provider\_id
- scheduled\_time
- actual\_time

## RETURNS

---

A DataFrame with:

- bag\_id
- airport\_code
- provider\_id
- delay\_days
- delayed\_flag

## WHERE

---

delay\_days is an integer.

delayed\_flag is an integer containing either 0 or 1.

## LOGIC

---

For each baggage record:

- Calculate `delay_days` as the number of days between the actual baggage availability date and scheduled baggage availability date.
- `delay_days =`
- `actual_time - scheduled_time`
- A positive value indicates delayed baggage.
- A zero value indicates baggage became available on schedule.
- A negative value indicates baggage became available before the scheduled time.

Derive `delayed_flag`:

- 1 when actual availability was after scheduled availability.
- 0 for all other cases.
- Return only the baggage identifiers together with these two derived measures.
- 4. `classify_delay(delay_df: DataFrame) → DataFrame`

## PURPOSE

Classify each baggage record according to the severity of its delivery delay.

## INPUTS

The DataFrame generated by `compute_baggage_delay`.

## COLUMNS USED

- `bag_id`
- `airport_code`
- `provider_id`
- `delay_days`
- `delayed_flag`

## RETURNS

The original columns plus:

- `delay_category`

The resulting DataFrame should contain:

- `bag_id`
- `airport_code`
- `provider_id`
- `delay_days`
- `delayed_flag`
- `delay_category`

## LOGIC

Classify the baggage records using the following rules:

- `delay_days <= 0`
- `→ On Time`

```
delay_days = 1
```

`→ Minor`

`delay_days` between 2 and 3

`→ Moderate`

`delay_days > 3`

`→ Severe`

For example:

| delay_days | delay_category |
|------------|----------------|
| 0          | On Time        |
| 1          | Minor          |
| 2          | Moderate       |
| 3          | Moderate       |
| 4          | Severe         |

The classification should be implemented using Spark DataFrame transformations.

5. `join_baggage_with_providers(baggage_df: DataFrame, providers_df: DataFrame) → DataFrame`

## PURPOSE

Enrich baggage delay information with baggage service provider attributes for combined performance reporting.

## INPUTS

Baggage DataFrame containing delay features and delay classification.

Provider metadata DataFrame from `load_baggage_providers`.

Join Key:

- `provider_id`

## RETURNS

A DataFrame containing:

- `bag_id`
- `airport_code`
- `provider_name`
- `service_type`
- `region`
- `delay_days`
- `delayed_flag`
- `delay_category`

## LOGIC

Join the baggage delay DataFrame with provider metadata using `provider_id`.

Use a LEFT JOIN so that every baggage record is preserved even when no matching provider reference is available.

Enrich each baggage record with:

- `provider_name`
- `service_type`
- `region`
- The raw `provider_id` should not be retained in the final result.
- Return only the columns listed in Returns.
- 6. `provider_performance_sql(enriched_df: DataFrame, spark: SparkSession) → DataFrame`

## PURPOSE

Produce an overall baggage service provider performance summary using Spark SQL.

## INPUTS

Enriched baggage DataFrame from `join_baggage_with_providers`.

SparkSession

## COLUMNS USED

- `provider_name`
- `delayed_flag`
- `delay_days`

## RETURNS

A DataFrame containing:

- `provider_name`
- `delay_percentage`
- `average_delay_days`
- `baggage_volume`

## WHERE

`delay_percentage` is a double.

`average_delay_days` is a double.

`baggage_volume` is a long.

## LOGIC

Using Spark SQL, summarize performance for each `provider_name` and report:

- `delay_percentage` — percentage of the provider's baggage records that were delayed, expressed out of 100 and rounded to two decimals.
- `average_delay_days` — the provider's mean baggage delay in days, rounded to two decimals.
- `baggage_volume` — total number of baggage records handled by the provider.
- The result should be ordered with the highest delay percentage first. If two providers have the same delay percentage, the higher average delay should appear first.
- Sample Input / Output

### Function 1 — `load_baggage(spark, csv_path)`

## SAMPLE INPUT

| bag_id | flight_id | airport_code | provider_id | scheduled_time | actual_time | passenger_type | bag_weight | priority_bag |
|--------|-----------|--------------|-------------|----------------|-------------|----------------|------------|--------------|
| B001   | F101      | DEL          | BP01        | 2025-09-01     | 2025-09-01  | Economy        | 18         | No           |
| B002   | F102      | BOM          | BP02        | 2025-09-02     | 2025-09-04  | Business       | 25         | Yes          |
| B003   | F103      | BLR          | BP01        | 2025-09-03     | 2025-09-04  | Economy        | 20         | No           |
| B004   | F104      | DEL          | BP03        | 2025-09-04     | 2025-09-04  | Business       | 28         | Yes          |
| B005   | F105      | BOM          | BP02        | 2025-09-05     | 2025-09-07  | Economy        | 16         | No           |
| B006   | F106      | BOM          | BP02        | 2025-09-06     | 2025-09-06  | Business       | 30         | Yes          |
| B007   | F107      | BLR          | BP01        | 2025-09-07     | 2025-09-10  | Economy        | 22         | No           |
| B008   | F108      | DEL          | BP03        | 2025-09-08     | 2025-09-09  | Business       | 27         | Yes          |

## SAMPLE OUTPUT

| bag_id | flight_id | airport_code | provider_id | scheduled_time | actual_time | passenger_type | bag_weight | priority_bag |
|--------|-----------|--------------|-------------|----------------|-------------|----------------|------------|--------------|
| B001   | F101      | DEL          | BP01        | 2025-09-01     | 2025-09-01  | Economy        | 18         | No           |

| bag_id | flight_id | airport_code | provider_id | scheduled_time | actual_time | passenger_type | bag_weight | priority_bag |
|--------|-----------|--------------|-------------|----------------|-------------|----------------|------------|--------------|
| B002   | F102      | BOM          | BP02        | 2025-09-02     | 2025-09-04  | Business       | 25         | Yes          |
| B003   | F103      | BLR          | BP01        | 2025-09-03     | 2025-09-04  | Economy        | 20         | No           |
| B004   | F104      | DEL          | BP03        | 2025-09-04     | 2025-09-04  | Business       | 28         | Yes          |
| B005   | F105      | BOM          | BP02        | 2025-09-05     | 2025-09-07  | Economy        | 16         | No           |
| B006   | F106      | BOM          | BP02        | 2025-09-06     | 2025-09-06  | Business       | 30         | Yes          |
| B007   | F107      | BLR          | BP01        | 2025-09-07     | 2025-09-10  | Economy        | 22         | No           |
| B008   | F108      | DEL          | BP03        | 2025-09-08     | 2025-09-09  | Business       | 27         | Yes          |

## Function 2 — load\_baggage\_providers(spark, csv\_path)

### SAMPLE INPUT

| provider_id | provider_name | service_type | region |
|-------------|---------------|--------------|--------|
| BP01        | BagAssist     | Standard     | South  |
| BP02        | AirSupport    | Premium      | West   |
| BP03        | SkyBaggage    | Standard     | North  |
| BP04        | FlyCare       | Premium      | East   |

### SAMPLE OUTPUT

| provider_id | provider_name | service_type | region |
|-------------|---------------|--------------|--------|
| BP01        | BagAssist     | Standard     | South  |
| BP02        | AirSupport    | Premium      | West   |
| BP03        | SkyBaggage    | Standard     | North  |
| BP04        | FlyCare       | Premium      | East   |

## Function 3 — compute\_baggage\_delay(baggage\_df)

### SAMPLE INPUT

| bag_id | airport_code | provider_id | scheduled_time | actual_time |
|--------|--------------|-------------|----------------|-------------|
| B001   | DEL          | BP01        | 2025-09-01     | 2025-09-01  |
| B002   | BOM          | BP02        | 2025-09-02     | 2025-09-04  |
| B003   | BLR          | BP01        | 2025-09-03     | 2025-09-04  |
| B004   | DEL          | BP03        | 2025-09-04     | 2025-09-04  |
| B005   | BOM          | BP02        | 2025-09-05     | 2025-09-07  |
| B006   | BOM          | BP02        | 2025-09-06     | 2025-09-06  |
| B007   | BLR          | BP01        | 2025-09-07     | 2025-09-10  |
| B008   | DEL          | BP03        | 2025-09-08     | 2025-09-09  |

### SAMPLE OUTPUT

| bag_id | airport_code | provider_id | delay_days | delayed_flag |
|--------|--------------|-------------|------------|--------------|
| B001   | DEL          | BP01        | 0          | 0            |
| B002   | BOM          | BP02        | 2          | 1            |
| B003   | BLR          | BP01        | 1          | 1            |
| B004   | DEL          | BP03        | 0          | 0            |
| B005   | BOM          | BP02        | 2          | 1            |

| bag_id | airport_code | provider_id | delay_days | delayed_flag |
|--------|--------------|-------------|------------|--------------|
| B006   | BOM          | BP02        | 0          | 0            |
| B007   | BLR          | BP01        | 3          | 1            |
| B008   | DEL          | BP03        | 1          | 1            |

#### Function 4 — classify\_delay(delay\_df)

##### SAMPLE INPUT

| bag_id | airport_code | provider_id | delay_days | delayed_flag |
|--------|--------------|-------------|------------|--------------|
| B001   | DEL          | BP01        | 0          | 0            |
| B002   | BOM          | BP02        | 2          | 1            |
| B003   | BLR          | BP01        | 1          | 1            |
| B004   | DEL          | BP03        | 0          | 0            |
| B005   | BOM          | BP02        | 2          | 1            |
| B006   | BOM          | BP02        | 0          | 0            |
| B007   | BLR          | BP01        | 3          | 1            |
| B008   | DEL          | BP03        | 1          | 1            |

##### SAMPLE OUTPUT

| bag_id | airport_code | provider_id | delay_days | delayed_flag | delay_category |
|--------|--------------|-------------|------------|--------------|----------------|
| B001   | DEL          | BP01        | 0          | 0            | On Time        |
| B002   | BOM          | BP02        | 2          | 1            | Moderate       |
| B003   | BLR          | BP01        | 1          | 1            | Minor          |
| B004   | DEL          | BP03        | 0          | 0            | On Time        |
| B005   | BOM          | BP02        | 2          | 1            | Moderate       |
| B006   | BOM          | BP02        | 0          | 0            | On Time        |
| B007   | BLR          | BP01        | 3          | 1            | Moderate       |
| B008   | DEL          | BP03        | 1          | 1            | Minor          |

#### Function 5 — join\_baggage\_with\_providers(baggage\_df, providers\_df)

##### SAMPLE INPUT

Baggage Delay Data

| bag_id | airport_code | provider_id | delay_days | delayed_flag | delay_category |
|--------|--------------|-------------|------------|--------------|----------------|
| B001   | DEL          | BP01        | 0          | 0            | On Time        |
| B002   | BOM          | BP02        | 2          | 1            | Moderate       |
| B003   | BLR          | BP01        | 1          | 1            | Minor          |
| B004   | DEL          | BP03        | 0          | 0            | On Time        |
| B005   | BOM          | BP02        | 2          | 1            | Moderate       |
| B006   | BOM          | BP02        | 0          | 0            | On Time        |
| B007   | BLR          | BP01        | 3          | 1            | Moderate       |
| B008   | DEL          | BP03        | 1          | 1            | Minor          |

Provider Data

| provider_id | provider_name | service_type | region |
|-------------|---------------|--------------|--------|
| BP01        | BagAssist     | Standard     | South  |
| BP02        | AirSupport    | Premium      | West   |
| BP03        | SkyBaggage    | Standard     | North  |
| BP04        | FlyCare       | Premium      | East   |

### SAMPLE OUTPUT

| bag_id | airport_code | provider_name | service_type | region | delay_days | delayed_flag | delay_category |
|--------|--------------|---------------|--------------|--------|------------|--------------|----------------|
| B001   | DEL          | BagAssist     | Standard     | South  | 0          | 0            | On Time        |
| B002   | BOM          | AirSupport    | Premium      | West   | 2          | 1            | Moderate       |
| B003   | BLR          | BagAssist     | Standard     | South  | 1          | 1            | Minor          |
| B004   | DEL          | SkyBaggage    | Standard     | North  | 0          | 0            | On Time        |
| B005   | BOM          | AirSupport    | Premium      | West   | 2          | 1            | Moderate       |
| B006   | BOM          | AirSupport    | Premium      | West   | 0          | 0            | On Time        |
| B007   | BLR          | BagAssist     | Standard     | South  | 3          | 1            | Moderate       |
| B008   | DEL          | SkyBaggage    | Standard     | North  | 1          | 1            | Minor          |

### Function 6 — provider\_performance\_sql(enriched\_df, spark)

#### SAMPLE INPUT

| bag_id | provider_name | delay_days | delayed_flag |
|--------|---------------|------------|--------------|
| B001   | BagAssist     | 0          | 0            |
| B002   | AirSupport    | 2          | 1            |
| B003   | BagAssist     | 1          | 1            |
| B004   | SkyBaggage    | 0          | 0            |
| B005   | AirSupport    | 2          | 1            |
| B006   | AirSupport    | 0          | 0            |
| B007   | BagAssist     | 3          | 1            |
| B008   | SkyBaggage    | 1          | 1            |

#### SAMPLE OUTPUT

| provider_name | delay_percentage | average_delay_days | baggage_volume |
|---------------|------------------|--------------------|----------------|
| BagAssist     | 66.67            | 1.33               | 3              |
| AirSupport    | 66.67            | 1.33               | 3              |
| SkyBaggage    | 50.00            | 0.50               | 2              |

## Q11. Warehouse Inventory KPI Analysis

Part 2 · PySpark — Medium — question 11 of 16

### PROBLEM STATEMENT

A nationwide retail distributor operates multiple warehouses that maintain inventory for different product categories. Each inventory transaction captures the warehouse, product, category, unit cost, quantity received, and the date on which the inventory was recorded. Alongside the inventory transaction log, the operations team maintains a product metadata table that defines a target inventory value for every product category.

The inventory analytics team requires an automated PySpark pipeline that ingests both datasets, cleans and enriches the inventory records, calculates the inventory value for each valid transaction, classifies each transaction against its category target, produces a per-category inventory KPI summary, and uses Spark SQL with window ranking to identify the highest-value inventory transaction per category while retaining ties.

### OBJECTIVE

- Load inventory transaction and product category metadata datasets using explicitly defined schemas with appropriate data types for date, double, integer, and string columns.
- Clean inventory records by removing rows with missing transaction identifiers, non-positive quantity, or non-positive unit cost; then enrich the surviving rows via an inner join with category metadata.
- Compute inventory value as unit cost multiplied by quantity, and classify each transaction against its category target as Met or Below.
- Produce per-category KPIs covering total transactions, met transactions, achievement rate, and average inventory value.
- Use a Spark SQL temporary view with a DENSE\_RANK window function to identify the highest inventory-value transaction(s) per category, retaining ties.

### FILE STRUCTURE

```
Project
+--- data
|   +--- inventory.csv
|   +--- category_meta.csv
+--- main.py
+--- solution.py
+--- tests.py
```

### PURPOSE OF FILES

- main.py — Driver script that loads data, calls solution functions, and displays intermediate DataFrame outputs.
- solution.py — Contains the six functions to be implemented by the candidate.
- data/inventory.csv — Raw warehouse inventory transaction dataset used as input for the pipeline.
- data/category\_meta.csv — Product category metadata dataset containing per-category inventory value targets.
- tests.py — Contains automated test cases used to validate the implementation.

### DATASET DESCRIPTION

File: inventory.csv

### SCHEMA

| Column Name    | Data Type | Description                                            |
|----------------|-----------|--------------------------------------------------------|
| transaction_id | string    | Unique identifier for each inventory transaction       |
| warehouse_name | string    | Name of the warehouse where the inventory was recorded |
| category       | string    | Product category for the inventory                     |

| Column Name    | Data Type | Description                                           |
|----------------|-----------|-------------------------------------------------------|
| unit_cost      | double    | Cost of one unit of the product                       |
| quantity       | integer   | Number of units recorded in the inventory transaction |
| inventory_date | date      | Date when the inventory transaction was recorded      |

Sample Data – inventory.csv

| transaction_id | warehouse_name | category       | unit_cost | quantity | inventory_date |
|----------------|----------------|----------------|-----------|----------|----------------|
| T1001          | Mumbai Central | Electronics    | 12000.00  | 2        | 2025-09-02     |
| T1002          | Delhi North    | Electronics    | 4500.00   | 1        | 2025-09-03     |
| T1003          | Bangalore East | Electronics    | 2500.00   | 3        | 2025-09-04     |
| T1004          | Mumbai Central | Apparel        | 1200.00   | 4        | 2025-09-08     |
| T1005          | Delhi North    | Apparel        | 2500.00   | 2        | 2025-09-10     |
| T1006          | Pune West      | Home & Kitchen | 1800.00   | 2        | 2025-09-12     |
| T1007          | Mumbai Central | Books          | 650.00    | 3        | 2025-09-14     |
| T1008          | Delhi North    | Sports         | -500.00   | 2        | 2025-09-15     |
| T1009          | Pune West      | Apparel        | 900.00    | 0        | 2025-09-16     |

File: category\_meta.csv

**SCHEMA**

| Column Name            | Data Type | Description                             |
|------------------------|-----------|-----------------------------------------|
| category               | string    | Product category name                   |
| target_inventory_value | integer   | Target inventory value for the category |

Sample Data – category\_meta.csv

| category       | target_inventory_value |
|----------------|------------------------|
| Electronics    | 10000                  |
| Apparel        | 5000                   |
| Home & Kitchen | 4000                   |
| Books          | 1500                   |
| Sports         | 2000                   |

**TASK****FUNCTIONS TO IMPLEMENT IN SOLUTION.PY****Function 1 — load\_inventory(spark, path)****PURPOSE**

Load the inventory transaction dataset from a CSV file with an explicitly defined schema.

**PARAMETERS**

**spark** (SparkSession) – Active Spark session passed in from main.py.

**path** (string) — File path to the inventory CSV file passed in from main.py.

**LOGIC**

Define a schema with:

- transaction\_id as string
- warehouse\_name as string
- category as string
- unit\_cost as double
- quantity as integer
- inventory\_date as date
- Read the CSV file with the header enabled and the schema applied so that numeric and date values are parsed using the required data types rather than being treated as strings.

## RETURNS

DataFrame with the six columns matching the defined schema.

### Function 2 — load\_category\_metadata(spark, path)

## PURPOSE

Load the category metadata dataset from a CSV file with an explicitly defined schema.

## PARAMETERS

spark (SparkSession) — Active Spark session passed in from main.py.

path (string) — File path to the category metadata CSV file passed in from main.py.

## LOGIC

Define a schema with:

- category as string
- target\_inventory\_value as integer
- Read the CSV file with the header enabled and the schema applied.

## RETURNS

DataFrame with two columns matching the defined schema.

### Function 3 — clean\_and\_join\_inventory(df\_inventory, df\_meta)

## PURPOSE

Remove invalid inventory records and enrich the surviving rows with the target inventory value for their category.

## PARAMETERS

df\_inventory (DataFrame) — Raw inventory DataFrame produced by load\_inventory.

df\_meta (DataFrame) — Category metadata DataFrame produced by load\_category\_metadata.

## LOGIC

Filter out rows where:

- transaction\_id is null.
- quantity is zero or less.
- unit\_cost is zero or less.
- Perform an inner join with the metadata DataFrame on the category column.
- Project the seven enriched columns.

## RETURNS

DataFrame with columns:

- transaction\_id
- warehouse\_name
- category
- unit\_cost
- quantity
- inventory\_date
- target\_inventory\_value

## Function 4 — compute\_inventory\_value\_and\_status(df\_joined)

### PURPOSE

Derive inventory value and classify each inventory transaction against its category target.

### PARAMETERS

df\_joined (DataFrame) — Cleaned and joined DataFrame produced by clean\_and\_join\_inventory.

### LOGIC

Add an inventory\_value column computed as:

- unit\_cost × quantity

Add an inventory\_status column equal to:

- Met

when:

- inventory\_value >= target\_inventory\_value

Otherwise:

- Below
- The boundary case where inventory value exactly equals target\_inventory\_value is treated as Met.
- Project the final six columns.

### RETURNS

DataFrame with columns:

- transaction\_id
- warehouse\_name
- category
- inventory\_value
- target\_inventory\_value
- inventory\_status

## Function 5 — compute\_category\_kpis(df\_enriched)

### PURPOSE

Aggregate per-category inventory performance KPIs from the enriched DataFrame.

### PARAMETERS

df\_enriched (DataFrame) — Enriched DataFrame produced by compute\_inventory\_value\_and\_status, containing inventory value and inventory status per transaction.

### LOGIC

Group records by category and compute:

- `total_transactions` as the count of inventory transactions.
- `met_transactions` as the count of transactions whose `inventory_status` is Met.
- `achievement_rate` as `met_transactions` divided by `total_transactions`, rounded to two decimal places.
- `avg_inventory_value` as the average inventory value rounded to two decimal places.

## NOTE

Spark stores these values as doubles rounded to two decimals. On console display, trailing zeros may be trimmed.

## RETURNS

DataFrame with columns:

- `category`
- `total_transactions`
- `met_transactions`
- `achievement_rate`
- `avg_inventory_value`

## Function 6 — `top_inventory_per_category(spark, df_enriched)`

### PURPOSE

Identify the highest-value inventory transaction(s) per category using a Spark SQL temporary view and a window function.

### PARAMETERS

`spark` (SparkSession) — Active Spark session used to execute the SQL query.

`df_enriched` (DataFrame) — Enriched DataFrame produced by `compute_inventory_value_and_status`, containing inventory value per transaction.

### LOGIC

Register the enriched DataFrame as a temporary SQL view.

Run a Spark SQL query that applies:

### `DENSE_RANK()`

partitioned by category and ordered by `inventory_value` descending.

Filter the results to:

```
rank = 1
```

Ties are intentionally retained so multiple highest-value inventory transactions for the same category appear together.

Order the final output by category and `warehouse_name`.

## RETURNS

DataFrame with columns:

- `category`
- `transaction_id`
- `warehouse_name`
- `inventory_value`
- SAMPLE INPUT/OUTPUT

## Function 1 — `load_inventory(spark, path)`

## SAMPLE OUTPUT

| transaction_id | warehouse_name | category    | unit_cost | quantity | inventory_date |
|----------------|----------------|-------------|-----------|----------|----------------|
| T1001          | Mumbai Central | Electronics | 12000.0   | 2        | 2025-09-02     |
| T1002          | Delhi North    | Electronics | 4500.0    | 1        | 2025-09-03     |
| T1003          | Bangalore East | Electronics | 2500.0    | 3        | 2025-09-04     |
| T1004          | Mumbai Central | Apparel     | 1200.0    | 4        | 2025-09-08     |
| T1005          | Delhi North    | Apparel     | 2500.0    | 2        | 2025-09-10     |

## Function 2 — load\_category\_metadata(spark, path)

## SAMPLE OUTPUT

| category       | target_inventory_value |
|----------------|------------------------|
| Electronics    | 10000                  |
| Apparel        | 5000                   |
| Home & Kitchen | 4000                   |
| Books          | 1500                   |
| Sports         | 2000                   |

## Function 3 — clean\_and\_join\_inventory(df\_inventory, df\_meta)

## SAMPLE INPUT

| transaction_id | warehouse_name | category    | unit_cost | quantity | inventory_date |
|----------------|----------------|-------------|-----------|----------|----------------|
| T1001          | Mumbai Central | Electronics | 12000.00  | 2        | 2025-09-02     |
| null           | Delhi North    | Electronics | 4500.00   | 1        | 2025-09-03     |
| T1009          | Pune West      | Apparel     | 900.00    | 0        | 2025-09-16     |
| T1008          | Delhi North    | Sports      | -500.00   | 2        | 2025-09-15     |
| T1004          | Mumbai Central | Apparel     | 1200.00   | 4        | 2025-09-08     |
| T1005          | Delhi North    | Apparel     | 2500.00   | 2        | 2025-09-10     |

## SAMPLE OUTPUT

| transaction_id | warehouse_name | category    | unit_cost | quantity | inventory_date | target_inventory_value |
|----------------|----------------|-------------|-----------|----------|----------------|------------------------|
| T1001          | Mumbai Central | Electronics | 12000.00  | 2        | 2025-09-02     | 10000                  |
| T1004          | Mumbai Central | Apparel     | 1200.00   | 4        | 2025-09-08     | 5000                   |
| T1005          | Delhi North    | Apparel     | 2500.00   | 2        | 2025-09-10     | 5000                   |

## Function 4 — compute\_inventory\_value\_and\_status(df\_joined)

## SAMPLE INPUT

| transaction_id | warehouse_name | category    | unit_cost | quantity | target_inventory_value |
|----------------|----------------|-------------|-----------|----------|------------------------|
| T1001          | Mumbai Central | Electronics | 12000.00  | 2        | 10000                  |
| T1004          | Mumbai Central | Apparel     | 1200.00   | 4        | 5000                   |
| T1005          | Delhi North    | Apparel     | 2500.00   | 2        | 5000                   |
| T1007          | Mumbai Central | Books       | 650.00    | 3        | 1500                   |

## SAMPLE OUTPUT

| transaction_id | warehouse_name | category    | inventory_value | target_inventory_value | inventory_status |
|----------------|----------------|-------------|-----------------|------------------------|------------------|
| T1001          | Mumbai Central | Electronics | 24000.0         | 10000                  | Met              |
| T1004          | Mumbai Central | Apparel     | 4800.0          | 5000                   | Below            |
| T1005          | Delhi North    | Apparel     | 5000.0          | 5000                   | Met              |
| T1007          | Mumbai Central | Books       | 1950.0          | 1500                   | Met              |

### Function 5 — compute\_category\_kpis(df\_enriched)

#### SAMPLE INPUT

| transaction_id | category    | inventory_value | target_inventory_value | inventory_status |
|----------------|-------------|-----------------|------------------------|------------------|
| T1001          | Electronics | 24000.0         | 10000                  | Met              |
| T1002          | Electronics | 4500.0          | 10000                  | Below            |
| T1003          | Electronics | 7500.0          | 10000                  | Below            |
| T1004          | Apparel     | 4800.0          | 5000                   | Below            |
| T1005          | Apparel     | 5000.0          | 5000                   | Met              |
| T1007          | Books       | 1950.0          | 1500                   | Met              |
| T1008          | Books       | 1100.0          | 1500                   | Below            |

#### SAMPLE OUTPUT

| category    | total_transactions | met_transactions | achievement_rate | avg_inventory_value |
|-------------|--------------------|------------------|------------------|---------------------|
| Apparel     | 2                  | 1                | 0.50             | 4900.0              |
| Books       | 2                  | 1                | 0.50             | 1525.0              |
| Electronics | 3                  | 1                | 0.33             | 12000.0             |

### Function 6 — top\_inventory\_per\_category(spark, df\_enriched)

#### SAMPLE INPUT

| transaction_id | warehouse_name | category    | inventory_value |
|----------------|----------------|-------------|-----------------|
| T1001          | Mumbai Central | Electronics | 24000.0         |
| T1002          | Delhi North    | Electronics | 4500.0          |
| T1003          | Bangalore East | Electronics | 7500.0          |
| T1004          | Mumbai Central | Apparel     | 4800.0          |
| T1005          | Delhi North    | Apparel     | 5000.0          |
| T1010          | Pune West      | Books       | 1950.0          |
| T1007          | Mumbai Central | Books       | 1950.0          |
| T1008          | Delhi North    | Books       | 1100.0          |

#### SAMPLE OUTPUT

| category    | transaction_id | warehouse_name | inventory_value |
|-------------|----------------|----------------|-----------------|
| Apparel     | T1005          | Delhi North    | 5000.0          |
| Books       | T1007          | Mumbai Central | 1950.0          |
| Books       | T1010          | Pune West      | 1950.0          |
| Electronics | T1001          | Mumbai Central | 24000.0         |

Here, both Books transactions are retained because they have the same highest inventory value.



## Q12. Hotel Booking Revenue KPI Analysis

Part 2 · PySpark — Medium — question 12 of 16

### PROBLEM STATEMENT

A hospitality company operates multiple hotels across major cities and receives room bookings from customers throughout the year. Each booking captures the guest name, room category, nightly room rate, number of nights booked, and the booking date. Alongside the booking records, the revenue management team maintains a room category metadata table that defines a minimum expected revenue target for every room category.

The hotel analytics team requires an automated PySpark pipeline that ingests both datasets, cleans and enriches the booking records, calculates booking revenue, classifies each booking against its room-category target, produces a per-category revenue KPI summary, and uses Spark SQL with window ranking to identify the highest-revenue booking per room category while retaining ties.

### OBJECTIVE

- Load booking and room category metadata datasets using explicitly defined schemas with appropriate data types for date, double, integer, and string columns.
- Clean booking records by removing rows with missing booking identifiers, non-positive number of nights, or non-positive nightly room rates; then enrich the surviving rows via an inner join with room category metadata.
- Compute booking revenue as nightly room rate multiplied by number of nights, and classify each booking against its category target as Met or Below.
- Produce per-category KPIs covering total bookings, met bookings, achievement rate, and average booking revenue.
- Use a Spark SQL temporary view with a DENSE\_RANK window function to identify the highest-revenue booking(s) per room category, retaining ties.

### FILE STRUCTURE

```
Project
+--- data
|   +--- bookings.csv
|   +--- room_category_meta.csv
+--- main.py
+--- solution.py
+--- tests.py
```

### PURPOSE OF FILES

- main.py — Driver script that loads data, calls solution functions, and displays intermediate DataFrame outputs.
- solution.py — Contains the six functions to be implemented by the candidate.
- data/bookings.csv — Raw hotel booking records dataset used as input for the pipeline.
- data/room\_category\_meta.csv — Room category metadata dataset containing per-category revenue targets.
- tests.py — Contains automated test cases used to validate the implementation.

### DATASET DESCRIPTION

File: bookings.csv

### SCHEMA

| Column Name | Data Type | Description                              |
|-------------|-----------|------------------------------------------|
| booking_id  | string    | Unique identifier for each hotel booking |
| guest_name  | string    | Name of the guest making the booking     |

| Column Name   | Data Type | Description                       |
|---------------|-----------|-----------------------------------|
| room_category | string    | Room category booked by the guest |
| nightly_rate  | double    | Price charged per night           |
| nights        | integer   | Number of nights booked           |
| booking_date  | date      | Date when the booking was created |

## Sample Data – bookings.csv

| booking_id | guest_name  | room_category | nightly_rate | nights | booking_date |
|------------|-------------|---------------|--------------|--------|--------------|
| B1001      | Riya Sharma | Deluxe        | 6500.00      | 3      | 2025-09-02   |
| B1002      | Arjun Mehta | Deluxe        | 4500.00      | 2      | 2025-09-03   |
| B1003      | Neha Kapoor | Deluxe        | 3000.00      | 4      | 2025-09-04   |
| B1004      | Karan Singh | Suite         | 12000.00     | 2      | 2025-09-05   |
| B1005      | Ananya Rao  | Suite         | 8500.00      | 3      | 2025-09-07   |
| B1006      | Rahul Verma | Standard      | 2800.00      | 2      | 2025-09-08   |
| B1007      | Priya Nair  | Standard      | 3500.00      | 3      | 2025-09-09   |
| B1008      | Aman Shah   | Executive     | -5000.00     | 2      | 2025-09-10   |
| B1009      | Kavya Jain  | Standard      | 2500.00      | 0      | 2025-09-11   |

## File: room\_category\_meta.csv

## SCHEMA

| Column Name    | Data Type | Description                                      |
|----------------|-----------|--------------------------------------------------|
| room_category  | string    | Room category name                               |
| target_revenue | integer   | Per-booking revenue target for the room category |

## Sample Data – room\_category\_meta.csv

| room_category | target_revenue |
|---------------|----------------|
| Deluxe        | 10000          |
| Suite         | 20000          |
| Standard      | 7500           |
| Executive     | 15000          |

## TASK

## FUNCTIONS TO IMPLEMENT IN SOLUTION.PY

## Function 1 — load\_bookings(spark, path)

## PURPOSE

Load the hotel bookings dataset from a CSV file with an explicitly defined schema.

## PARAMETERS

spark (SparkSession) – Active Spark session passed in from main.py.

path (string) — File path to the bookings CSV file passed in from main.py.

## LOGIC

Define a schema with:

- booking\_id as string
- guest\_name as string
- room\_category as string
- nightly\_rate as double
- nights as integer
- booking\_date as date
- Read the CSV file with the header enabled and the schema applied so that the date and numeric values are parsed using the required data types.

## RETURNS

DataFrame with the six columns matching the defined schema.

## Function 2 — load\_room\_category\_metadata(spark, path)

### PURPOSE

Load the room category metadata dataset from a CSV file with an explicitly defined schema.

### PARAMETERS

spark (SparkSession) — Active Spark session passed in from main.py.

path (string) — File path to the room category metadata CSV file passed in from main.py.

### LOGIC

Define a schema with:

- room\_category as string
- target\_revenue as integer
- Read the CSV file with the header enabled and the schema applied.

## RETURNS

DataFrame with two columns matching the defined schema.

## Function 3 — clean\_and\_join\_bookings(df\_bookings, df\_meta)

### PURPOSE

Remove invalid booking records and enrich the surviving rows with the target revenue for their room category.

### PARAMETERS

df\_bookings (DataFrame) — Raw bookings DataFrame produced by load\_bookings.

df\_meta (DataFrame) — Room category metadata DataFrame produced by load\_room\_category\_metadata.

### LOGIC

Filter out rows where:

- booking\_id is null.
- nights is zero or less.
- nightly\_rate is zero or less.
- Perform an inner join with the metadata DataFrame on the room\_category column.
- Project the seven enriched columns.

## RETURNS

DataFrame with columns:

- booking\_id

- guest\_name
- room\_category
- nightly\_rate
- nights
- booking\_date
- target\_revenue

## Function 4 — compute\_revenue\_and\_status(df\_joined)

### PURPOSE

Derive booking revenue and classify each booking against its room-category target.

### PARAMETERS

df\_joined (DataFrame) — Cleaned and joined DataFrame produced by clean\_and\_join\_bookings.

### LOGIC

Add a revenue column computed as:

- $\text{nightly\_rate} \times \text{nights}$

Add a performance\_status column equal to:

- Met

when:

- $\text{revenue} \geq \text{target\_revenue}$

Otherwise:

- Below
- The boundary case where revenue exactly equals target\_revenue is treated as Met.
- Project the final six columns.

### RETURNS

DataFrame with columns:

- booking\_id
- guest\_name
- room\_category
- revenue
- target\_revenue
- performance\_status

## Function 5 — compute\_room\_category\_kpis(df\_enriched)

### PURPOSE

Aggregate per-room-category revenue performance KPIs from the enriched DataFrame.

### PARAMETERS

df\_enriched (DataFrame) — Enriched DataFrame produced by compute\_revenue\_and\_status, containing revenue and performance status per booking.

### LOGIC

Group records by room\_category and compute:

- total\_bookings as the count of bookings.
- met\_bookings as the count of bookings whose performance\_status is Met.

- achievement\_rate as met\_bookings divided by total\_bookings, rounded to two decimal places.
- avg\_revenue as the average booking revenue rounded to two decimal places.

## NOTE

Spark stores these values as doubles rounded to two decimals. On console display, trailing zeros may be trimmed.

## RETURNS

DataFrame with columns:

- room\_category
- total\_bookings
- met\_bookings
- achievement\_rate
- avg\_revenue

## Function 6 — top\_bookings\_per\_category(spark, df\_enriched)

### PURPOSE

Identify the highest-revenue booking(s) per room category using a Spark SQL temporary view and a window function.

### PARAMETERS

spark (SparkSession) — Active Spark session used to execute the SQL query.

df\_enriched (DataFrame) — Enriched DataFrame produced by compute\_revenue\_and\_status, containing revenue per booking.

### LOGIC

Register the enriched DataFrame as a temporary SQL view.

Run a Spark SQL query that applies:

### DENSE\_RANK()

partitioned by room\_category and ordered by revenue descending.

Filter to:

```
rank = 1
```

Ties are intentionally retained so multiple top-revenue bookings for the same room category appear together.

Order the final output by room\_category and guest\_name.

### RETURNS

DataFrame with columns:

- room\_category
- booking\_id
- guest\_name
- revenue
- SAMPLE INPUT/OUTPUT

## Function 1 — load\_bookings(spark, path)

### SAMPLE OUTPUT

| booking_id | guest_name  | room_category | nightly_rate | nights | booking_date |
|------------|-------------|---------------|--------------|--------|--------------|
| B1001      | Riya Sharma | Deluxe        | 6500.0       | 3      | 2025-09-02   |
| B1002      | Arjun Mehta | Deluxe        | 4500.0       | 2      | 2025-09-03   |
| B1003      | Neha Kapoor | Deluxe        | 3000.0       | 4      | 2025-09-04   |
| B1004      | Karan Singh | Suite         | 12000.0      | 2      | 2025-09-05   |
| B1005      | Ananya Rao  | Suite         | 8500.0       | 3      | 2025-09-07   |

## Function 2 — load\_room\_category\_metadata(spark, path)

### SAMPLE OUTPUT

| room_category | target_revenue |
|---------------|----------------|
| Deluxe        | 10000          |
| Suite         | 20000          |
| Standard      | 7500           |
| Executive     | 15000          |

## Function 3 — clean\_and\_join\_bookings(df\_bookings, df\_meta)

### SAMPLE INPUT

| booking_id | guest_name  | room_category | nightly_rate | nights | booking_date |
|------------|-------------|---------------|--------------|--------|--------------|
| B1001      | Riya Sharma | Deluxe        | 6500.00      | 3      | 2025-09-02   |
| null       | Arjun Mehta | Deluxe        | 4500.00      | 2      | 2025-09-03   |
| B1009      | Kavya Jain  | Standard      | 2500.00      | 0      | 2025-09-11   |
| B1008      | Aman Shah   | Executive     | -5000.00     | 2      | 2025-09-10   |
| B1004      | Karan Singh | Suite         | 12000.00     | 2      | 2025-09-05   |
| B1005      | Ananya Rao  | Suite         | 8500.00      | 3      | 2025-09-07   |

### SAMPLE OUTPUT

| booking_id | guest_name  | room_category | nightly_rate | nights | booking_date | target_revenue |
|------------|-------------|---------------|--------------|--------|--------------|----------------|
| B1001      | Riya Sharma | Deluxe        | 6500.00      | 3      | 2025-09-02   | 10000          |
| B1004      | Karan Singh | Suite         | 12000.00     | 2      | 2025-09-05   | 20000          |
| B1005      | Ananya Rao  | Suite         | 8500.00      | 3      | 2025-09-07   | 20000          |

## Function 4 — compute\_revenue\_and\_status(df\_joined)

### SAMPLE INPUT

| booking_id | guest_name  | room_category | nightly_rate | nights | target_revenue |
|------------|-------------|---------------|--------------|--------|----------------|
| B1001      | Riya Sharma | Deluxe        | 6500.00      | 3      | 10000          |
| B1002      | Arjun Mehta | Deluxe        | 4500.00      | 2      | 10000          |
| B1004      | Karan Singh | Suite         | 12000.00     | 2      | 20000          |
| B1005      | Ananya Rao  | Suite         | 8500.00      | 3      | 20000          |

### SAMPLE OUTPUT

| booking_id | guest_name  | room_category | revenue | target_revenue | performance_status |
|------------|-------------|---------------|---------|----------------|--------------------|
| B1001      | Riya Sharma | Deluxe        | 19500.0 | 10000          | Met                |

| booking_id | guest_name  | room_category | revenue | target_revenue | performance_status |
|------------|-------------|---------------|---------|----------------|--------------------|
| B1002      | Arjun Mehta | Deluxe        | 9000.0  | 10000          | Below              |
| B1004      | Karan Singh | Suite         | 24000.0 | 20000          | Met                |
| B1005      | Ananya Rao  | Suite         | 25500.0 | 20000          | Met                |

### Function 5 — compute\_room\_category\_kpis(df\_enriched)

#### SAMPLE INPUT

| booking_id | guest_name  | room_category | revenue | target_revenue | performance_status |
|------------|-------------|---------------|---------|----------------|--------------------|
| B1001      | Riya Sharma | Deluxe        | 19500.0 | 10000          | Met                |
| B1002      | Arjun Mehta | Deluxe        | 9000.0  | 10000          | Below              |
| B1003      | Neha Kapoor | Deluxe        | 12000.0 | 10000          | Met                |
| B1004      | Karan Singh | Suite         | 24000.0 | 20000          | Met                |
| B1005      | Ananya Rao  | Suite         | 25500.0 | 20000          | Met                |
| B1006      | Rahul Verma | Standard      | 5600.0  | 7500           | Below              |
| B1007      | Priya Nair  | Standard      | 10500.0 | 7500           | Met                |

#### SAMPLE OUTPUT

| room_category | total_bookings | met_bookings | achievement_rate | avg_revenue |
|---------------|----------------|--------------|------------------|-------------|
| Deluxe        | 3              | 2            | 0.67             | 13500.0     |
| Standard      | 2              | 1            | 0.50             | 8050.0      |
| Suite         | 2              | 2            | 1.00             | 24750.0     |

### Function 6 — top\_bookings\_per\_category(spark, df\_enriched)

#### SAMPLE INPUT

| booking_id | guest_name  | room_category | revenue |
|------------|-------------|---------------|---------|
| B1001      | Riya Sharma | Deluxe        | 19500.0 |
| B1002      | Arjun Mehta | Deluxe        | 9000.0  |
| B1010      | Neha Kapoor | Deluxe        | 19500.0 |
| B1004      | Karan Singh | Suite         | 24000.0 |
| B1005      | Ananya Rao  | Suite         | 25500.0 |
| B1006      | Rahul Verma | Standard      | 5600.0  |
| B1007      | Priya Nair  | Standard      | 10500.0 |

#### SAMPLE OUTPUT

| room_category | booking_id | guest_name  | revenue |
|---------------|------------|-------------|---------|
| Deluxe        | B1001      | Riya Sharma | 19500.0 |
| Deluxe        | B1010      | Neha Kapoor | 19500.0 |
| Standard      | B1007      | Priya Nair  | 10500.0 |
| Suite         | B1005      | Ananya Rao  | 25500.0 |

## Q13. City-Level Ride Service Analytics

Part 2 · PySpark — Medium — question 13 of 16

### PROBLEM STATEMENT

A multi-city ride-hailing platform is working to improve trip reliability and passenger experience. The operations team maintains two datasets — a trip-level feed containing booking details, trip timings, and driver information, and a city master table containing city, vehicle category, and expected trip-duration metadata.

They have asked the data engineering team to develop a clean, reproducible PySpark pipeline that loads the data, derives trip-performance indicators, generates driver-level reliability metrics, and produces city-level KPI reports that operations managers can review regularly.

### OBJECTIVE

- Load the trips and city master CSV files using the header as column names and infer the field types.
- Enrich every trip with a binary delayed-trip flag and the hour of day extracted from the booking timestamp.
- Summarise trip reliability for each driver.
- Join the enriched trips with the city master data to calculate city-level KPIs.
- Rank cities based on their delayed-trip rate.

### FILE STRUCTURE

```
Project
├── data
│   ├── trips.csv
│   └── cities.csv
├── main.py
├── solution.py
├── tests.py
└── requirements.txt
```

### PURPOSE OF FILES

- main.py – Driver script that creates the Spark session, executes the pipeline, displays intermediate DataFrames, and stops the Spark session.
- solution.py – Module to be completed by the candidate; contains the six required pipeline functions.
- data/trips.csv – Daily trip data used as the primary pipeline input.
- data/cities.csv – City master data used to enrich trips with city-level information.
- requirements.txt – Python dependencies required to execute the project.

### DATASET DESCRIPTION

Schema — data/trips.csv

| Column Name           | Data Type | Description                                       |
|-----------------------|-----------|---------------------------------------------------|
| trip_id               | int       | Unique trip identifier.                           |
| city_id               | int       | Foreign key referencing the city master table.    |
| driver_name           | string    | Driver assigned to the trip.                      |
| pickup_area           | string    | Area from which the passenger was picked up.      |
| booking_timestamp     | timestamp | Date and time when the trip was booked.           |
| promised_trip_minutes | int       | Expected trip duration promised to the passenger. |
| actual_trip_minutes   | int       | Actual duration taken to complete the trip.       |

| Column Name         | Data Type | Description                                                  |
|---------------------|-----------|--------------------------------------------------------------|
| passenger_rating    | int       | Passenger rating after trip completion, from 1 to 5.         |
| vehicle_driver_name | string    | Driver/vehicle operator responsible for completing the trip. |

To keep the task structurally equivalent to the original, `vehicle_driver_name` is used as the grouping field for driver-level analysis.

Sample Data — `data/trips.csv`

`trip_id,city_id,driver_name,pickup_area,booking_timestamp,promised_trip_minutes,actual_trip_minutes,passenger_rating,vehicle_driver_name`

```
6001,401,Metro Cab,Andheri,2025-05-10 08:15:00,30,36,3,RidePro
6002,402,City Wheels,Banjara Hills,2025-05-10 10:30:00,25,22,5,GoDrive
6003,405,Urban Ride,Connaught Place,2025-05-10 13:20:00,35,40,4,SwiftRide
6004,403,Rapid Cars,T Nagar,2025-05-10 18:10:00,40,44,3,RidePro
6005,402,City Wheels,Banjara Hills,2025-05-10 19:45:00,30,32,5,GoDrive
6006,404,Metro Cab,Koregaon Park,2025-05-10 20:15:00,35,38,4,SwiftRide
6007,405,Urban Ride,Karol Bagh,2025-05-11 11:10:00,30,30,5,SwiftRide
```

Schema — `data/cities.csv`

| Column Name           | Data Type | Description                                      |
|-----------------------|-----------|--------------------------------------------------|
| city_id               | int       | Primary key identifying the city.                |
| city_name             | string    | Name of the city.                                |
| operating_region      | string    | Region in which the city operates.               |
| vehicle_category      | string    | Main vehicle category available in the city.     |
| expected_trip_minutes | int       | Typical expected duration for trips in the city. |

Sample Data — `data/cities.csv`

```
city_id,city_name,operating_region,vehicle_category,expected_trip_minutes
401,Mumbai,West,Sedan,32
402,Hyderabad,South,SUV,28
403,Chennai,South,Hatchback,35
404,Pune,West,Sedan,30
405,Delhi,North,SUV,34
406,Bengaluru,South,Hatchback,31
```

## TASK

### IMPLEMENTATION REQUIREMENTS

- Implement the following six functions in `solution.py`.

#### Function 1 — `load_trips(spark, trips_csv_path)`

#### PURPOSE

Load the daily trip feed from CSV into a typed Spark DataFrame for downstream processing.

## PARAMETERS

`spark` – Active SparkSession supplied by `main.py`.

`trips_csv_path` – String containing the path to the trips CSV, for example `data/trips.csv`.

## REQUIREMENT

Load the trips such that:

- The first row provides column names.
- Numeric fields are inferred as integers.
- `booking_timestamp` is read as a timestamp.
- Other fields retain their appropriate inferred types.

## RETURNS

A Spark DataFrame containing the nine trip columns with types inferred from the CSV.

### Function 2 — `load_cities(spark, cities_csv_path)`

## PURPOSE

Load the city master data from CSV into a typed Spark DataFrame for use as the join dimension.

## PARAMETERS

`spark` – Active SparkSession.

`cities_csv_path` – String containing the path to the cities CSV, for example `data/cities.csv`.

## REQUIREMENT

Load the city master data such that:

- The first row provides the column names.
- Numeric fields are read as integers.
- Other fields retain their appropriate inferred types.

## RETURNS

A Spark DataFrame containing the five city columns.

### Function 3 — `derive_trip_flags(trips_df)`

## PURPOSE

Enrich trips with two derived performance features used by the KPI calculations.

## PARAMETERS

`trips_df` is the Spark DataFrame returned by `load_trips`, containing:

- `booking_timestamp`
- `promised_trip_minutes`
- `actual_trip_minutes`

## REQUIREMENT

Add the following two integer columns:

- 1. `is_delayed_trip`
- Value 1 when `actual_trip_minutes` is strictly greater than `promised_trip_minutes`.
- Value 0 otherwise.
- 2. `booking_hour_of_day`
- Extract the hour component from `booking_timestamp`.

- The value must range from 0 to 23.

## RETURNS

A Spark DataFrame containing the original nine columns plus:

- is\_delayed\_trip
- booking\_hour\_of\_day

## Function 4 — summarize\_by\_driver(trips\_df)

### PURPOSE

Summarise trip reliability for each driver/operator.

### PARAMETERS

trips\_df is the enriched DataFrame returned by derive\_trip\_flags.

It contains:

- vehicle\_driver\_name
- is\_delayed\_trip

### REQUIREMENT

For each vehicle\_driver\_name, calculate:

- Number of trips.
- Average delayed-trip rate.
- The average of is\_delayed\_trip must be rounded to three decimal places.

## RETURNS

A Spark DataFrame containing:

- vehicle\_driver\_name
- trips\_count
- delayed\_trip\_rate

## WHERE

trips\_count is a long/integer count.

delayed\_trip\_rate is a double rounded to three decimal places.

## Function 5 — kpis\_by\_city\_with\_join(trips\_df, cities\_df)

### PURPOSE

Produce city-level operational KPIs by joining enriched trips with the city master data.

### PARAMETERS

trips\_df – Enriched trips DataFrame returned by derive\_trip\_flags.

cities\_df – City master DataFrame returned by load\_cities.

### REQUIREMENT

Join the two DataFrames using:

- city\_id
- Use a join that keeps only matching records.

For each city\_name, calculate:

- Number of trips.

- Average actual\_trip\_minutes, rounded to two decimal places.
- Average is\_delayed\_trip, rounded to three decimal places.

## RETURNS

A Spark DataFrame containing:

- city\_name
- trips\_count
- avg\_trip\_minutes
- delayed\_trip\_rate

## WHERE

trips\_count is a long/integer count.

avg\_trip\_minutes is a double rounded to two decimal places.

delayed\_trip\_rate is a double rounded to three decimal places.

## Function 6 — rank\_cities\_by\_delay\_rate(city\_kpis\_df, top\_n)

### PURPOSE

Identify the cities having the highest delayed-trip rates for operational attention.

### PARAMETERS

city\_kpis\_df – Per-city KPI DataFrame returned by kpis\_by\_city\_with\_join.

top\_n – Integer specifying the number of cities to return.

### REQUIREMENT

Sort the cities:

- By delayed\_trip\_rate from highest to lowest.
- Break ties using city\_name in ascending alphabetical order.
- Return only the first top\_n rows.

## RETURNS

A Spark DataFrame containing the top top\_n cities with the same four columns as city\_kpis\_df.

## SAMPLE I/O

### Function 1 — load\_trips(spark, trips\_csv\_path)

#### SAMPLE INPUT

trip\_id,city\_id,driver\_name,pickup\_area,booking\_timestamp,promised\_trip\_minutes,actual\_trip\_minutes,passenger\_rating,vehicle\_driver\_name

7001,501,Metro Ride,Central,2025-04-10 09:00:00,30,35,4,DriveMax

7002,502,City Taxi,Lakeside,2025-04-10 18:30:00,25,23,5,GoCab

7003,501,Metro Ride,Central,2025-04-11 13:15:00,30,30,5,DriveMax

#### EXPECTED OUTPUT

trip\_id | city\_id | driver\_name | pickup\_area | booking\_timestamp | promised\_trip\_minutes | actual\_trip\_minutes | passenger\_rating | vehicle\_driver\_name

|      |     |            |         |                     |    |    |   |          |
|------|-----|------------|---------|---------------------|----|----|---|----------|
| 7001 | 501 | Metro Ride | Central | 2025-04-10 09:00:00 | 30 | 35 | 4 | DriveMax |
|------|-----|------------|---------|---------------------|----|----|---|----------|

|          |     |            |          |                     |    |    |   |  |
|----------|-----|------------|----------|---------------------|----|----|---|--|
| 7002     | 502 | City Taxi  | Lakeside | 2025-04-10 18:30:00 | 25 | 23 | 5 |  |
| GoCab    |     |            |          |                     |    |    |   |  |
| 7003     | 501 | Metro Ride | Central  | 2025-04-11 13:15:00 | 30 | 30 | 5 |  |
| DriveMax |     |            |          |                     |    |    |   |  |

### Function 2 — load\_cities(spark, cities\_csv\_path)

#### SAMPLE INPUT

city\_id,city\_name,operating\_region,vehicle\_category,expected\_trip\_minutes

501,Riverton,West,Sedan,26

502,Lakeside,North,SUV,29

#### EXPECTED OUTPUT

city\_id | city\_name | operating\_region | vehicle\_category | expected\_trip\_minutes

501 | Riverton | West | Sedan | 26

502 | Lakeside | North | SUV | 29

### Function 3 — derive\_trip\_flags(trips\_df)

#### SAMPLE INPUT

trip\_id | city\_id | booking\_timestamp | promised\_trip\_minutes | actual\_trip\_minutes

7001 | 501 | 2025-04-10 09:00:00 | 30 | 35

7002 | 502 | 2025-04-10 18:30:00 | 25 | 23

7003 | 501 | 2025-04-11 13:15:00 | 30 | 30

#### EXPECTED OUTPUT

trip\_id | city\_id | booking\_timestamp | promised\_trip\_minutes | actual\_trip\_minutes | is\_delayed\_trip | booking\_hour\_of\_day

7001 | 501 | 2025-04-10 09:00:00 | 30 | 35 | 1 | 9

7002 | 502 | 2025-04-10 18:30:00 | 25 | 23 | 0 | 18

7003 | 501 | 2025-04-11 13:15:00 | 30 | 30 | 0 | 13

### Function 4 — summarize\_by\_driver(trips\_df)

#### SAMPLE INPUT

vehicle\_driver\_name | is\_delayed\_trip

DriveMax | 1

DriveMax | 0

GoCab | 1

#### EXPECTED OUTPUT

vehicle\_driver\_name | trips\_count | delayed\_trip\_rate

DriveMax | 2 | 0.5

GoCab | 1 | 1.0

### Function 5 — kpis\_by\_city\_with\_join(trips\_df, cities\_df)

Sample Input — trips\_df

trip\_id | city\_id | actual\_trip\_minutes | is\_delayed\_trip

|      |     |    |   |
|------|-----|----|---|
| 7001 | 501 | 32 | 1 |
| 7002 | 501 | 28 | 0 |
| 7003 | 502 | 24 | 0 |

Sample Input — cities\_df

city\_id | city\_name | operating\_region

501 | Riverton | West

502 | Lakeside | North

#### EXPECTED OUTPUT

city\_name | trips\_count | avg\_trip\_minutes | delayed\_trip\_rate

Riverton | 2 | 30.0 | 0.5

Lakeside | 1 | 24.0 | 0.0

#### Function 6 — rank\_cities\_by\_delay\_rate(city\_kpis\_df, top\_n)

##### SAMPLE INPUT

city\_name | trips\_count | avg\_trip\_minutes | delayed\_trip\_rate

Riverton | 4 | 30.0 | 0.750

Lakeside | 3 | 31.0 | 0.500

Hillview | 3 | 32.0 | 0.667

##### INPUT

top\_n = 2

##### EXPECTED OUTPUT

city\_name | trips\_count | avg\_trip\_minutes | delayed\_trip\_rate

Riverton | 4 | 30.0 | 0.750

Hillview | 3 | 32.0 | 0.667

## Q14. Delivery Performance Analyzer

Part 2 · PySpark — Medium — question 14 of 16

### PROBLEM STATEMENT

A logistics company is monitoring its delivery network by analyzing Delivery Detail Records (DDR) collected from distribution hubs. Operations teams have reported an increase in failed deliveries around certain hubs, but the existing dashboard is too coarse-grained to quickly identify problematic locations. The data engineering team must develop a daily PySpark pipeline that ingests the raw DDR feed, standardizes the diagnostic columns, keeps only meaningful delivery outcomes, and produces per-partner and per-hub performance metrics for the Operations team to identify and prioritize problematic hubs.

### OBJECTIVE

- Ingest the raw DDR feed from CSV into Spark, taking column names from the header and inferring field types.
- Standardize the diagnostic columns by selecting and renaming them with clean aliases.
- Filter out non-diagnostic statuses, keeping only DELIVERED and FAILED deliveries.
- Summarize delivery reliability per partner, aggregate per-hub quality metrics, and rank the worst hubs by failure rate.

### Function 1 — load\_delivery\_data(path)

Implement:

```
def load_delivery_data(path):
```

### REQUIREMENT

Read the raw delivery CSV file into a PySpark DataFrame.

The CSV must be read with:

- Header enabled
- Schema inference enabled
- Return the resulting DataFrame.

### SAMPLE INPUT

distribution\_center,service\_provider,delivery\_duration,delivery\_result

HUB\_DEL\_001,FastTrack,120,DELIVERED

HUB\_DEL\_001,ShipNow,60,DELIVERED

HUB\_DEL\_002,ParcelGo,45,DELIVERED

HUB\_DEL\_003,FastTrack,75,DELIVERED

HUB\_DEL\_003,ShipNow,20,FAILED

HUB\_DEL\_004,QuickShip,50,DELIVERED

HUB\_DEL\_004,QuickShip,10,FAILED

HUB\_DEL\_001,QuickShip,30,FAILED

HUB\_DEL\_002,ShipNow,15,FAILED

HUB\_DEL\_003,QuickShip,0,FAILED

### SAMPLE OUTPUT

| distribution_center | service_provider | delivery_duration | delivery_result |
|---------------------|------------------|-------------------|-----------------|
| HUB_DEL_001         | FastTrack        | 120               | DELIVERED       |
| HUB_DEL_001         | ShipNow          | 60                | DELIVERED       |

| distribution_center | service_provider | delivery_duration | delivery_result |
|---------------------|------------------|-------------------|-----------------|
| HUB_DEL_002         | ParcelGo         | 45                | DELIVERED       |
| HUB_DEL_003         | FastTrack        | 75                | DELIVERED       |
| HUB_DEL_003         | ShipNow          | 20                | FAILED          |
| HUB_DEL_004         | QuickShip        | 50                | DELIVERED       |
| HUB_DEL_004         | QuickShip        | 10                | FAILED          |
| HUB_DEL_001         | QuickShip        | 30                | FAILED          |
| HUB_DEL_002         | ShipNow          | 15                | FAILED          |
| HUB_DEL_003         | QuickShip        | 0                 | FAILED          |

## Function 2 — standardize\_delivery\_columns(df)

Implement:

```
def standardize_delivery_columns(df):
```

### REQUIREMENT

Select and rename the required columns as follows:

| Original Column     | New Column      |
|---------------------|-----------------|
| distribution_center | hub_id          |
| service_provider    | partner         |
| delivery_duration   | duration_min    |
| delivery_result     | delivery_status |

Return only these four columns:

- hub\_id
- partner
- duration\_min
- delivery\_status

### SAMPLE INPUT

| distribution_center | service_provider | delivery_duration | delivery_result |
|---------------------|------------------|-------------------|-----------------|
| HUB_DEL_001         | FastTrack        | 120               | DELIVERED       |
| HUB_DEL_001         | ShipNow          | 60                | DELIVERED       |
| HUB_DEL_002         | ParcelGo         | 45                | DELIVERED       |
| HUB_DEL_003         | FastTrack        | 75                | DELIVERED       |
| HUB_DEL_003         | ShipNow          | 20                | FAILED          |
| HUB_DEL_004         | QuickShip        | 50                | DELIVERED       |
| HUB_DEL_004         | QuickShip        | 10                | FAILED          |
| HUB_DEL_001         | QuickShip        | 30                | FAILED          |
| HUB_DEL_002         | ShipNow          | 15                | FAILED          |
| HUB_DEL_003         | QuickShip        | 0                 | FAILED          |

### SAMPLE OUTPUT

| hub_id      | partner   | duration_min | delivery_status |
|-------------|-----------|--------------|-----------------|
| HUB_DEL_001 | FastTrack | 120          | DELIVERED       |
| HUB_DEL_001 | ShipNow   | 60           | DELIVERED       |

| hub_id      | partner   | duration_min | delivery_status |
|-------------|-----------|--------------|-----------------|
| HUB_DEL_002 | ParcelGo  | 45           | DELIVERED       |
| HUB_DEL_003 | FastTrack | 75           | DELIVERED       |
| HUB_DEL_003 | ShipNow   | 20           | FAILED          |
| HUB_DEL_004 | QuickShip | 50           | DELIVERED       |
| HUB_DEL_004 | QuickShip | 10           | FAILED          |
| HUB_DEL_001 | QuickShip | 30           | FAILED          |
| HUB_DEL_002 | ShipNow   | 15           | FAILED          |
| HUB_DEL_003 | QuickShip | 0            | FAILED          |

### Function 3 — filter\_delivery\_status(df)

Implement:

```
def filter_delivery_status(df):
```

#### REQUIREMENT

Filter the DataFrame and keep only records where delivery\_status is:

**DELIVERED**

**FAILED**

Any other status must be removed.

#### SAMPLE INPUT

| hub_id      | partner   | duration_min | delivery_status |
|-------------|-----------|--------------|-----------------|
| HUB_DEL_001 | FastTrack | 120          | DELIVERED       |
| HUB_DEL_001 | ShipNow   | 60           | DELIVERED       |
| HUB_DEL_002 | ParcelGo  | 45           | DELIVERED       |
| HUB_DEL_003 | FastTrack | 75           | DELIVERED       |
| HUB_DEL_003 | ShipNow   | 20           | FAILED          |
| HUB_DEL_004 | QuickShip | 50           | DELIVERED       |
| HUB_DEL_004 | QuickShip | 10           | FAILED          |
| HUB_DEL_001 | QuickShip | 30           | FAILED          |
| HUB_DEL_002 | ShipNow   | 15           | FAILED          |
| HUB_DEL_003 | QuickShip | 0            | FAILED          |
| HUB_DEL_002 | ParcelGo  | 25           | CANCELLED       |
| HUB_DEL_004 | FastTrack | 40           | PENDING         |

#### SAMPLE OUTPUT

| hub_id      | partner   | duration_min | delivery_status |
|-------------|-----------|--------------|-----------------|
| HUB_DEL_001 | FastTrack | 120          | DELIVERED       |
| HUB_DEL_001 | ShipNow   | 60           | DELIVERED       |
| HUB_DEL_002 | ParcelGo  | 45           | DELIVERED       |
| HUB_DEL_003 | FastTrack | 75           | DELIVERED       |
| HUB_DEL_003 | ShipNow   | 20           | FAILED          |
| HUB_DEL_004 | QuickShip | 50           | DELIVERED       |
| HUB_DEL_004 | QuickShip | 10           | FAILED          |
| HUB_DEL_001 | QuickShip | 30           | FAILED          |

| hub_id      | partner   | duration_min | delivery_status |
|-------------|-----------|--------------|-----------------|
| HUB_DEL_002 | ShipNow   | 15           | FAILED          |
| HUB_DEL_003 | QuickShip | 0            | FAILED          |

#### Function 4 — summarize\_by\_partner(df)

Implement:

```
def summarize_by_partner(df):
```

#### REQUIREMENT

Group the delivery records by partner and calculate:

- total\_deliveries → total number of deliveries
- failed\_deliveries → number of failed deliveries
- failure\_rate → failed\_deliveries / total\_deliveries

Return the following columns:

- partner
- total\_deliveries
- failed\_deliveries
- failure\_rate

#### SAMPLE INPUT

| hub_id      | partner   | duration_min | delivery_status |
|-------------|-----------|--------------|-----------------|
| HUB_DEL_001 | FastTrack | 120          | DELIVERED       |
| HUB_DEL_001 | ShipNow   | 60           | DELIVERED       |
| HUB_DEL_002 | ParcelGo  | 45           | DELIVERED       |
| HUB_DEL_003 | FastTrack | 75           | DELIVERED       |
| HUB_DEL_003 | ShipNow   | 20           | FAILED          |
| HUB_DEL_004 | QuickShip | 50           | DELIVERED       |
| HUB_DEL_004 | QuickShip | 10           | FAILED          |
| HUB_DEL_001 | QuickShip | 30           | FAILED          |
| HUB_DEL_002 | ShipNow   | 15           | FAILED          |
| HUB_DEL_003 | QuickShip | 0            | FAILED          |

#### SAMPLE OUTPUT

| partner   | total_deliveries | failed_deliveries | failure_rate       |
|-----------|------------------|-------------------|--------------------|
| FastTrack | 2                | 0                 | 0.0                |
| ParcelGo  | 1                | 0                 | 0.0                |
| QuickShip | 4                | 3                 | 0.75               |
| ShipNow   | 3                | 2                 | 0.6666666666666666 |

#### Function 5 — aggregate\_quality\_by\_hub(df)

Implement:

```
def aggregate_quality_by_hub(df):
```

#### REQUIREMENT

Group the records by hub\_id and calculate:

- total\_deliveries

- failed\_deliveries
- failure\_rate
- avg\_duration\_min

## WHERE

```
failure_rate = failed_deliveries / total_deliveries
```

and:

```
avg_duration_min = average(duration_min)
```

## RETURN

hub\_id

total\_deliveries

failed\_deliveries

failure\_rate

avg\_duration\_min

## SAMPLE INPUT

| hub_id      | partner   | duration_min | delivery_status |
|-------------|-----------|--------------|-----------------|
| HUB_DEL_001 | FastTrack | 120          | DELIVERED       |
| HUB_DEL_001 | ShipNow   | 60           | DELIVERED       |
| HUB_DEL_002 | ParcelGo  | 45           | DELIVERED       |
| HUB_DEL_003 | FastTrack | 75           | DELIVERED       |
| HUB_DEL_003 | ShipNow   | 20           | FAILED          |
| HUB_DEL_004 | QuickShip | 50           | DELIVERED       |
| HUB_DEL_004 | QuickShip | 10           | FAILED          |
| HUB_DEL_001 | QuickShip | 30           | FAILED          |
| HUB_DEL_002 | ShipNow   | 15           | FAILED          |
| HUB_DEL_003 | QuickShip | 0            | FAILED          |

## SAMPLE OUTPUT

| hub_id      | total_deliveries | failed_deliveries | failure_rate       | avg_duration_min   |
|-------------|------------------|-------------------|--------------------|--------------------|
| HUB_DEL_001 | 3                | 1                 | 0.3333333333333333 | 70.0               |
| HUB_DEL_002 | 2                | 1                 | 0.5                | 30.0               |
| HUB_DEL_003 | 3                | 2                 | 0.6666666666666666 | 31.666666666666668 |
| HUB_DEL_004 | 2                | 1                 | 0.5                | 30.0               |

## Function 6 — rank\_worst\_hubs(hub\_metrics\_df, top\_n)

Implement:

```
def rank_worst_hubs(hub_metrics_df, top_n):
```

## REQUIREMENT

Rank the hubs based on their failure rate.

Sort by:

- failure\_rate in descending order
- failed\_deliveries in descending order when the failure rates are equal
- Return only the first top\_n records.

## SAMPLE INPUT

hub\_metrics\_df

| hub_id      | total_deliveries | failed_deliveries | failure_rate       | avg_duration_min   |
|-------------|------------------|-------------------|--------------------|--------------------|
| HUB_DEL_001 | 3                | 1                 | 0.3333333333333333 | 70.0               |
| HUB_DEL_002 | 2                | 1                 | 0.5                | 30.0               |
| HUB_DEL_003 | 3                | 2                 | 0.6666666666666666 | 31.666666666666668 |
| HUB_DEL_004 | 2                | 1                 | 0.5                | 30.0               |

top\_n

3

## SAMPLE OUTPUT

| hub_id      | total_deliveries | failed_deliveries | failure_rate       | avg_duration_min   |
|-------------|------------------|-------------------|--------------------|--------------------|
| HUB_DEL_003 | 3                | 2                 | 0.6666666666666666 | 31.666666666666668 |
| HUB_DEL_002 | 2                | 1                 | 0.5                | 30.0               |
| HUB_DEL_004 | 2                | 1                 | 0.5                | 30.0               |

## Q15. Flight Reservation Cancellation Analytics

Part 2 · PySpark — Medium — question 15 of 16

### PROBLEM STATEMENT

A global airline operates flights across multiple cities and records every reservation through its centralized booking platform. The operations and revenue teams have noticed changing cancellation and no-show patterns that affect flight planning, crew allocation, seat availability, and seasonal promotions. They require an analytics pipeline that loads raw flight reservations and route master data, standardizes reservation and travel dates, enriches reservations with route information, summarizes cancellations for each route, and identifies the busiest travel months using Spark SQL.

The processed insights will help the airline optimize flight schedules, staffing, pricing, and promotional campaigns for upcoming travel seasons.

### OBJECTIVE

- Load raw flight reservation records and the route master dataset.
- Normalize reservation and travel dates to date type and derive a cancellation flag that covers both customer cancellations and no-shows.
- Enrich reservation records with route metadata using a left join so that reservations without a matching route are still preserved.
- Aggregate total and cancelled reservation counts at the origin-city and route level.
- Use Spark SQL to identify the busiest travel months by counting only active, non-cancelled reservations.
- Return all DataFrames with the required columns and ordering specified in each function.

### FILE STRUCTURE

```
Project
├── data
│   ├── reservations.csv
│   └── routes.csv
├── main.py
├── solution.py
├── tests.py
└── requirements.txt
```

### PURPOSE OF FILES

- main.py – Driver script that loads data, calls each solution function in sequence, and displays intermediate results.
- solution.py – Contains the six pipeline functions to be implemented by the candidate.
- data/reservations.csv – Raw flight reservation-level transactional records.
- data/routes.csv – Route master data containing route, origin, destination, and aircraft information.
- requirements.txt – Python dependencies required to execute the project.

### DATASET DESCRIPTION

Schema (data/reservations.csv)

| Column Name      | Data Type | Description                                            |
|------------------|-----------|--------------------------------------------------------|
| reservation_id   | string    | Unique reservation identifier                          |
| route_id         | string    | Foreign key linking the reservation to a route         |
| passenger_id     | string    | Identifier of the passenger                            |
| reservation_date | date      | Date on which the reservation was created (YYYY-MM-DD) |

| Column Name   | Data Type | Description                                        |
|---------------|-----------|----------------------------------------------------|
| travel_date   | date      | Planned flight travel date (YYYY-MM-DD)            |
| passengers    | integer   | Number of passengers in the reservation            |
| baggage_count | integer   | Number of checked bags                             |
| status        | string    | Reservation status (confirmed, cancelled, no_show) |
| fare_total    | double    | Total fare value of the reservation                |
| channel       | string    | Booking source (website, mobile_app, agent)        |

## Sample Data (data/reservations.csv)

| reservation_id | route_id | passenger_id | reservation_date | travel_date | passengers | baggage_count | status    | fare_total |
|----------------|----------|--------------|------------------|-------------|------------|---------------|-----------|------------|
| R1001          | RT01     | P001         | 2025-01-04       | 2025-02-12  | 1          | 1             | confirmed | 450.00     |
| R1002          | RT02     | P002         | 2025-01-10       | 2025-02-18  | 2          | 2             | confirmed | 920.00     |
| R1003          | RT05     | P003         | 2025-01-16       | 2025-02-25  | 1          | 1             | cancelled | 380.00     |
| R1006          | RT01     | P006         | 2025-02-11       | 2025-03-16  | 2          | 2             | no_show   | 780.00     |
| R1008          | RT04     | P008         | 2025-03-02       | 2025-04-08  | 1          | 1             | confirmed | 1250.00    |

## Schema (data/routes.csv)

| Column Name      | Data Type | Description                        |
|------------------|-----------|------------------------------------|
| route_id         | string    | Unique route identifier            |
| route_name       | string    | Display name of the flight route   |
| origin_city      | string    | City from which the flight departs |
| destination_city | string    | City where the flight arrives      |
| aircraft_type    | string    | Aircraft assigned to the route     |

## Sample Data (data/routes.csv)

| route_id | route_name         | origin_city | destination_city | aircraft_type |
|----------|--------------------|-------------|------------------|---------------|
| RT01     | New York - Toronto | New York    | Toronto          | Boeing 737    |
| RT02     | London - Paris     | London      | Paris            | Airbus A320   |
| RT03     | Dubai - Mumbai     | Dubai       | Mumbai           | Boeing 787    |
| RT04     | Sydney - Singapore | Sydney      | Singapore        | Airbus A350   |
| RT05     | Paris - Rome       | Paris       | Rome             | Airbus A320   |

## TASK

## IMPLEMENTATION REQUIREMENTS

## FUNCTIONS TO IMPLEMENT IN SOLUTION.PY

## FUNCTIONS TO IMPLEMENT

## Function 1 — load\_reservations(spark, path)

## PURPOSE

Load the raw flight reservation dataset from a CSV file.

## PARAMETERS

spark is the active SparkSession passed from main.py.

path is the string file path to reservations.csv.

## REQUIREMENT

From the reservation dataset, keep only the following ten required fields and discard any additional columns that may be present in the file:

reservation\_id

route\_id

passenger\_id

reservation\_date

travel\_date

passengers

baggage\_count

status

fare\_total

channel

## RETURNS

A DataFrame containing the ten required columns in exactly the order listed above.

### Function 2 — normalize\_reservations(df)

#### PURPOSE

Convert reservation and travel date fields to proper date types and derive a cancellation flag.

#### PARAMETERS

df is the DataFrame returned by load\_reservations.

#### REQUIREMENT

Convert reservation\_date and travel\_date into calendar date types.

Add a new boolean field called is\_cancelled.

is\_cancelled must be true when the reservation status is either:

- cancelled
- no\_show
- Otherwise, is\_cancelled must be false.

#### RETURNS

A DataFrame containing the original ten columns, with:

- reservation\_date converted to date
- travel\_date converted to date
- is\_cancelled appended as the final column
- Total: 11 columns

### Function 3 — load\_routes(spark, path)

#### PURPOSE

Load the airline route master dataset from a CSV file.

## PARAMETERS

spark is the active SparkSession passed from main.py.

path is the string file path to routes.csv.

## REQUIREMENT

Load the complete route master dataset. All five fields are required downstream, so none of them should be removed.

## RETURNS

A DataFrame containing these five route metadata columns:

- route\_id
- route\_name
- origin\_city
- destination\_city
- aircraft\_type

## Function 4 — join\_routes(df\_reservations, df\_routes)

### PURPOSE

Enrich every reservation with its corresponding route information while preserving reservations whose route\_id does not exist in the route master dataset.

### PARAMETERS

df\_reservations is the normalized reservation DataFrame returned by normalize\_reservations.

df\_routes is the route DataFrame returned by load\_routes.

### REQUIREMENT

Combine each reservation with its route metadata using route\_id.

The join must be a left join, so that a reservation without a matching route still appears in the output with empty route metadata fields.

The fields must be arranged in exactly this order:

- reservation\_id
- route\_id
- route\_name
- origin\_city
- destination\_city
- aircraft\_type
- passenger\_id
- reservation\_date
- travel\_date
- passengers
- baggage\_count
- status
- is\_cancelled
- fare\_total
- channel

## RETURNS

A DataFrame with exactly 15 columns in the order specified above.

### Function 5 — `aggregate_cancellations(df_enriched)`

#### PURPOSE

Calculate total and cancelled reservation counts for each combination of origin city and route.

#### PARAMETERS

`df_enriched` is the 15-column DataFrame returned by `join_routes`.

#### REQUIREMENT

For every combination of:

- `origin_city`
- `route_id`

#### CALCULATE

`total_reservations` – total number of reservations.

`cancelled_reservations` – number of reservations where `is_cancelled` is true.

#### RETURNS

A DataFrame containing exactly four columns:

- `origin_city`
- `route_id`
- `total_reservations`
- `cancelled_reservations`

### Function 6 — `busiest_travel_months_sql(df_enriched, spark)`

#### PURPOSE

Identify the busiest travel months based only on active, non-cancelled reservations.

#### PARAMETERS

`df_enriched` is the 15-column DataFrame returned by `join_routes`.

`spark` is the active `SparkSession` passed from `main.py`.

#### REQUIREMENT

Using Spark SQL:

- Register the enriched DataFrame as a temporary view.
- Extract the month number from `travel_date`.
- Consider only reservations where `is_cancelled` = false.
- Count the number of active reservations for each travel month.
- Sort the results from the most active month to the least active month.

The month number should be stored in a column called:

- `month_num`

The reservation count should be stored in:

- `active_reservations`

#### RETURNS

A DataFrame with exactly two columns:

- month\_num
- active\_reservations

## SAMPLE INPUT AND OUTPUT

### Function 1 — load\_reservations(spark, path)

#### SAMPLE INPUT

| reservation_id | route_id | passenger_id | reservation_date | travel_date | passengers | baggage_count | status    | fare_total |
|----------------|----------|--------------|------------------|-------------|------------|---------------|-----------|------------|
| R1001          | RT01     | P001         | 2025-01-04       | 2025-02-12  | 1          | 1             | confirmed | 450.00     |
| R1003          | RT05     | P003         | 2025-01-16       | 2025-02-25  | 1          | 1             | cancelled | 380.00     |
| R1006          | RT01     | P006         | 2025-02-11       | 2025-03-16  | 2          | 2             | no_show   | 780.00     |
| R1009          | RT02     | P009         | 2025-03-08       | 2025-04-14  | 1          | 1             | cancelled | 520.00     |
| R1010          | RT07     | P010         | 2025-03-14       | 2025-04-21  | 2          | 1             | confirmed | 1150.00    |
| R1012          | RT01     | P012         | 2025-04-02       | 2025-05-10  | 1          | 2             | confirmed | 640.00     |
| R1017          | RT02     | P017         | 2025-05-01       | 2025-06-07  | 1          | 1             | confirmed | 420.00     |
| R1020          | RT01     | P020         | 2025-05-18       | 2025-06-24  | 2          | 1             | confirmed | 890.00     |

#### SAMPLE OUTPUT

The output should contain the same ten required columns in the specified order:

| reservation_id | route_id | passenger_id | reservation_date | travel_date | passengers | baggage_count | status    | fare_total |
|----------------|----------|--------------|------------------|-------------|------------|---------------|-----------|------------|
| R1001          | RT01     | P001         | 2025-01-04       | 2025-02-12  | 1          | 1             | confirmed | 450.0      |
| R1003          | RT05     | P003         | 2025-01-16       | 2025-02-25  | 1          | 1             | cancelled | 380.0      |
| R1006          | RT01     | P006         | 2025-02-11       | 2025-03-16  | 2          | 2             | no_show   | 780.0      |
| R1009          | RT02     | P009         | 2025-03-08       | 2025-04-14  | 1          | 1             | cancelled | 520.0      |
| R1010          | RT07     | P010         | 2025-03-14       | 2025-04-21  | 2          | 1             | confirmed | 1150.0     |
| R1012          | RT01     | P012         | 2025-04-02       | 2025-05-10  | 1          | 2             | confirmed | 640.0      |
| R1017          | RT02     | P017         | 2025-05-01       | 2025-06-07  | 1          | 1             | confirmed | 420.0      |
| R1020          | RT01     | P020         | 2025-05-18       | 2025-06-24  | 2          | 1             | confirmed | 890.0      |

### Function 2 — normalize\_reservations(df)

#### SAMPLE OUTPUT

| reservation_id | route_id | passenger_id | reservation_date | travel_date | passengers | baggage_count | status    | fare_total |
|----------------|----------|--------------|------------------|-------------|------------|---------------|-----------|------------|
| R1001          | RT01     | P001         | 2025-01-04       | 2025-02-12  | 1          | 1             | confirmed | 450.0      |
| R1003          | RT05     | P003         | 2025-01-16       | 2025-02-25  | 1          | 1             | cancelled | 380.0      |
| R1006          | RT01     | P006         | 2025-02-11       | 2025-03-16  | 2          | 2             | no_show   | 780.0      |
| R1009          | RT02     | P009         | 2025-03-08       | 2025-04-14  | 1          | 1             | cancelled | 520.0      |
| R1010          | RT07     | P010         | 2025-03-14       | 2025-04-21  | 2          | 1             | confirmed | 1150.0     |
| R1012          | RT01     | P012         | 2025-04-02       | 2025-05-10  | 1          | 2             | confirmed | 640.0      |
| R1017          | RT02     | P017         | 2025-05-01       | 2025-06-07  | 1          | 1             | confirmed | 420.0      |
| R1020          | RT01     | P020         | 2025-05-18       | 2025-06-24  | 2          | 1             | confirmed | 890.0      |

### Function 3 — load\_routes(spark, path)

#### SAMPLE INPUT

| route_id | route_name         | origin_city | destination_city | aircraft_type |
|----------|--------------------|-------------|------------------|---------------|
| RT01     | New York - Toronto | New York    | Toronto          | Boeing 737    |
| RT02     | London - Paris     | London      | Paris            | Airbus A320   |
| RT05     | Paris - Rome       | Paris       | Rome             | Airbus A320   |
| RT07     | Dubai - Singapore  | Dubai       | Singapore        | Boeing 787    |

## SAMPLE OUTPUT

| route_id | route_name         | origin_city | destination_city | aircraft_type |
|----------|--------------------|-------------|------------------|---------------|
| RT01     | New York - Toronto | New York    | Toronto          | Boeing 737    |
| RT02     | London - Paris     | London      | Paris            | Airbus A320   |
| RT05     | Paris - Rome       | Paris       | Rome             | Airbus A320   |
| RT07     | Dubai - Singapore  | Dubai       | Singapore        | Boeing 787    |

## Function 4 — join\_routes(df\_reservations, df\_routes)

Sample Input 1 — Reservations

| reservation_id | route_id | passenger_id | reservation_date | travel_date | passengers | baggage_count | status    | fare_total |
|----------------|----------|--------------|------------------|-------------|------------|---------------|-----------|------------|
| R1001          | RT01     | P001         | 2025-01-04       | 2025-02-12  | 1          | 1             | confirmed | 450.0      |
| R1003          | RT05     | P003         | 2025-01-16       | 2025-02-25  | 1          | 1             | cancelled | 380.0      |
| R1006          | RT01     | P006         | 2025-02-11       | 2025-03-16  | 2          | 2             | no_show   | 780.0      |
| R1009          | RT02     | P009         | 2025-03-08       | 2025-04-14  | 1          | 1             | cancelled | 520.0      |
| R1010          | RT07     | P010         | 2025-03-14       | 2025-04-21  | 2          | 1             | confirmed | 1150.0     |
| R1012          | RT01     | P012         | 2025-04-02       | 2025-05-10  | 1          | 2             | confirmed | 640.0      |
| R1017          | RT02     | P017         | 2025-05-01       | 2025-06-07  | 1          | 1             | confirmed | 420.0      |
| R1020          | RT01     | P020         | 2025-05-18       | 2025-06-24  | 2          | 1             | confirmed | 890.0      |

Sample Input 2 — Routes

| route_id | route_name         | origin_city | destination_city | aircraft_type |
|----------|--------------------|-------------|------------------|---------------|
| RT01     | New York - Toronto | New York    | Toronto          | Boeing 737    |
| RT02     | London - Paris     | London      | Paris            | Airbus A320   |
| RT05     | Paris - Rome       | Paris       | Rome             | Airbus A320   |
| RT07     | Dubai - Singapore  | Dubai       | Singapore        | Boeing 787    |

## SAMPLE OUTPUT

| reservation_id | route_id | route_name         | origin_city | destination_city | aircraft_type | passenger_id | reservation_date | travel_date |
|----------------|----------|--------------------|-------------|------------------|---------------|--------------|------------------|-------------|
| R1001          | RT01     | New York - Toronto | New York    | Toronto          | Boeing 737    | P001         | 2025-01-04       | 2025-02-12  |
| R1003          | RT05     | Paris - Rome       | Paris       | Rome             | Airbus A320   | P003         | 2025-01-16       | 2025-02-25  |
| R1006          | RT01     | New York - Toronto | New York    | Toronto          | Boeing 737    | P006         | 2025-02-11       | 2025-03-16  |
| R1009          | RT02     | London - Paris     | London      | Paris            | Airbus A320   | P009         | 2025-03-08       | 2025-04-14  |
| R1010          | RT07     | Dubai - Singapore  | Dubai       | Singapore        | Boeing 787    | P010         | 2025-03-14       | 2025-04-21  |
| R1012          | RT01     | New York - Toronto | New York    | Toronto          | Boeing 737    | P012         | 2025-04-02       | 2025-05-10  |

| reservation_id | route_id | route_name         | origin_city | destination_city | aircraft_type | passenger_id | reservation_date | travel_date |
|----------------|----------|--------------------|-------------|------------------|---------------|--------------|------------------|-------------|
| R1017          | RT02     | London - Paris     | London      | Paris            | Airbus A320   | P017         | 2025-05-01       | 2025-06-07  |
| R1020          | RT01     | New York - Toronto | New York    | Toronto          | Boeing 737    | P020         | 2025-05-18       | 2025-06-04  |

**Note:** If a reservation contains a route\_id that does not exist in routes.csv, that reservation must still appear in the output because the required join is a left join. The route-related fields should be empty/null.

### Function 5 — aggregate\_cancellations(df\_enriched)

#### SAMPLE INPUT

| reservation_id | route_id | route_name         | origin_city | destination_city | aircraft_type | passenger_id | reservation_date | travel_date |
|----------------|----------|--------------------|-------------|------------------|---------------|--------------|------------------|-------------|
| R1001          | RT01     | New York - Toronto | New York    | Toronto          | Boeing 737    | P001         | 2025-01-04       | 2025-02-12  |
| R1003          | RT05     | Paris - Rome       | Paris       | Rome             | Airbus A320   | P003         | 2025-01-16       | 2025-02-25  |
| R1006          | RT01     | New York - Toronto | New York    | Toronto          | Boeing 737    | P006         | 2025-02-11       | 2025-03-16  |
| R1009          | RT02     | London - Paris     | London      | Paris            | Airbus A320   | P009         | 2025-03-08       | 2025-04-14  |
| R1010          | RT07     | Dubai - Singapore  | Dubai       | Singapore        | Boeing 787    | P010         | 2025-03-14       | 2025-04-21  |
| R1012          | RT01     | New York - Toronto | New York    | Toronto          | Boeing 737    | P012         | 2025-04-02       | 2025-05-10  |
| R1017          | RT02     | London - Paris     | London      | Paris            | Airbus A320   | P017         | 2025-05-01       | 2025-06-07  |
| R1020          | RT01     | New York - Toronto | New York    | Toronto          | Boeing 737    | P020         | 2025-05-18       | 2025-06-04  |

#### SAMPLE OUTPUT

| origin_city | route_id | total_reservations | cancelled_reservations |
|-------------|----------|--------------------|------------------------|
| Dubai       | RT07     | 1                  | 0                      |
| London      | RT02     | 2                  | 1                      |
| New York    | RT01     | 4                  | 1                      |
| Paris       | RT05     | 1                  | 1                      |

### Function 6 — busiest\_travel\_months\_sql(df\_enriched, spark)

#### SAMPLE INPUT

The function receives the enriched DataFrame containing reservation and route information.

For the sample data, the relevant travel dates are:

| reservation_id | travel_date | is_cancelled |
|----------------|-------------|--------------|
| R1001          | 2025-02-12  | false        |
| R1003          | 2025-02-25  | true         |
| R1006          | 2025-03-16  | true         |
| R1009          | 2025-04-14  | true         |
| R1010          | 2025-04-21  | false        |
| R1012          | 2025-05-10  | false        |
| R1017          | 2025-06-07  | false        |

| reservation_id | travel_date | is_cancelled |
|----------------|-------------|--------------|
| R1020          | 2025-06-24  | false        |

Only records where is\_cancelled = false should be counted.

#### SAMPLE OUTPUT

| month_num | active_reservations |
|-----------|---------------------|
| 6         | 2                   |
| 2         | 1                   |
| 4         | 1                   |
| 5         | 1                   |

## Q16. Employee Training Assessment Analysis

Part 2 · PySpark — Medium — question 16 of 16

### PROBLEM STATEMENT

A corporate learning platform conducts periodic technical assessments for employees across multiple training modules. The learning and quality team needs visibility into module-level performance to evaluate training effectiveness and compare overall performance at the individual employee level. They require a lightweight Spark pipeline that ingests two CSV inputs — raw assessment scores and module-wise passing criteria — joins them using the module key, projects only assessment-relevant columns, calculates pass and fail KPIs for each module, and produces an employee-level performance summary using a SQL step over a temporary view.

### OBJECTIVE

- Load the employee assessment scores and training module metadata CSV files.
- Join assessment records with module metadata using the module column to enrich each attempt with its applicable passing criteria.
- Project only the columns required for downstream performance KPI calculations.
- Compute module-level performance metrics: total participants, passed participants, and pass rate.
- Use Spark SQL to produce an employee-level performance summary containing modules attempted, average score, and highest score.

### FILE STRUCTURE

```
Project
├── data
│   ├── training_scores.csv
│   └── module_meta.csv
├── main.py
├── solution.py
├── tests.py
└── requirements.txt
```

### PURPOSE OF FILES

- main.py – Driver script that loads the data, calls each solution function in sequence, and prints intermediate results.
- solution.py – Contains the six pipeline functions to be implemented by the candidate.
- data/training\_scores.csv – Raw employee-level training assessment score records.
- data/module\_meta.csv – Module-level metadata containing maximum marks and minimum passing marks.
- requirements.txt – Python dependencies required to execute the project.

### DATASET DESCRIPTION

Schema (data/training\_scores.csv)

| Column Name     | Data Type | Description                                     |
|-----------------|-----------|-------------------------------------------------|
| employee_id     | string    | Unique employee identifier                      |
| employee_name   | string    | Full name of the employee                       |
| module          | string    | Training module name and join key with metadata |
| score           | integer   | Marks obtained in the module                    |
| assessment_date | date      | Date on which the assessment was conducted      |

Sample Data (data/training\_scores.csv)

| employee_id | employee_name | module | score | assessment_date |
|-------------|---------------|--------|-------|-----------------|
| EMP01       | Rahul Mehta   | Python | 86    | 2025-04-10      |
| EMP02       | Neha Sharma   | Python | 68    | 2025-04-10      |
| EMP06       | Karan Singh   | Python | 94    | 2025-04-10      |
| EMP01       | Rahul Mehta   | SQL    | 78    | 2025-04-11      |
| EMP01       | Rahul Mehta   | Cloud  | 91    | 2025-04-12      |

Schema (data/module\_meta.csv)

| Column Name      | Data Type | Description                          |
|------------------|-----------|--------------------------------------|
| module           | string    | Training module name and join key    |
| maximum_marks    | integer   | Maximum marks assigned to the module |
| qualifying_marks | integer   | Minimum marks required to pass       |

Sample Data (data/module\_meta.csv)

| module | maximum_marks | qualifying_marks |
|--------|---------------|------------------|
| Python | 100           | 50               |
| SQL    | 100           | 50               |
| Cloud  | 100           | 45               |
| DevOps | 100           | 50               |

## TASK

### IMPLEMENTATION REQUIREMENTS

#### FUNCTIONS TO IMPLEMENT IN SOLUTION.PY

##### Function 1 — load\_training\_scores(spark, scores\_path)

#### PURPOSE

Load the raw employee training assessment dataset from a CSV file.

#### PARAMETERS

spark is the active SparkSession passed from main.py.

scores\_path is the string file path to training\_scores.csv.

#### REQUIREMENT

Load the training assessment dataset. All of its fields are required by later stages, so none should be removed.

#### RETURNS

A DataFrame with these five columns:

- employee\_id
- employee\_name
- module
- score
- assessment\_date

##### Function 2 — load\_module\_metadata(spark, metadata\_path)

## PURPOSE

Load the module-wise assessment metadata from a CSV file.

## PARAMETERS

spark is the active SparkSession passed from main.py.

metadata\_path is the string file path to module\_meta.csv.

## REQUIREMENT

Load the module metadata dataset.

## RETURNS

A DataFrame with these three columns:

- module
- maximum\_marks
- qualifying\_marks

### Function 3 — join\_scores\_with\_modules(df\_scores, df\_modules)

## PURPOSE

Enrich every assessment record with the qualifying criteria applicable to its training module.

## PARAMETERS

df\_scores is the scores DataFrame returned by load\_training\_scores.

df\_modules is the metadata DataFrame returned by load\_module\_metadata.

## REQUIREMENT

Combine assessment records with their corresponding module metadata using module as the join key.

Only assessment records whose module exists in the metadata should be retained.

The join must therefore be an inner join.

The resulting DataFrame must contain all score fields along with the metadata fields.

The join key should appear first, giving the following seven columns:

- module
- employee\_id
- employee\_name
- score
- assessment\_date
- maximum\_marks
- qualifying\_marks
- Column ordering beyond these seven columns is not required to be asserted, provided all seven columns are present.

## RETURNS

A DataFrame containing the seven required columns.

### Function 4 — select\_performance\_columns(df\_joined)

## PURPOSE

Project only the columns required for calculating module-level performance KPIs.

## PARAMETERS

df\_joined is the joined DataFrame returned by join\_scores\_with\_modules.

## REQUIREMENT

Keep only the five required fields in exactly this order:

- employee\_id
- employee\_name
- module
- score
- qualifying\_marks

## RETURNS

A DataFrame with exactly five columns in the specified order.

### Function 5 — compute\_module\_metrics(df\_performance)

## PURPOSE

Calculate module-level pass and fail performance metrics.

## PARAMETERS

df\_performance is the DataFrame returned by select\_performance\_columns.

## REQUIREMENT

For every module, calculate:

- total\_participants – total number of employees who attempted the module.
- passed\_participants – number of employees whose score is greater than or equal to qualifying\_marks.
- pass\_rate – fraction of participants who passed.

## RETURNS

A DataFrame with exactly four columns:

- module
- total\_participants
- passed\_participants
- pass\_rate

### Function 6 — employee\_performance\_summary(spark, df\_performance)

## PURPOSE

Produce an employee-level performance summary across all attempted training modules using Spark SQL.

## PARAMETERS

spark is the active SparkSession passed from main.py.

df\_performance is the performance DataFrame returned by select\_performance\_columns.

## REQUIREMENT

Using Spark SQL, create a temporary view from the supplied DataFrame.

For each employee, identified by employee\_id and employee\_name, report:

- Number of modules attempted.
- Average assessment score rounded to two decimal places.

- Highest score achieved across all attempted modules.
- The result must be ordered by employee\_id.

## RETURNS

A DataFrame with exactly five columns:

- employee\_id
- employee\_name
- modules\_attempted
- average\_score
- highest\_score

## SAMPLE INPUT AND OUTPUT

### Function 1 — load\_training\_scores(spark, scores\_path)

#### SAMPLE INPUT

| employee_id | employee_name | module | score | assessment_date |
|-------------|---------------|--------|-------|-----------------|
| EMP01       | Rahul Mehta   | Python | 86    | 2025-04-10      |
| EMP01       | Rahul Mehta   | SQL    | 78    | 2025-04-11      |
| EMP01       | Rahul Mehta   | Cloud  | 91    | 2025-04-12      |
| EMP04       | Sneha Iyer    | Python | 44    | 2025-04-10      |
| EMP04       | Sneha Iyer    | SQL    | 64    | 2025-04-11      |
| EMP04       | Sneha Iyer    | Cloud  | 58    | 2025-04-12      |
| EMP09       | Vikram Kapoor | Python | 53    | 2025-04-10      |
| EMP09       | Vikram Kapoor | SQL    | 36    | 2025-04-11      |
| EMP09       | Vikram Kapoor | Cloud  | 46    | 2025-04-12      |

#### SAMPLE OUTPUT

| employee_id | employee_name | module | score | assessment_date |
|-------------|---------------|--------|-------|-----------------|
| EMP01       | Rahul Mehta   | Python | 86    | 2025-04-10      |
| EMP01       | Rahul Mehta   | SQL    | 78    | 2025-04-11      |
| EMP01       | Rahul Mehta   | Cloud  | 91    | 2025-04-12      |
| EMP04       | Sneha Iyer    | Python | 44    | 2025-04-10      |
| EMP04       | Sneha Iyer    | SQL    | 64    | 2025-04-11      |
| EMP04       | Sneha Iyer    | Cloud  | 58    | 2025-04-12      |
| EMP09       | Vikram Kapoor | Python | 53    | 2025-04-10      |
| EMP09       | Vikram Kapoor | SQL    | 36    | 2025-04-11      |
| EMP09       | Vikram Kapoor | Cloud  | 46    | 2025-04-12      |

### Function 2 — load\_module\_metadata(spark, metadata\_path)

#### SAMPLE INPUT

| module | maximum_marks | qualifying_marks |
|--------|---------------|------------------|
| Python | 100           | 50               |
| SQL    | 100           | 50               |
| Cloud  | 100           | 45               |

## SAMPLE OUTPUT

| module | maximum_marks | qualifying_marks |
|--------|---------------|------------------|
| Python | 100           | 50               |
| SQL    | 100           | 50               |
| Cloud  | 100           | 45               |

## Function 3 — join\_scores\_with\_modules(df\_scores, df\_modules)

Sample Input 1

| employee_id | employee_name | module | score | assessment_date |
|-------------|---------------|--------|-------|-----------------|
| EMP01       | Rahul Mehta   | Python | 86    | 2025-04-10      |
| EMP01       | Rahul Mehta   | SQL    | 78    | 2025-04-11      |
| EMP01       | Rahul Mehta   | Cloud  | 91    | 2025-04-12      |
| EMP04       | Sneha Iyer    | Python | 44    | 2025-04-10      |
| EMP04       | Sneha Iyer    | SQL    | 64    | 2025-04-11      |
| EMP04       | Sneha Iyer    | Cloud  | 58    | 2025-04-12      |
| EMP09       | Vikram Kapoor | Python | 53    | 2025-04-10      |
| EMP09       | Vikram Kapoor | SQL    | 36    | 2025-04-11      |
| EMP09       | Vikram Kapoor | Cloud  | 46    | 2025-04-12      |

Sample Input 2

| module | maximum_marks | qualifying_marks |
|--------|---------------|------------------|
| Python | 100           | 50               |
| SQL    | 100           | 50               |
| Cloud  | 100           | 45               |

## SAMPLE OUTPUT

| module | employee_id | employee_name | score | assessment_date | maximum_marks | qualifying_marks |
|--------|-------------|---------------|-------|-----------------|---------------|------------------|
| Python | EMP01       | Rahul Mehta   | 86    | 2025-04-10      | 100           | 50               |
| SQL    | EMP01       | Rahul Mehta   | 78    | 2025-04-11      | 100           | 50               |
| Cloud  | EMP01       | Rahul Mehta   | 91    | 2025-04-12      | 100           | 45               |
| Python | EMP04       | Sneha Iyer    | 44    | 2025-04-10      | 100           | 50               |
| SQL    | EMP04       | Sneha Iyer    | 64    | 2025-04-11      | 100           | 50               |
| Cloud  | EMP04       | Sneha Iyer    | 58    | 2025-04-12      | 100           | 45               |
| Python | EMP09       | Vikram Kapoor | 53    | 2025-04-10      | 100           | 50               |
| SQL    | EMP09       | Vikram Kapoor | 36    | 2025-04-11      | 100           | 50               |
| Cloud  | EMP09       | Vikram Kapoor | 46    | 2025-04-12      | 100           | 45               |

## Function 4 — select\_performance\_columns(df\_joined)

SAMPLE INPUT

| module | employee_id | employee_name | score | assessment_date | maximum_marks | qualifying_marks |
|--------|-------------|---------------|-------|-----------------|---------------|------------------|
| Python | EMP01       | Rahul Mehta   | 86    | 2025-04-10      | 100           | 50               |
| SQL    | EMP01       | Rahul Mehta   | 78    | 2025-04-11      | 100           | 50               |
| Cloud  | EMP01       | Rahul Mehta   | 91    | 2025-04-12      | 100           | 45               |
| Python | EMP04       | Sneha Iyer    | 44    | 2025-04-10      | 100           | 50               |

| module | employee_id | employee_name | score | assessment_date | maximum_marks | qualifying_marks |
|--------|-------------|---------------|-------|-----------------|---------------|------------------|
| SQL    | EMP04       | Sneha Iyer    | 64    | 2025-04-11      | 100           | 50               |
| Cloud  | EMP04       | Sneha Iyer    | 58    | 2025-04-12      | 100           | 45               |
| Python | EMP09       | Vikram Kapoor | 53    | 2025-04-10      | 100           | 50               |
| SQL    | EMP09       | Vikram Kapoor | 36    | 2025-04-11      | 100           | 50               |
| Cloud  | EMP09       | Vikram Kapoor | 46    | 2025-04-12      | 100           | 45               |

## SAMPLE OUTPUT

| employee_id | employee_name | module | score | qualifying_marks |
|-------------|---------------|--------|-------|------------------|
| EMP01       | Rahul Mehta   | Python | 86    | 50               |
| EMP01       | Rahul Mehta   | SQL    | 78    | 50               |
| EMP01       | Rahul Mehta   | Cloud  | 91    | 45               |
| EMP04       | Sneha Iyer    | Python | 44    | 50               |
| EMP04       | Sneha Iyer    | SQL    | 64    | 50               |
| EMP04       | Sneha Iyer    | Cloud  | 58    | 45               |
| EMP09       | Vikram Kapoor | Python | 53    | 50               |
| EMP09       | Vikram Kapoor | SQL    | 36    | 50               |
| EMP09       | Vikram Kapoor | Cloud  | 46    | 45               |

## Function 5 — compute\_module\_metrics(df\_performance)

### SAMPLE INPUT

| employee_id | employee_name | module | score | qualifying_marks |
|-------------|---------------|--------|-------|------------------|
| EMP01       | Rahul Mehta   | Python | 86    | 50               |
| EMP01       | Rahul Mehta   | SQL    | 78    | 50               |
| EMP01       | Rahul Mehta   | Cloud  | 91    | 45               |
| EMP04       | Sneha Iyer    | Python | 44    | 50               |
| EMP04       | Sneha Iyer    | SQL    | 64    | 50               |
| EMP04       | Sneha Iyer    | Cloud  | 58    | 45               |
| EMP09       | Vikram Kapoor | Python | 53    | 50               |
| EMP09       | Vikram Kapoor | SQL    | 36    | 50               |
| EMP09       | Vikram Kapoor | Cloud  | 46    | 45               |

### SAMPLE OUTPUT

| module | total_participants | passed_participants | pass_rate          |
|--------|--------------------|---------------------|--------------------|
| Cloud  | 3                  | 3                   | 1.0                |
| Python | 3                  | 2                   | 0.6666666666666666 |
| SQL    | 3                  | 2                   | 0.6666666666666666 |

Calculation logic

Python

Rahul:  $86 \geq 50 \rightarrow$  Pass

Sneha:  $44 < 50 \rightarrow$  Fail

Vikram:  $53 \geq 50 \rightarrow$  Pass

Therefore:

```
total_participants = 3
passed_participants = 2
pass_rate = 2 / 3
```

## SQL

Rahul:  $78 \geq 50 \rightarrow$  Pass

Sneha:  $64 \geq 50 \rightarrow$  Pass

Vikram:  $36 < 50 \rightarrow$  Fail

Therefore:

```
total_participants = 3
passed_participants = 2
pass_rate = 2 / 3
```

Cloud

Rahul:  $91 \geq 45 \rightarrow$  Pass

Sneha:  $58 \geq 45 \rightarrow$  Pass

Vikram:  $46 \geq 45 \rightarrow$  Pass

Therefore:

```
total_participants = 3
passed_participants = 3
pass_rate = 1.0
```

## Function 6 — employee\_performance\_summary(spark, df\_performance)

### SAMPLE INPUT

| employee_id | employee_name | module | score | qualifying_marks |
|-------------|---------------|--------|-------|------------------|
| EMP01       | Rahul Mehta   | Python | 86    | 50               |
| EMP01       | Rahul Mehta   | SQL    | 78    | 50               |
| EMP01       | Rahul Mehta   | Cloud  | 91    | 45               |
| EMP04       | Sneha Iyer    | Python | 44    | 50               |
| EMP04       | Sneha Iyer    | SQL    | 64    | 50               |
| EMP04       | Sneha Iyer    | Cloud  | 58    | 45               |
| EMP09       | Vikram Kapoor | Python | 53    | 50               |
| EMP09       | Vikram Kapoor | SQL    | 36    | 50               |
| EMP09       | Vikram Kapoor | Cloud  | 46    | 45               |

### SAMPLE OUTPUT

| employee_id | employee_name | modules_attempted | average_score | highest_score |
|-------------|---------------|-------------------|---------------|---------------|
| EMP01       | Rahul Mehta   | 3                 | 85.0          | 91            |
| EMP04       | Sneha Iyer    | 3                 | 55.33         | 64            |
| EMP09       | Vikram Kapoor | 3                 | 45.0          | 53            |